

The Book of Kahuna

Kahuna: Distributed Coordination
for Working Developers



Kahuna

Distributed Coordination for Working Developers

Andres Gutierrez

2025

Contents

1	Preface	8
2	What Is Kahuna?	9
2.1	The Coordination Problem	9
2.2	What Kahuna Is	10
2.3	The Three Primitives	10
2.4	When to Use Kahuna	12
2.5	When Not to Use Kahuna	13
2.6	How Kahuna Compares	13
2.7	Architecture at a Glance	15
2.8	A First Look at the Client SDK	16
2.9	What's Ahead	17
3	Getting Started	19
3.1	Prerequisites	19
3.2	Installing the Server	19
3.3	Installing the Command-Line Client	20
3.4	First Operations with the CLI	20
3.5	Running a Three-Node Cluster	21
3.6	Connecting from C#	23
3.7	A Complete Example	25
3.8	Your First Failure Experiment	26
3.9	Summary	28
4	Key-Value Operations	29
4.1	Keys and Values	29
4.2	Revisions	29
4.3	Conditional Writes	30
4.4	Time-to-Live (TTL)	31
4.5	Durability Levels	32
4.6	Deleting Keys	32
4.7	Checking Existence	33
4.8	No-Revision Writes	33
4.9	Batch Operations	33
4.10	Prefix Queries	34
4.11	Range Scans	35
4.12	Snapshot Reads	36
4.13	The KahunaKeyValue Object	36
4.14	CLI Reference	37
4.15	Putting It Together	38
4.16	Summary	39
5	Distributed Locking	40
5.1	Why Locks Need Leases	40
5.2	Acquiring a Lock	40
5.3	Fencing Tokens	41
5.4	Waiting for a Lock	43
5.5	Extending a Lease	44
5.6	What Happens When a Lock Holder Crashes	45

5.7	Unlocking	45
5.8	Querying Lock State	46
5.9	Ephemeral Locks	46
5.10	CLI Lock Operations	46
5.11	Common Patterns	47
5.12	Summary	48
6	The Distributed Sequencer	49
6.1	Creating a Sequence	49
6.2	Getting the Next Value	49
6.3	Reserving a Range	49
6.4	How Block Allocation Works	50
6.5	Increment	51
6.6	Maximum Value	51
6.7	Idempotent Allocation	52
6.8	Querying a Sequence	53
6.9	Deleting a Sequence	53
6.10	CLI Commands	53
6.11	Server Configuration	54
6.12	Use Cases	54
6.13	Guarantees and Non-Guarantees	55
6.14	Summary	55
7	Transactions	56
7.1	Script Transactions	56
7.2	Interactive Transactions	58
7.3	Locking Modes	61
7.4	Read Validation and Write-Skew Detection	62
7.5	RetryableTransaction	62
7.6	Transaction Options Reference	64
7.7	Common Patterns	65
7.8	Summary	67
8	The Script Language	68
8.1	Language Overview	68
8.2	Data Types	68
8.3	Variables	68
8.4	Key-Value Commands	69
8.5	Placeholders	71
8.6	Expressions	72
8.7	Control Flow	73
8.8	Transactions in Scripts	74
8.9	RETURN	75
8.10	SLEEP	75
8.11	THROW	76
8.12	Built-in Functions	76
8.13	Complete Example: Balance Transfer	79
8.14	Complete Example: Batch Initialization	79
8.15	Complete Example: Snapshot Audit	80

8.16	Script Caching	80
8.17	Reserved Words	80
8.18	Operator Precedence	80
8.19	Summary	81
9	Leader Election and Job Coordination	82
9.1	The Simplest Leader Election	82
9.2	Heartbeat Renewal	83
9.3	The Leader Election Loop	84
9.4	Why Fencing Tokens Matter	86
9.5	Publishing Leader Identity	88
9.6	Distributed Task Queue	89
9.7	Failure Scenarios	92
9.8	Tuning Leader Election	92
9.9	Combining Leader Election with Task Queues	93
9.10	Summary	94
10	Idempotency and Duplicate Prevention	95
10.1	The Retry Problem	95
10.2	Delivery Semantics	95
10.3	Naturally Idempotent Operations	95
10.4	Non-Idempotent Operations	96
10.5	Revision-Based Idempotency (CRAS)	96
10.6	Value-Based Idempotency (CVAS)	97
10.7	SetIfNotExists as an Idempotency Guard	97
10.8	Fencing-Based Exactly-Once Writes	99
10.9	Sequence-Based Deduplication	100
10.10	Transaction Operation IDs	101
10.11	Building an Idempotent Pipeline	101
10.12	Choosing the Right Mechanism	103
10.13	Common Mistakes	104
10.14	Summary	104
11	Globally Unique Identifiers	106
11.1	The ID Generation Problem	106
11.2	Common ID Schemes	106
11.3	Comparison Table	108
11.4	Patterns for ID Generation	108
11.5	Tuning Block Size	111
11.6	Failure Scenarios	113
11.7	When Not to Use Kahuna Sequences	113
11.8	A Complete Example	114
11.9	Summary	115
12	Coordinating Services	116
12.1	Distributed Configuration	116
12.2	Service Presence Detection	118
12.3	Session Management	121
12.4	Game Lobby Coordination	123
12.5	Failure Scenarios	126

12.6	Choosing Between Persistent and Ephemeral	127
12.7	Summary	127
13	Transactional Workflows	128
13.1	The Problem with Separate Writes	128
13.2	Single-Transaction Workflows	128
13.3	Choosing a Locking Mode	130
13.4	Multi-Transaction Workflows	131
13.5	Transaction Boundaries	133
13.6	Handling Conflicts	134
13.7	Script Transactions for Complex Logic	135
13.8	Failure Scenarios	137
13.9	Workflow Design Checklist	137
13.10	Summary	138
14	Handling Failures and Retries	139
14.1	Error Classification	139
14.2	The Error Decision Table	140
14.3	Retry Strategies	140
14.4	Timeout Configuration	142
14.5	Failure Scenarios	143
14.6	Application-Level Circuit Breaker	145
14.7	Error Handling Checklist	146
14.8	Summary	147
15	Architecture Overview	148
15.1	Project Layout	148
15.2	External Libraries	148
15.3	The IKahuna Facade	149
15.4	KahunaManager	149
15.5	The Shared Pattern: Manager, Locator, Actor	150
15.6	Inter-Node Communication	151
15.7	Persistence Backends	152
15.8	The Meta Partition	152
15.9	Client Communication	152
15.10	Summary	153
16	The Request Lifecycle	154
16.1	The Write Path: Setting a Key	154
16.2	The Read Path: Getting a Key	157
16.3	Read Path vs Write Path	158
16.4	Synchronous vs Asynchronous Boundaries	159
16.5	Where Latency Comes From	159
16.6	Failure Scenarios	160
16.7	Summary	161
17	Partitioning and Data Distribution	162
17.1	Two Routing Modes	162
17.2	The Range Map	163
17.3	The Meta Partition	165
17.4	Range Splits	165

17.5	Range Merges	167
17.6	The Generation Fence	168
17.7	State Transfer During Splits	168
17.8	Prefix Operations and Splits	169
17.9	Server Configuration	169
17.10	Failure Scenarios	170
17.11	Summary	171
18	Consensus and Replication	172
18.1	The Consensus Problem	172
18.2	Kommander: The Raft Library	172
18.3	Per-Partition Raft Groups	172
18.4	Leader Election	173
18.5	Log Entry Types	175
18.6	The Replication Pipeline	176
18.7	Linearizable Reads	177
18.8	Hybrid Logical Clocks	178
18.9	Voter and Learner Roles	179
18.10	Replication Factor	180
18.11	State Transfer	180
18.12	WAL Compaction	181
18.13	Failure Scenarios	181
18.14	Summary	182
19	The Actor Model and Concurrency	184
19.1	The Actor Model	184
19.2	Nixie: The Actor Framework	184
19.3	Consistent-Hash Routing	186
19.4	Kahuna's Actor Hierarchy	186
19.5	KeyValueActor	187
19.6	Ephemeral vs Persistent Actors	189
19.7	LockActor	189
19.8	SequenceActor	189
19.9	PartitionWriteAggregatorActor	190
19.10	BackgroundWriterActor	190
19.11	The Single-Threaded Invariant	190
19.12	Failure Scenarios	191
19.13	Summary	191
20	MVCC and Snapshot Isolation	193
20.1	The Version Chain	193
20.2	Write Intents	195
20.3	MVCC Entries: Read-Your-Writes	196
20.4	Snapshot Reads	196
20.5	The Snapshot Floor	197
20.6	Revision Pruning	198
20.7	Safe Timestamp	199
20.8	Prepared Intents and MVCC	199
20.9	Failure Scenarios	200

20.10	Summary	200
21	Transaction Internals	202
21.1	Transaction Sessions	202
21.2	Admission Control	202
21.3	The Two-Phase Commit Protocol	203
21.4	Durable Transaction Finalization	203
21.5	Deferred Settlement	205
21.6	Conflict Detection	206
21.7	Script Transactions	206
21.8	Transaction Record Store	206
21.9	Completion Receipts	207
21.10	Session Reaping	207
21.11	Transaction Recovery	208
21.12	Outcome Contract	208
21.13	Failure Scenarios	209
21.14	Summary	210
22	Lock and Sequencer Internals	211
22.1	Lock Entry	211
22.2	The Lock Pipeline	211
22.3	Fencing Token Monotonicity	213
22.4	Lock Replication	214
22.5	Lock Restoration	214
22.6	The Unflushed Writes Overlay	214
22.7	Lock Collection	215
22.8	Sequence Internals	215
22.9	Failure Scenarios	219
22.10	Summary	220
23	Persistence and Storage Engines	222
23.1	The Persistence Interface	222
23.2	The Memory Backend	223
23.3	The SQLite Backend	223
23.4	The RocksDB Backend	224
23.5	The Background Writer	226
23.6	The Unflushed Overlay	227
23.7	Flush Notification	228
23.8	IO Scheduling and Backpressure	229
23.9	Cache Eviction	229
23.10	Failure Scenarios	230

1 Preface

To be written after the main chapters are complete.

2 What Is Kahuna?

Imagine you are running three copies of a payment service behind a load balancer. A customer submits an order. The request lands on node A, which checks the customer's balance, deducts the amount, and records the charge. But what happens if the same request, retried after a timeout, lands on node B? Node B checks the balance, sees the old value (the deduction hasn't propagated yet), and charges the customer a second time.

This is not a hypothetical scenario. It is one of the most common bugs in distributed systems: two processes operating on shared state without coordination. The fix is not more careful coding. The fix is a coordination primitive (a lock, a conditional write, or an atomic transaction) that both processes agree to respect.

Building those coordination primitives yourself is hard. Getting them right under network partitions, process crashes, and clock drift is harder still. This is the problem that Kahuna solves.

2.1 The Coordination Problem

When an application runs on a single machine, coordination is straightforward. A thread acquires a mutex, does its work, and releases the mutex. The operating system guarantees that only one thread holds the mutex at a time. The application doesn't need to think about what happens if the CPU loses power mid-operation, because the OS handles that, too.

Distributed systems don't have a single machine. They have multiple processes running on multiple nodes, connected by a network that can delay, reorder, or drop messages. There is no shared memory. There is no global clock. There is no single entity that can decide "this process goes first."

And yet, distributed applications need all of the things that a single-machine application takes for granted:

- **Mutual exclusion.** Only one worker should process a given job at a time.
- **Consistent state.** If two clients update the same key, every subsequent read should see the same value, regardless of which node handles the read.
- **Atomic updates.** When a transaction writes to three keys, either all three writes take effect or none of them do.
- **Ordered identifiers.** Order numbers, event IDs, and log sequence numbers need to be unique and monotonically increasing across the entire cluster.

Each of these requirements is a coordination problem. Solving any one of them requires agreement among multiple nodes. Solving them correctly, under failures, requires a consensus protocol: a formal mechanism by which a group of nodes agrees on a single value or a single order of operations, even when some nodes are unreachable.

Consensus protocols are well-studied. Raft, Paxos, and Zab have been implemented in production systems for over a decade. But implementing a consensus protocol is not the same as having a coordination system. A coordination system wraps the protocol in a usable API: locks you can acquire, keys you can read and write, transactions you can commit, and sequences you can allocate from.

That is what Kahuna provides.

2.2 What Kahuna Is

Kahuna is an open-source distributed coordination system built in C# on .NET. It provides three core primitives:

1. **A distributed key-value store.** Store and retrieve data across a cluster of nodes, with strong consistency, multi-version concurrency control (MVCC), and support for transactions.
2. **Distributed locks.** Acquire time-bounded locks (leases) on named resources, with monotonically increasing fencing tokens that prevent stale lock holders from corrupting data.
3. **A distributed sequencer.** Generate globally unique, monotonically increasing numbers without a single-node bottleneck.

These three primitives share a single cluster, a single consensus layer (Raft), and a single operational surface. You deploy one system instead of three.

Kahuna organizes data into partitions. Each partition is an independent unit with its own Raft group: its own leader, its own replicas, and its own log. This design allows Kahuna to scale horizontally: more partitions means more throughput, because different partitions can be led by different nodes.

Every write in Kahuna goes through Raft consensus. This means that once a write is acknowledged, it is durable on a majority of replicas. If a node fails, another node can take over as leader for that partition without losing committed data.

Kahuna is a Hawaiian word that refers to an expert in any field. Historically, it has been used to refer to doctors, surgeons, priests, and sorcerers.

2.3 The Three Primitives

2.3.1 Key-Value Storage

At its core, Kahuna is a distributed key-value store. You write a value under a string key, and you can read it back from any node in the cluster. Kahuna routes the key to the correct partition, forwards the request to the partition's leader, and replicates the write through Raft before acknowledging it.

But Kahuna's key-value store goes well beyond simple get and set. It provides:

- **Revisions.** Every write to a key increments a revision number. You can read the current revision, and you can use it for compare-and-swap operations: "set this key to X, but only if its current revision is 7." If another writer changed the key in the meantime, the operation fails instead of silently overwriting.
- **Conditional writes.** Beyond compare-and-swap on revisions, you can set a key only if it doesn't exist yet, only if it already exists, or only if its current value matches a specific value.
- **Time-to-live (TTL).** Keys can expire after a specified duration. This is useful for session tokens, ephemeral state, and presence detection.
- **Durability levels.** Keys can be *persistent* (survive restarts, replicated via Raft) or *ephemeral* (fast, in-memory only, lost on restart). You choose per key.

- **Range queries.** You can scan keys by prefix or by range, with pagination support for large result sets.
- **Transactions.** You can read and write multiple keys atomically, with either optimistic or pessimistic concurrency control.

Here is what a simple key-value operation looks like in C#:

```
var client = new KahunaClient("http://localhost:2070");

// Set a key
await client.SetKeyValue("user:1001:name", "Alice");

// Get a key
KahunaKeyValue result = await client.GetKeyValue("user:1001:name");
Console.WriteLine(result.ValueAsString()); // "Alice"
Console.WriteLine(result.Revision);        // 1
```

The key `user:1001:name` is routed to a partition based on its prefix. The write goes through Raft consensus. The read goes to the partition's leader, which confirms it is still the leader before responding.

2.3.2 Distributed Locks

A distributed lock is a lease: a time-bounded claim on a named resource. While a process holds the lock, no other process can acquire it. When the lease expires (because the holder crashed, or because it simply didn't renew in time), the lock becomes available to other processes.

Leases alone are not enough, though. Consider this scenario:

1. Process A acquires a lock with a 10-second lease.
2. Process A starts a long operation.
3. The lease expires because the operation takes longer than expected.
4. Process B acquires the lock.
5. Process A finishes the operation and writes its result, but it no longer holds the lock.

Process A just wrote to a shared resource without holding the lock. This is the *stale lock holder* problem, and it can corrupt data even when the lock implementation itself is correct.

Kahuna prevents this with **fencing tokens**. Every time a lock is acquired, Kahuna assigns it a monotonically increasing integer. Process A might get fencing token 5. When the lease expires and process B acquires the lock, process B gets fencing token 6. If the downstream system checks the fencing token before accepting a write, it can reject process A's stale write (token 5) because it has already seen a higher token (6).

```
// Acquire a lock with a 10-second lease
await using KahunaLock lockHandle =
    await client.GetOrCreateLock(
        "resource:payment:1001",
        expiresMs: 10000);

if (lockHandle.IsAcquired)
{
```

```

        Console.WriteLine($"Lock acquired, fencing token:
{lockHandle.FencingToken}");
        // Do the protected work
    }
    // Lock is released automatically when lockHandle is disposed

```

The `await using` pattern ensures the lock is released even if the code throws an exception. If the process crashes instead of throwing, the lease expires and the lock becomes available.

2.3.3 Distributed Sequencer

Many applications need unique identifiers: order numbers, event IDs, invoice numbers, log sequence numbers. These identifiers must be:

- **Unique** across the entire cluster.
- **Monotonically increasing**. A higher number always means a later event.
- **Available without a single-node bottleneck**. The system should not funnel all ID generation through one machine.

Database auto-increment columns satisfy the first two requirements, but not the third. UUIDs satisfy the first and third, but not the second (they are not sortable by generation time in their standard form). Snowflake IDs satisfy all three, but require careful clock management and machine ID assignment.

Kahuna's sequencer takes a different approach. It uses **block-based allocation**: a node reserves a block of sequence values (for example, 1000 at a time) with a single Raft commit. It then serves individual values from that block without any further consensus operations. When the block is exhausted, it reserves another one.

This design means that one Raft commit can produce hundreds or thousands of unique values. The trade-off is **gaps**: if a node crashes with 300 values remaining in its block, those 300 values are never used. The sequence jumps from wherever it was to the next block boundary. Values are guaranteed to be unique, but they are not guaranteed to be gap-free.

```

// Create a sequence
await client.CreateSequence("order-ids");

// Allocate the next value
KahunaSequence seq = await client.NextSequenceValue("order-ids");
Console.WriteLine($"Order ID: {seq.CurrentValue}"); // 1

// Allocate a batch of values
KahunaSequenceRange range = await client.ReserveSequenceRange("order-ids", 100);
Console.WriteLine($"Range: {range.Start} to {range.End}"); // e.g. 2 to 101

```

For most applications, gaps are acceptable. If you need gap-free sequences (for example, invoice numbers in jurisdictions that require them), you can set the block size to 1, which forces a Raft commit per value. This eliminates gaps but reduces throughput.

2.4 When to Use Kahuna

Kahuna is designed for **coordination workloads**: operations where multiple processes need to agree on shared state. Here are the scenarios where it fits well:

Distributed locking. Protecting shared resources across services: job processing queues, rate limiters, payment flows, leader election, and any workflow where “only one at a time” is a correctness requirement.

Coordination metadata. Storing configuration that multiple services read and update: feature flags, service registrations, routing tables, cluster membership records.

Transactional updates to small key sets. Atomically updating a handful of related keys: account balances, inventory counts, state machines. Kahuna’s transactions are designed for small working sets (tens of keys), not bulk data operations.

Unique ID generation. Producing globally unique, monotonically increasing identifiers without a centralized bottleneck.

Session and presence management. Tracking which services or users are currently active, using ephemeral keys with TTL.

2.5 When Not to Use Kahuna

Kahuna is not a general-purpose database, and using it as one will lead to poor results.

Bulk data storage. If you need to store millions of rows with complex queries, use a database (PostgreSQL, CockroachDB, CamusDB). Kahuna stores coordination metadata and small working sets, not application data.

Message queuing. If you need publish-subscribe, message routing, or stream processing, use a message broker (Kafka, RabbitMQ, NATS). Kahuna can coordinate producers and consumers, but it is not a message transport.

High-throughput caching. For large-scale cache workloads with millions of reads per second, use Redis or Memcached. Kahuna can serve as a cache for low-to-medium workloads (it supports TTL, ephemeral durability, and no-revision writes), but its strong consistency and Raft consensus path add overhead that dedicated cache systems avoid.

Analytics and reporting. If you need aggregations, joins, or full-text search, use a database or search engine. Kahuna’s query model is key-based, not relational.

The simplest rule: if the data is *coordination state* (who holds the lock, what is the current sequence number, what is the latest configuration version), Kahuna is a good fit. If the data is *application state* (user profiles, product catalogs, order histories), use a database.

2.6 How Kahuna Compares

Several established systems provide overlapping capabilities. Understanding the differences helps you choose the right tool.

2.6.1 etcd

etcd is a distributed key-value store built on Raft. It is the coordination backbone of Kubernetes. Like Kahuna, it provides linearizable reads and writes, leases, and watches.

Differences: etcd does not provide a built-in sequencer, a transaction scripting language, or MVCC-based snapshot isolation for multi-key transactions. etcd’s lock API (in the concurrency package) does not return a fencing token directly. A developer can read the key’s `CreateRevision` and use it as a fencing value, but the application must implement that pattern manually. Kahuna returns a fencing token as an explicit field on every lock acquisition. etcd

uses a single Raft group, so every node holds every key. This limits etcd to small clusters (typically 3 or 5 nodes) and a total data size of a few gigabytes. Kahuna uses multiple Raft groups (one per partition) and a configurable replication factor. Each partition is replicated to a subset of nodes, not all of them. This means a Kahuna cluster can grow well beyond 5 or 9 nodes: keys and locks are distributed across many partitions, and each partition only consumes resources on its replicas. etcd is written in Go; Kahuna is written in C#.

2.6.2 Apache ZooKeeper

ZooKeeper is the original distributed coordination service. It provides a hierarchical namespace (similar to a filesystem), ephemeral nodes, watches, and sequential nodes for ordering.

Differences: ZooKeeper's data model is tree-structured (znodes), while Kahuna's is flat (key-value). ZooKeeper does not provide multi-key transactions, a scripting language, or a built-in sequencer with block allocation. ZooKeeper's lock recipe uses sequential ephemeral znodes. The sequence number on the znode (or the znode's `czxid`) is monotonically increasing and can serve as a fencing token, but the application must read and propagate it manually. Like etcd, ZooKeeper uses a single consensus group (Zab), so every node holds every znode. This limits cluster size and total data capacity. Kahuna distributes data across partitions with per-partition replication, which allows it to scale to larger clusters. ZooKeeper is written in Java; Kahuna is written in C#.

2.6.3 HashiCorp Consul

Consul provides service discovery, health checking, and a key-value store. Its KV store supports CAS operations and distributed locks via sessions.

Differences: Consul is primarily a service mesh and discovery tool; its KV store is a secondary feature. Consul does not provide multi-key transactions, MVCC, snapshot isolation, or a sequencer. Consul's KV entries carry a `ModifyIndex` that increases monotonically, so a lock holder can use it as a fencing value, but the application must implement that pattern. Consul is written in Go; Kahuna is written in C#.

2.6.4 Redis

Redis is an in-memory data structure store. With Redlock (a distributed lock algorithm), it provides distributed locking. Redis Cluster provides horizontal scaling with hash-slot partitioning.

Differences: Redis prioritizes throughput and low latency over strong consistency. Redlock has been the subject of academic debate regarding its safety guarantees under certain failure modes. Kahuna's locks are built on Raft consensus with linearizable operations and fencing tokens. Redis does not provide multi-key transactions with snapshot isolation across partitions. Redis is written in C; Kahuna is written in C#.

2.6.5 Summary

Feature	Kahuna	etcd	ZooKeeper	Consul	Redis
Distributed KV store	Yes	Yes	Yes (tree)	Yes	Yes
Strong consistency	Yes (Raft)	Yes (Raft)	Yes (Zab)	Yes (Raft)	No (async replication)
Distributed locks	Yes	Yes (leases)	Yes (ephemeral nodes)	Yes (sessions)	Yes (Red-lock)
Fencing tokens	Yes (built-in)	Manual (via revision)	Manual (via czxid)	Manual (via ModifyIndex)	No
Multi-key transactions	Yes (2PC + MVCC)	Yes (mini-transactions)	No	No	No
Snapshot isolation	Yes	No	No	No	No
Distributed sequencer	Yes	No	Yes (sequential nodes)	No	No
Transaction scripting	Yes	No	No	No	Yes (Lua)
Partitioned (multi-Raft-group)	Yes	No (single group)	No (single group)	No (single group)	Yes (hash slots)
Per-partition replication factor	Yes	No (all nodes)	No (all nodes)	No (all nodes)	Yes (per shard)
Horizontal data scaling	Yes	Limited	Limited	Limited	Yes
Primary language	C#	Go	Java	Go	C

This comparison is factual, not promotional. Each system makes different trade-offs, and the right choice depends on your workload, your team's expertise, and your operational requirements.

2.7 Architecture at a Glance

Before diving into the details in later chapters, here is a brief sketch of how Kahuna is organized internally. You don't need to understand all of this now. Part III of the book covers each component in depth.

A Kahuna cluster consists of multiple **nodes**. Each node runs the same server binary and can serve any request by routing it to the appropriate partition leader.

Data is organized into **partitions**. Each partition is a range of keys managed by its own independent **Raft group**. A Raft group has a **leader** (which handles writes and linearizable reads) and **followers** (which replicate the leader's log). If the leader fails, the remaining replicas elect a new leader.

When a client sends a request, the receiving node determines which partition owns the key (based on hashing or range mapping), finds the leader for that partition, and forwards the request. If the receiving node is the leader, it handles the request locally. The leader proposes the write to Raft, waits for a majority of replicas to acknowledge, and then responds to the client.

Client

- Node (any node can receive)
- Partition resolution (which partition owns this key?)
- Leader (forward to leader if this node isn't it)
- Raft consensus (replicate to majority)
- Acknowledge to client

Kahuna uses **Hybrid Logical Clocks (HLC)** for ordering events across nodes. HLC combines a physical timestamp with a logical counter to produce globally unique, causally ordered timestamps without requiring synchronized clocks. Every committed write gets an HLC timestamp that establishes its position in the total order of operations.

The key-value store uses **Multi-Version Concurrency Control (MVCC)**: each key retains multiple versions, indexed by their HLC timestamp. This allows transactions to read a consistent snapshot of the data at a specific point in time, even while other transactions are writing to the same keys.

2.8 A First Look at the Client SDK

To close this chapter, here is a complete C# program that connects to a Kahuna cluster, uses all three primitives, and handles basic errors. Don't worry about understanding every detail. The following chapters will explain each operation thoroughly.

```
using Kahuna.Client;

// Connect to a Kahuna cluster
var client = new KahunaClient("http://localhost:2070");

// --- Key-Value ---
// Store a value
await client.SetKeyValue("config:feature:dark-mode", "enabled");

// Read it back
var kv = await client.GetKeyValue("config:feature:dark-mode");
Console.WriteLine($"Key: {kv.Key}, Value: {kv.ValueAsString()}, Revision: {kv.Revision}");

// Conditional write: update only if the revision hasn't changed
```

```

var updated = await client.TryCompareRevisionAndSetKeyValue(
    "config:feature:dark-mode",
    "disabled",
    kv.Revision);
Console.WriteLine($"Update succeeded: {updated.Success}");

// --- Distributed Lock ---
// Acquire a lock with a 5-second lease
await using var lockHandle = await client.GetOrCreateLock(
    "job:nightly-report",
    expiresMs: 5000);

if (lockHandle.IsAcquired)
{
    Console.WriteLine($"Acquired lock with fencing token:
{lockHandle.FencingToken}");
    // Protected work goes here
}

// --- Sequencer ---
// Create a sequence and allocate values
await client.CreateSequence("invoice-numbers");
var next = await client.NextSequenceValue("invoice-numbers");
Console.WriteLine($"Next invoice number: {next.CurrentValue}");

```

This program doesn't handle retries, leader changes, or transaction conflicts yet. Chapter 13 covers all of those. For now, the point is to see the shape of the API: connect, operate, and let the cluster handle replication and consistency.

2.9 What's Ahead

The rest of this book is organized into four parts:

Part I, Using Kahuna (Chapters 2 to 7), walks through each primitive in detail. You'll start a cluster, learn the full key-value API, master distributed locks and fencing tokens, use the sequencer, write transactions, and learn the Kahuna scripting language.

Part II, Building Distributed Applications (Chapters 8 to 13), applies these primitives to real engineering problems: leader election, idempotent workflows, service coordination, transactional pipelines, and failure handling.

Part III, Understanding Kahuna Internals (Chapters 14 to 23), opens the hood. You'll trace a request from the client through routing, consensus, the actor model, MVCC, the two-phase commit protocol, persistence, and recovery. Every chapter connects to actual source code.

Part IV, Operating and Extending Kahuna (Chapters 24 to 29), covers production operations: deployment, cluster membership, range management, backup and point-in-time recovery, observability, performance tuning, and Jepsen testing.

You don't need to read the book linearly. Parts I and II are designed for developers who want to use Kahuna. Parts III and IV are for those who want to understand or operate it. But the concepts build on each other, and the book is designed to be read in order if you have the time.

Let's begin. The next chapter gets a cluster running on your machine.

3 Getting Started

This chapter takes you from zero to a working Kahuna environment. By the end, you will have a running cluster, a command-line client, and a C# application that reads, writes, locks, and generates sequence numbers. You will also crash a node and see the cluster recover on its own.

3.1 Prerequisites

Kahuna runs on .NET. You need:

- **.NET 10 SDK** (or later). Download it from dotnet.microsoft.com.
- **Docker** (optional, for multi-node clusters). Any recent version of Docker Desktop or Docker Engine works.

Verify your .NET installation:

```
dotnet --version
```

If this prints a version number of 10.0 or higher, you are ready.

3.2 Installing the Server

Kahuna is distributed as a .NET global tool. Install it with a single command:

```
dotnet tool install -g Kahuna.Server
```

This places the `kahuna-server` command on your PATH. Run it:

```
kahuna-server
```

With no arguments, the server starts a standalone node. It listens for HTTP connections on port 2070. The output shows the storage paths and the ports the node is using.

By default, the node stores its key-value data and Raft write-ahead log under your user data directory:

- **Linux/macOS:** `~/ .local/share/kahuna`
- **Windows:** `%LOCALAPPDATA%\kahuna`

Both paths are printed at startup. To store data in a different location, set the `KAHUNA_HOME` environment variable, or pass `--storage-path` and `--wal-path` explicitly.

The standalone node elects itself as the leader for every partition. It does not need any peer configuration. This makes it the fastest way to start experimenting.

3.2.1 Storage Backends

Kahuna supports three storage backends:

Backend	Flag	Persistence	Use case
RocksDB	<code>--storage rocksdb</code>	Durable	Production (default)
SQLite	<code>--storage sqlite</code>	Durable	Alternative persistent backend
Memory	<code>--storage memory</code>	Ephemeral	Testing and prototyping

The default is RocksDB. For quick experiments where you do not need data to survive a restart, use the memory backend:

```
kahuna-server --storage memory --wal-storage memory
```

3.2.2 HTTPS

The standalone node serves HTTP only. To enable HTTPS, supply a certificate:

```
kahuna-server \  
  --https-ports 2071 \  
  --https-certificate /path/to/certificate.pfx
```

For local development, a self-signed certificate is sufficient. The client SDK has an option to skip certificate validation (covered later in this chapter).

3.3 Installing the Command-Line Client

The CLI is a separate .NET global tool:

```
dotnet tool install -g Kahuna.Control
```

This installs the `kahuna-cli` command. By default, it connects to `http://localhost:2070`, which matches the standalone server's default port.

3.4 First Operations with the CLI

Start a standalone server in one terminal window. Open a second terminal and try the following commands.

3.4.1 Setting and Getting a Key

Write a value:

```
kahuna-cli --set "users/alice" --value '{"name":"Alice","role":"admin"}'
```

Read it back:

```
kahuna-cli --get "users/alice"
```

The output shows the value and metadata, including the revision number. Every write increments the revision.

3.4.2 Setting a Key with an Expiry

Keys can have a time-to-live (TTL). The `--expires` flag takes a value in milliseconds:

```
kahuna-cli --set "session/abc123" --value "active" --expires 30000
```

This key expires after 30 seconds. After that, a `--get` on this key returns nothing.

3.4.3 Acquiring and Releasing a Lock

Acquire a lock named `jobs/send-email`. The `--expires` flag specifies the lease duration in milliseconds:

```
kahuna-cli --lock "jobs/send-email" --owner "worker-1" --expires 10000
```

The output confirms the lock is acquired. It also shows the fencing token, a monotonically increasing number. Each time the lock changes hands, the fencing token increases.

Release the lock:

```
kahuna-cli --unlock "jobs/send-email" --owner "worker-1"
```

Only the owner that acquired the lock can release it.

3.4.4 Creating and Using a Sequence

Create a sequence named `order-ids`:

```
kahuna-cli --create-sequence "order-ids"
```

Retrieve the next value:

```
kahuna-cli --next-sequence "order-ids"
```

Each call returns a new, unique, monotonically increasing number. You can also reserve a range of values at once:

```
kahuna-cli --reserve-sequence "order-ids" --count 10
```

This reserves 10 values and returns the start and end of the range.

3.4.5 Output Formats

By default, the CLI prints results in a human-readable console format. For scripting, use JSON output:

```
kahuna-cli --get "users/alice" --format json
```

3.5 Running a Three-Node Cluster

A single node is fine for development. For testing replication and fault tolerance, you need multiple nodes. Kahuna provides two ways to run a local cluster: Docker Compose and shell scripts.

3.5.1 Option 1: Docker Compose

The Kahuna repository includes a Docker Compose file at `docker/local.yml`. This file defines a three-node cluster on a Docker bridge network.

Clone the repository and start the cluster:

```
git clone https://github.com/kahunakv/kahuna.git
cd kahuna
docker compose -f docker/local.yml up -d
```

The three nodes expose these ports on your host:

Node	HTTP Port	HTTPS/Raft Port
kahuna1	8081	8082
kahuna2	8083	8084
kahuna3	8085	8086

Each node has its own storage volume. Each node's `--initial-cluster` flag lists the other two nodes' Raft endpoints. This is how nodes discover each other at startup.

Verify the cluster is running:

```
docker compose -f docker/local.yml ps
```

All three containers should show a healthy status.

Connect the CLI to any node in the cluster. For the Docker cluster, the HTTPS ports use a self-signed certificate, so add `--insecure` to skip validation:

```
kahuna-cli -c "https://localhost:8082" --insecure --set "test/key" --value "hello"
kahuna-cli -c "https://localhost:8084" --insecure --get "test/key"
```

Notice that you write to node 1 and read from node 2. Kahuna routes the request to the correct partition leader internally, and the value is replicated through Raft. The read returns the same value regardless of which node you connect to.

3.5.2 Option 2: Local Shell Script

If you prefer to run the cluster outside Docker, the repository includes a script that starts three nodes as local processes:

```
cd kahuna
./scripts/run-cluster.sh
```

This script builds the server from source, then starts three nodes on localhost. The port layout is the same as the Docker cluster:

Node	HTTP	HTTPS/Raft
kahuna1	8081	8082
kahuna2	8083	8084
kahuna3	8085	8086

Press Ctrl+C to stop all three nodes.

For a standalone node from source, use:

```
./scripts/run-standalone.sh
```

Both scripts accept environment variables for customization:

Variable	Default	Description
KAHUNA_STORAGE	rocksdb	Storage backend (rocksdb or memory)
KAHUNA_PARTITIONS	3	Initial partition count
KAHUNA_DATA_DIR	/tmp/kahuna-cluster	Data directory (rocksdb only)

For example, to run a fully ephemeral cluster:

```
KAHUNA_STORAGE=memory ./scripts/run-cluster.sh
```

3.5.3 How Cluster Discovery Works

When a node starts, it needs to know about its peers. Kahuna uses static discovery: each node's `--initial-cluster` flag lists the Raft endpoints of the other nodes. The node contacts those peers and forms a Raft group for each partition.

The key configuration flags for clustering are:

Flag	Purpose
<code>--raft-nodename</code>	Unique name for this node
<code>--raft-nodeid</code>	Unique integer ID for this node
<code>--raft-host</code>	Host address for Raft communication
<code>--raft-port</code>	Port for Raft communication
<code>--initial-cluster</code>	Raft endpoints of the other nodes
<code>--initial-cluster-partitions</code>	Number of partitions (default: 3)

For example, to manually start node 1 of a three-node cluster:

```
kahuna-server \  
  --raft-nodename kahuna1 \  
  --raft-nodeid 1 \  
  --raft-host 127.0.0.1 \  
  --raft-port 8082 \  
  --http-ports 8081 \  
  --https-ports 8082 \  
  --https-certificate certificate.pfx \  
  --initial-cluster 127.0.0.1:8084 127.0.0.1:8086 \  
  --initial-cluster-partitions 3
```

Nodes 2 and 3 use the same pattern, with their own node name, ID, ports, and peer list. Each node lists only the other nodes in `--initial-cluster`, not itself.

3.6 Connecting from C#

The Kahuna .NET client library communicates with the server over gRPC. Install it from NuGet:

```
dotnet add package Kahuna.Client
```

3.6.1 Creating a Client

For a single endpoint:

```
using Kahuna.Client;  
  
var client = new KahunaClient("http://localhost:2070");
```

For a cluster with multiple endpoints, pass an array of URLs:

```
var client = new KahunaClient(new[]  
{  
    "https://localhost:8082",  
    "https://localhost:8084",  
    "https://localhost:8086"  
});
```

The client distributes requests across the endpoints using round-robin. Kahuna routes each request to the correct partition leader internally, so any endpoint can handle any request.

3.6.2 Skipping Certificate Validation

When connecting to a cluster that uses self-signed certificates (such as the local Docker cluster), configure the client to skip TLS validation:

```
var client = new KahunaClient(  
    new[] { "https://localhost:8082", "https://localhost:8084", "https://  
localhost:8086" },  
    options: new KahunaOptions { AllowInsecureCertificateValidation = true }  
);
```

Do not use this setting in production.

3.6.3 Key-Value Operations

Write and read a value:

```
KahunaKeyValue setResult = await client.SetKeyValue("users/alice", "admin");  
Console.WriteLine($"Revision: {setResult.Revision}");  
  
KahunaKeyValue getResult = await client.GetKeyValue("users/alice");  
Console.WriteLine($"Value: {getResult.ValueAsString()}");  
Console.WriteLine($"Revision: {getResult.Revision}");
```

Every successful write returns a `KahunaKeyValue` object. The `Success` property tells you whether the write went through. The `Revision` property is a monotonically increasing number that changes with every update to that key.

Write with a TTL (in milliseconds):

```
await client.SetKeyValue("session/token-xyz", "active", expiryTime: 60000);
```

This key expires after 60 seconds.

3.6.4 Distributed Locks

Acquire a lock, do some work, and release it:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(  
    resource: "jobs/send-email",  
    expiry: TimeSpan.FromSeconds(10),  
    durability: LockDurability.Persistent  
);  
  
if (lockHandle.IsAcquired)  
{  
    Console.WriteLine($"Lock acquired. Fencing token:  
{lockHandle.FencingToken}");  
    // Do the protected work here.  
}  
else  
{  
    Console.WriteLine("Could not acquire the lock. Another process holds it.");  
}  
  
// The lock is released automatically when the using block ends.
```

The `GetOrCreateLock` method returns a `KahunaLock` object. If the lock is already held by another owner, `IsAcquired` is false. The lock implements `IAsyncDisposable`, so it is released automatically at the end of the `await using` block.

The `FencingToken` is a monotonically increasing number. Each time a lock changes hands, the fencing token increases. You can pass this token to downstream services to ensure that a stale lock holder cannot overwrite newer data. Chapter 4 covers fencing in detail.

3.6.5 Sequences

Create a sequence and generate values:

```
KahunaSequence seq = await client.CreateSequence("order-ids");
Console.WriteLine($"Sequence created. Current value: {seq.CurrentValue}");

long nextId = await client.NextSequenceValue("order-ids");
Console.WriteLine($"Next value: {nextId}");

KahunaSequenceRange range = await client.ReserveSequenceRange("order-ids",
count: 10);
Console.WriteLine($"Reserved range: {range.Start} to {range.End}");
```

`NextSequenceValue` returns a single value. `ReserveSequenceRange` reserves a batch of values and returns the start and end of the range. The values in a reserved range are guaranteed to be unique and monotonically increasing across all clients.

3.7 A Complete Example

The following console application connects to a standalone Kahuna node and exercises all three primitives:

```
using Kahuna.Client;
using Kahuna.Shared.Lock;

var client = new KahunaClient("http://localhost:2070");

// --- Key-Value ---
KahunaKeyValue kv = await client.SetKeyValue("demo/greeting", "Hello, Kahuna!");
Console.WriteLine($"Set key 'demo/greeting' at revision {kv.Revision}");

KahunaKeyValue read = await client.GetKeyValue("demo/greeting");
Console.WriteLine($"Read: {read.ValueAsString()} (revision {read.Revision})");

// Update the key.
KahunaKeyValue updated = await client.SetKeyValue("demo/greeting", "Hello
again!");
Console.WriteLine($"Updated to revision {updated.Revision}");

// --- Distributed Lock ---
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    resource: "demo/my-lock",
    expiry: TimeSpan.FromSeconds(30),
```

```

        durability: LockDurability.Persistent
    );

    if (lockHandle.IsAcquired)
    {
        Console.WriteLine($"Lock    acquired.    Fencing    token:
{lockHandle.FencingToken}");
    }

    // --- Sequence ---
    await client.CreateSequence("demo/counter");
    for (int i = 0; i < 5; i++)
    {
        long id = await client.NextSequenceValue("demo/counter");
        Console.WriteLine($"Sequence value: {id}");
    }

    Console.WriteLine("Done.");

```

To run this example, create a new console project:

```

mkdir kahuna-demo && cd kahuna-demo
dotnet new console
dotnet add package Kahuna.Client

```

Replace the contents of Program.cs with the code above. Start a standalone Kahuna server in another terminal (kahuna-server), then run the application:

```
dotnet run
```

You should see output similar to:

```

Set key 'demo/greeting' at revision 1
Read: Hello, Kahuna! (revision 1)
Updated to revision 2
Lock acquired. Fencing token: 1
Sequence value: 0
Sequence value: 1
Sequence value: 2
Sequence value: 3
Sequence value: 4
Done.

```

3.8 Your First Failure Experiment

Distributed systems earn their value when something goes wrong. This experiment demonstrates that Kahuna keeps serving requests after a node failure.

3.8.1 Setup

Start the three-node Docker cluster:

```
docker compose -f docker/local.yml up -d
```

Write a key through node 1:

```
kahuna-cli -c "https://localhost:8082" --insecure \  
--set "experiment/counter" --value "42"
```

Verify the key is readable from all three nodes:

```
kahuna-cli -c "https://localhost:8082" --insecure --get "experiment/counter"  
kahuna-cli -c "https://localhost:8084" --insecure --get "experiment/counter"  
kahuna-cli -c "https://localhost:8086" --insecure --get "experiment/counter"
```

All three should return 42.

3.8.2 Kill a Node

Stop the second container:

```
docker stop kahuna2
```

Now try reading and writing through the remaining nodes:

```
kahuna-cli -c "https://localhost:8082" --insecure --get "experiment/counter"  
kahuna-cli -c "https://localhost:8086" --insecure \  
--set "experiment/counter" --value "43"  
kahuna-cli -c "https://localhost:8082" --insecure --get "experiment/counter"
```

The reads and writes still work. Two out of three nodes form a majority (a quorum), so Raft can still reach consensus on new writes.

3.8.3 Bring the Node Back

```
docker start kahuna2
```

After a few seconds, node 2 catches up with the other two. Read the key through node 2:

```
kahuna-cli -c "https://localhost:8084" --insecure --get "experiment/counter"
```

It returns 43, the value that was written while it was down. Raft replicated the missed writes when node 2 rejoined the cluster.

3.8.4 What Happens If Two Nodes Go Down?

Stop two nodes:

```
docker stop kahuna2 kahuna3
```

Now try to write:

```
kahuna-cli -c "https://localhost:8082" --insecure \  
--set "experiment/counter" --value "44"
```

This request fails or times out. With only one node remaining out of three, Raft cannot form a quorum. The cluster refuses to accept writes because it cannot guarantee that the write will be durable on a majority of replicas.

This is intentional. Kahuna chooses consistency over availability: it will not accept a write that it cannot safely replicate.

Bring the nodes back:

```
docker start kahuna2 kahuna3
```

After a few seconds, the cluster resumes normal operation.

3.9 Summary

In this chapter you installed Kahuna's server and CLI tools, ran a standalone node, operated a three-node cluster, and connected a C# application. You used all three primitives (key-value storage, distributed locks, and sequences) from both the CLI and the client SDK. You also ran a failure experiment that demonstrated Raft's ability to maintain availability with a majority of nodes and to replicate missed writes when a failed node returns.

The next chapter explores key-value operations in depth: conditional writes, revisions, TTL, range queries, and durability levels.

4 Key-Value Operations

Chapter 2 showed how to set and get a key. This chapter covers the full key-value API: conditional writes, revisions, time-to-live, prefix queries, range scans, batch operations, durability levels, and snapshot reads. By the end, you will know every tool Kahuna provides for working with key-value data.

4.1 Keys and Values

A Kahuna key is a string. A value is a byte array (or a UTF-8 string, which the client converts to bytes). There is no schema, no type enforcement, and no size hierarchy. Every key is flat.

Keys often follow a convention of prefixed namespaces separated by `/`:

```
users/alice
users/bob
config/feature-flags/dark-mode
orders/2024/00042
```

This convention is not just cosmetic. Kahuna uses the key's prefix (everything up to and including the last `/`) to determine which partition stores the key. Keys that share a prefix route to the same partition. This matters for prefix queries (covered later in this chapter) and for transactions (covered in Chapter 6).

4.2 Revisions

Every key in Kahuna has a revision number. The revision starts at 0 on the first write and increases by one with every subsequent write to the same key.

```
var client = new KahunaClient("http://localhost:2070");

KahunaKeyValue r1 = await client.SetKeyValue("counter/a", "first");
Console.WriteLine(r1.Revision); // 0

KahunaKeyValue r2 = await client.SetKeyValue("counter/a", "second");
Console.WriteLine(r2.Revision); // 1

KahunaKeyValue r3 = await client.SetKeyValue("counter/a", "third");
Console.WriteLine(r3.Revision); // 2
```

Revisions are per-key. Writing to `counter/a` does not affect the revision of `counter/b`.

The revision number serves two purposes:

1. **Conflict detection.** Conditional writes can check the revision before updating. If another client changed the key since you last read it, the revision will not match and the write will fail. This is optimistic concurrency control.
2. **History.** Kahuna stores previous revisions. You can read the value of a key at any past revision.

4.2.1 Reading a Specific Revision

```
KahunaKeyValue atRev1 = await client.GetKeyValueRevision("counter/a", revision:
1);
Console.WriteLine(atRev1.ValueAsString()); // "second"
```

If the requested revision does not exist, `Success` is false and `Value` is null.

4.3 Conditional Writes

An unconditional `SetKeyValue` always overwrites the current value. Conditional writes add a check: the write only succeeds if a condition is met. If the condition fails, `Success` is false and the key is unchanged.

Kahuna provides four conditional write modes, controlled by the `KeyValueFlags` parameter.

4.3.1 Set If Not Exists

Write only if the key does not already exist:

```
KahunaKeyValue result = await client.SetKeyValue(
    "config/db-connection",
    "Server=primary;Database=app",
    flags: KeyValueFlags.SetIfNotExists
);

if (result.Success)
    Console.WriteLine("Configuration key created.");
else
    Console.WriteLine("Key already exists. Not overwritten.");
```

This is useful for one-time initialization. Multiple processes can race to set the key. Exactly one will succeed.

4.3.2 Set If Exists

Write only if the key already exists:

```
KahunaKeyValue result = await client.SetKeyValue(
    "config/db-connection",
    "Server=secondary;Database=app",
    flags: KeyValueFlags.SetIfExists
);

if (result.Success)
    Console.WriteLine("Configuration updated.");
else
    Console.WriteLine("Key does not exist. Nothing to update.");
```

4.3.3 Compare-Revision-And-Set (CRAS)

Write only if the key's current revision matches a specified value. This is the primary mechanism for optimistic concurrency control.

```
// Read the current value and its revision.
KahunaKeyValue current = await client.GetKeyValue("inventory/widget-stock");
Console.WriteLine($"Current stock: {current.ValueAsString()}, revision: {current.Revision}");

// Update the value, but only if nobody else changed it.
KahunaKeyValue updated = await client.TryCompareRevisionAndSetKeyValue(
```



```

    "inventory/widget-stock",
    "95",
    compareRevision: current.Revision
);

if (updated.Success)
    Console.WriteLine($"Updated to revision {updated.Revision}.");
else
    Console.WriteLine("Conflict: another client modified the key. Read and
retry.");

```

The pattern is read, modify, write-if-unchanged. If another client wrote to the key between your read and your write, the revision will differ and the write will fail. Your code can then re-read and retry.

4.3.4 Compare-Value-And-Set (CVAS)

Write only if the key's current value matches a specified byte sequence:

```

KahunaKeyValue result = await client.TryCompareValueAndSetKeyValue(
    "state-machine/order-123",
    value: "shipped",
    compareValue: "paid"
);

if (result.Success)
    Console.WriteLine("State transitioned from 'paid' to 'shipped'.");
else
    Console.WriteLine("Current value is not 'paid'. Transition rejected.");

```

This is useful for state machines. The transition from “paid” to “shipped” only succeeds if the current state is “paid.” If another process already moved the state to “cancelled,” the write fails.

4.4 Time-to-Live (TTL)

Keys can have an expiration time. After the TTL elapses, the key is no longer readable. The TTL is specified in milliseconds.

```

// This key expires after 60 seconds.
await client.SetKeyValue("session/token-abc", "active", expiryTime: 60000);

```

You can also use a TimeSpan:

```

await client.SetKeyValue("session/token-abc",
    "active", TimeSpan.FromMinutes(5));

```

An expiry of 0 (the default) means the key does not expire.

4.4.1 Extending a TTL

If a key is about to expire but you need it to live longer, extend its TTL without changing its value:

```

await client.ExtendKeyValue("session/token-abc", expiresMs: 60000);

```

Or using a TimeSpan:

```
await client.ExtendKeyValue("session/token-abc", TimeSpan.FromMinutes(5));
```

The extension resets the expiration clock from the current time. If the key does not exist, Success is false.

4.4.2 TTL and Conditional Writes

You can combine TTL with conditional write flags:

```
// Create a session key only if it doesn't already exist, with a 30-minute TTL.
KahunaKeyValue session = await client.SetKeyValue(
    "session/user-42",
    "session-data-here",
    expiryTime: 1800000,
    flags: KeyValueFlags.SetIfNotExists
);
```

4.5 Durability Levels

Every key-value operation accepts a durability parameter with two options:

Level	Behavior
Persistent	The write goes through Raft consensus and is replicated to a majority of nodes. This is the default.
Ephemeral	The write is stored in memory on the receiving node only. It is not replicated and does not survive a node restart.

```
// Persistent (default): safe, replicated.
await client.SetKeyValue("config/important", "value", durability:
    KeyValueDurability.Persistent);

// Ephemeral: fast, local, not replicated.
await client.SetKeyValue("cache/temp-result", "value", durability:
    KeyValueDurability.Ephemeral);
```

Use ephemeral durability for data you can afford to lose: caches, temporary counters, rate-limit windows. Use persistent durability for anything that must survive failures.

A read with a specific durability level only sees keys written at that same level. A key written as Ephemeral is not visible to a Persistent read, and vice versa.

4.6 Deleting Keys

Delete a key:

```
KahunaKeyValue deleted = await client.DeleteKeyValue("users/alice");
if (deleted.Success)
    Console.WriteLine("Key deleted.");
else
    Console.WriteLine("Key did not exist.");
```

A delete on a non-existent key sets Success to false.

4.7 Checking Existence

If you only need to know whether a key exists (without fetching its value), use `ExistsKeyValue`:

```
KahunaKeyValue exists = await client.ExistsKeyValue("users/alice");
if (exists.Success)
    Console.WriteLine($"Key exists at revision {exists.Revision}.");
else
    Console.WriteLine("Key does not exist.");
```

This is lighter than `GetKeyValue` because the server does not transfer the value bytes.

4.8 No-Revision Writes

By default, every write archives a revision entry so you can read the key's history. For high-throughput keys where you never need history (caches, counters, ephemeral state), you can skip revision archiving:

```
await client.SetKeyValueNoRevision("metrics/request-count", "4217");
```

The revision counter still increments and conditional writes still work, but the previous value is not stored for historical reads. This reduces storage overhead for keys that change frequently.

4.9 Batch Operations

When you need to read, write, or delete many keys at once, batch operations reduce the number of round trips.

4.9.1 Batch Set

```
var items = new List<KahunaSetKeyValueRequestItem>
{
    new() { Key = "users/alice", Value = "admin"u8.ToArray() },
    new() { Key = "users/bob", Value = "editor"u8.ToArray() },
    new() { Key = "users/carol", Value = "viewer"u8.ToArray() }
};

List<KahunaKeyValue> results = await client.SetManyKeyValues(items);

foreach (KahunaKeyValue kv in results)
    Console.WriteLine($"{kv.Key}: revision {kv.Revision}, success: {kv.Success}");
```

Each item in the batch can have its own flags, expiry, and durability level.

4.9.2 Batch Get

```
var requests = new List<KahunaGetManyKeyValuesRequestItem>
{
    new() { Key = "users/alice" },
    new() { Key = "users/bob" },
    new() { Key = "users/carol" }
};
```

```
List<KahunaKeyValue> results = await client.GetManyKeyValues(requests);

foreach (KahunaKeyValue kv in results)
{
    if (kv.Success)
        Console.WriteLine($"{kv.Key} = {kv.ValueAsString()}");
    else
        Console.WriteLine($"{kv.Key}: not found");
}
```

4.9.3 Batch Delete

```
// Simple form: pass a list of keys.
List<KahunaKeyValue> deleted = await client.DeleteManyKeyValues(
    new[] { "users/alice", "users/bob", "users/carol" }
);

foreach (KahunaKeyValue kv in deleted)
    Console.WriteLine($"{kv.Key}: deleted = {kv.Success}");
```

4.9.4 Batch Exists

```
var requests = new List<KahunaGetManyKeyValuesRequestItem>
{
    new() { Key = "users/alice" },
    new() { Key = "users/bob" }
};

List<KahunaKeyValue> results = await client.ExistsManyKeyValues(requests);
```

All batch operations complete in a single round trip to the server.

4.10 Prefix Queries

Kahuna provides two ways to query keys by prefix. The difference between them is scope.

4.10.1 Single-Partition Prefix Query

GetByBucket returns all keys that share a prefix, within the partition that owns that prefix:

```
List<KahunaKeyValue> users = await client.GetByBucket(
    "users",
    KeyValueDurability.Persistent
);

foreach (KahunaKeyValue kv in users)
    Console.WriteLine($"{kv.Key} = {kv.ValueAsString()}");
```

Because keys with the same prefix hash to the same partition, this query contacts only one partition. It is efficient, but it only finds keys that share the exact prefix.

The result set is capped at 4,096 entries. For larger sets, use range scans (next section).

4.10.2 All-Node Prefix Scan

ScanAllByPrefix searches all nodes for keys that match a prefix:

```
List<KahunaKeyValue> allConfigs = await client.ScanAllByPrefix(
    "config",
    KeyValueDurability.Persistent
);
```

This is a broader operation. It contacts every node in the cluster. Use it when you need to find keys across partitions. Like `GetByBucket`, the result is capped at 4,096 entries.

4.11 Range Scans

For large data sets or fine-grained control over which keys to return, use range scans.

4.11.1 Paginated Range Query

`GetByRange` returns keys within a prefix, optionally bounded by start and end keys:

```
List<KahunaKeyValue> page = await client.GetByRange(
    prefix: "orders/2024",
    startKey: "orders/2024/00100",
    startInclusive: true,
    endKey: "orders/2024/00200",
    endInclusive: false,
    limit: 50
);
```

This returns up to 50 keys in the range `[orders/2024/00100, orders/2024/00200)` within the `orders/2024` prefix. The `startInclusive` and `endInclusive` parameters control whether the boundary keys are included.

If you omit `startKey` and `endKey`, the query returns the first `limit` keys in the prefix.

4.11.2 Streaming Range Scan

`ScanByRange` returns an `IAsyncEnumerable<KahunaKeyValue>` that pages through results automatically:

```
await foreach (KahunaKeyValue kv in client.ScanByRange(
    prefix: "logs/2024-03",
    pageSize: 100))
{
    Console.WriteLine($"{kv.Key}: {kv.ValueAsString()}");
}
```

The client fetches pages of 100 entries from the server. Your code iterates over them as a continuous stream. This is the right choice when you do not know how many keys exist and want to process them one at a time without loading all results into memory.

You can also bound the scan with start and end keys:

```
await foreach (KahunaKeyValue kv in client.ScanByRange(
    prefix: "events",
    startKey: "events/2024-03-01",
    endKey: "events/2024-03-31",
    endInclusive: true,
    pageSize: 200))
```

```
{
    ProcessEvent(kv);
}
```

4.12 Snapshot Reads

By default, every read returns the latest committed value. Snapshot reads let you read the state of the key-value store at a specific point in time.

The `snapshotMs` parameter accepts a Unix epoch timestamp in milliseconds. When set, the read returns the value that was current at that moment.

```
// Write a value.
KahunaKeyValue v1 = await client.SetKeyValue("price/widget", "10.00");
long snapshotTime = v1.LastModified;

// Write a new value.
await client.SetKeyValue("price/widget", "12.00");

// Read the current value.
KahunaKeyValue current = await client.GetKeyValue("price/widget");
Console.WriteLine(current.ValueAsString()); // "12.00"

// Read the value at the snapshot time.
KahunaKeyValue snapshot = await client.GetKeyValue("price/widget", snapshotMs:
snapshotTime);
Console.WriteLine(snapshot.ValueAsString()); // "10.00"
```

The `LastModified` property on a `KahunaKeyValue` is a Unix epoch timestamp in milliseconds. You can use it as a snapshot anchor: read a key, save its `LastModified`, and later re-read at that exact point.

Snapshot reads also work with `GetByBucket`, `ScanAllByPrefix`, `GetByRange`, and `ScanByRange`. This gives you a consistent view across multiple keys at the same point in time.

```
// Read all user keys as they were at a specific moment.
List<KahunaKeyValue> usersAtSnapshot = await client.GetByBucket(
    "users",
    KeyValueDurability.Persistent,
    snapshotMs: snapshotTime
);
```

Snapshot reads depend on Kahuna's MVCC (multi-version concurrency control) layer. Chapter 19 covers the internals of how snapshots work.

4.13 The KahunaKeyValue Object

Every key-value operation returns a `KahunaKeyValue` object. Here is a summary of its properties and methods:

Member	Type	Description
Key	string	The key that was operated on
Success	bool	Whether the operation succeeded
Revision	long	The key's revision after the operation
Value	byte[]?	The raw value bytes (null if not found or not fetched)
Durability	KeyValueDurability	The durability level used
TimeElapsedMs	int	Server-side time for the operation
LastModified	long	Commit timestamp (Unix epoch ms)
ValueAsString()	string?	Decode the value as UTF-8
ValueAsLong()	long	Parse the value as a long integer
ValueAsBool()	bool	Parse the value as a boolean
Extend(TimeSpan)	Task<KahunaKeyValue>	Extend the key's TTL
Delete()	Task<KahunaKeyValue>	Delete this key

The Extend and Delete methods operate on the same key and durability level that produced the result. They are convenience methods so you can chain operations without repeating the key name.

```
KahunaKeyValue session = await client.GetKeyValue("session/user-42");
if (session.Success)
{
    // Extend the session by 30 more minutes.
    await session.Extend(TimeSpan.FromMinutes(30));
}
```

4.14 CLI Reference

The command-line client supports all key-value operations. Here is a quick reference:

Operation	Command
Set a key	<code>kahuna-cli --set "key" --value "val"</code>
Set with TTL	<code>kahuna-cli --set "key" --value "val" --expires 60000</code>
Get a key	<code>kahuna-cli --get "key"</code>
Extend TTL	<code>kahuna-cli --extend "key" --expires 60000</code>
Prefix query (single partition)	<code>kahuna-cli --get-by-prefix "prefix"</code>
Prefix scan (all nodes)	<code>kahuna-cli --scan-by-prefix "prefix"</code>
JSON output	<code>add --format json</code> to any command

To connect to a specific node, use `-c`:

```
kahuna-cli -c "https://localhost:8082" --insecure --get "users/alice"
```

4.15 Putting It Together

The following example demonstrates several key-value patterns in a single program:

```
using Kahuna.Client;
using Kahuna.Shared.KeyValue;

var client = new KahunaClient("http://localhost:2070");

// 1. Create a key only if it doesn't exist (one-time initialization).
KahunaKeyValue init = await client.SetKeyValue(
    "app/version", "1.0.0", flags: KeyValueFlags.SetIfNotExists);
Console.WriteLine($"Init: success={init.Success}, revision={init.Revision}");

// 2. Read, modify, write with optimistic concurrency.
KahunaKeyValue config = await client.GetKeyValue("app/version");
KahunaKeyValue updated = await client.TryCompareRevisionAndSetKeyValue(
    "app/version", "1.1.0", compareRevision: config.Revision);

if (updated.Success)
    Console.WriteLine($"Updated to {updated.Revision}");
else
    Console.WriteLine("Conflict detected. Retry needed.");

// 3. State machine transition via compare-value-and-set.
await client.SetKeyValue("order/99/status", "pending");

KahunaKeyValue transition = await client.TryCompareValueAndSetKeyValue(
    "order/99/status", value: "confirmed", compareValue: "pending");
Console.WriteLine($"Transition: {transition.Success}");
```



```

// 4. Batch write with TTL.
var items = new List<KahunaSetKeyValueRequestItem>
{
    new() { Key = "cache/result-a", Value = "42"u8.ToArray(), ExpiresMs = 30000 },
    new() { Key = "cache/result-b", Value = "99"u8.ToArray(), ExpiresMs = 30000 }
};
await client.SetManyKeyValues(items);

// 5. Prefix query.
List<KahunaKeyValue> cacheEntries = await client.GetByBucket(
    "cache", KeyValueDurability.Persistent);
Console.WriteLine($"Cache entries: {cacheEntries.Count}");

// 6. Snapshot read.
KahunaKeyValue v1 = await client.SetKeyValue("price/gold", "2000");
long anchor = v1.LastModified;

await client.SetKeyValue("price/gold", "2050");

KahunaKeyValue latest = await client.GetKeyValue("price/gold");
KahunaKeyValue past = await client.GetKeyValue("price/gold", snapshotMs:
anchor);

Console.WriteLine($"Latest: {latest.ValueAsString()}"); // 2050
Console.WriteLine($"At snapshot: {past.ValueAsString()}"); // 2000

```

4.16 Summary

Kahuna's key-value API goes beyond simple get and set. Conditional writes (SetIfNotExists, SetIfExists, CRAS, CVAS) give you optimistic concurrency control and state machine transitions without external locks. Revisions provide a per-key version history. TTL lets keys expire automatically. Prefix queries and range scans let you retrieve groups of related keys efficiently. Snapshot reads give you a consistent view of the store at a past point in time. Batch operations reduce round trips when you work with many keys at once.

The next chapter covers distributed locks: lease semantics, fencing tokens, lock contention, and what happens when a lock holder crashes.

5 Distributed Locking

On a single machine, a mutex is enough. A thread acquires the mutex, does its work, and releases it. The operating system guarantees that only one thread holds the mutex at a time.

In a distributed system, there is no shared memory and no single operating system. Two processes on different machines can both believe they hold “the lock” if the mechanism that coordinates them fails. This chapter explains how Kahuna’s distributed locks work, why leases and fencing tokens are necessary, and how to use them correctly.

5.1 Why Locks Need Leases

A naive distributed lock works like this: write your name into a key. If the key is empty, you hold the lock. Delete the key when you are done.

The problem is “when you are done.” What if the lock holder crashes before it deletes the key? The lock is held forever. No other process can acquire it.

The solution is a lease: a lock with an expiration time. If the holder does not release the lock before the lease expires, the lock becomes available to other clients. This guarantees progress. A crashed process cannot block the system indefinitely.

Kahuna’s locks are lease-based. Every lock acquisition requires an expiry time. When the expiry elapses, the lock is automatically released.

5.2 Acquiring a Lock

The simplest lock acquisition takes a resource name and an expiry:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(  
    resource: "jobs/send-invoice",  
    expiry: TimeSpan.FromSeconds(30),  
    durability: LockDurability.Persistent  
);  
  
if (lockHandle.IsAcquired)  
{  
    Console.WriteLine($"Lock acquired. Fencing token:  
{lockHandle.FencingToken}");  
    // Do the protected work here.  
}  
else  
{  
    Console.WriteLine("Lock is held by another process.");  
}
```

GetOrCreateLock returns a KahunaLock object. If the lock is already held by another owner, IsAcquired is false. The call does not block or wait. It tries once and returns immediately.

The lock implements IAsyncDisposable. When the await using block ends, the lock is released automatically. If the process crashes before the block ends, the lease expires and the lock becomes available.

5.2.1 Lock Parameters

Parameter	Type	Default	Description
resource	string	(required)	The name of the resource to lock
expiry	TimeSpan or int (ms)	30,000 ms	How long the lease lasts
wait	TimeSpan or int (ms)	0	How long to keep trying if the lock is held
retry	TimeSpan or int (ms)	0	How often to retry during the wait period
durability	LockDurability	Persistent	Replicated or in-memory

5.2.2 Lock Owner

Every lock acquisition generates a unique owner token (a GUID). This token identifies the holder. Only the owner that acquired the lock can release or extend it. You do not need to manage this token yourself; the `KahunaLock` object holds it internally.

You can read the owner for debugging:

```
if (lockHandle.IsAcquired)
    Console.WriteLine($"Owner: {lockHandle.OwnerAsString}");
```

5.3 Fencing Tokens

Leases solve the problem of a crashed lock holder. But they introduce a new problem: the stale lock holder.

5.3.1 The Stale Lock Holder Problem

Consider this scenario:

1. Process A acquires a lock with a 10-second lease.
2. Process A starts writing to a shared resource (a database, a file, an external API).
3. Process A experiences a long garbage collection pause, a network delay, or a slow disk. It does not crash, but it stops making progress for 15 seconds.
4. The lease expires after 10 seconds. Process A does not know this yet.
5. Process B acquires the lock. It starts writing to the same shared resource.
6. Process A resumes. It still believes it holds the lock. It writes to the shared resource, overwriting what process B just wrote.

Both processes believe they are the rightful lock holder. The data is now corrupted.

Extending the lease duration does not fix this. No matter how long the lease is, a sufficiently long pause can outlast it. The fundamental problem is that process A has no way to know its lease expired.

5.3.2 How Fencing Tokens Solve This

Kahuna assigns a fencing token to every lock acquisition. The fencing token is a monotonically increasing integer that is scoped to a specific resource. Each time the lock changes hands, the fencing token increases.

Here is the sequence with fencing tokens:

1. Process A acquires the lock. Fencing token: 5.
2. Process A pauses for 15 seconds. The lease expires.
3. Process B acquires the lock. Fencing token: 6.
4. Process A resumes and tries to write to the shared resource, passing fencing token 5.
5. The shared resource checks the fencing token. It has already seen token 6 from process B. It rejects token 5 because it is stale.

The downstream system (a database, an API, a message queue) must participate in fencing. It stores the highest fencing token it has seen and rejects any write with a lower token.

5.3.3 Fencing Token Behavior

- The first acquisition of a resource gets fencing token 0.
- Each new acquisition (by a different owner, or after expiry) increments the token by 1.
- Extending a lock does not change the fencing token.
- The same owner re-acquiring a lock before it expires gets the same fencing token back (no increment).
- Fencing tokens are per-resource. Locking jobs/a and jobs/b produce independent token sequences.

```
// First acquisition: token = 0
await using (KahunaLock first = await client.GetOrCreateLock("demo/fence",
    TimeSpan.FromSeconds(5)))
{
    Console.WriteLine(first.FencingToken); // 0
}

// Second acquisition (after release): token = 1
await using (KahunaLock second = await client.GetOrCreateLock("demo/fence",
    TimeSpan.FromSeconds(5)))
{
    Console.WriteLine(second.FencingToken); // 1
}
```

5.3.4 Using Fencing Tokens in Practice

Pass the fencing token to every downstream operation that the lock protects:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    "orders/process-batch",
    TimeSpan.FromSeconds(30)
);

if (lockHandle.IsAcquired)
{
    long token = lockHandle.FencingToken;

    // Pass the token to the database.
    await db.ExecuteAsync(
        "UPDATE orders SET status = 'processed' WHERE batch_id = @batch AND
```

```
fence_token < @token",
    new { batch = batchId, token }
);
}
```

The database query rejects the update if a higher fencing token has already been written to this batch. This prevents a stale lock holder from overwriting newer data.

Not every system supports fencing natively. For systems that do not, you can store the fencing token as a column or field and check it in your application logic before writing.

5.4 Waiting for a Lock

By default, `GetOrCreateLock` tries once. If the lock is held, it returns immediately with `IsAcquired = false`. For workloads where you want to wait until the lock becomes available, pass `wait` and `retry` parameters:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    resource: "jobs/send-invoice",
    expiry: TimeSpan.FromSeconds(30),
    wait: TimeSpan.FromSeconds(10),
    retry: TimeSpan.FromMilliseconds(200)
);
```

This tries to acquire the lock. If it fails, it retries every 200 milliseconds (with a small random jitter of plus or minus 50 ms) until either the lock is acquired or 10 seconds elapse.

If the wait period expires without acquiring the lock, `IsAcquired` is false. The call does not throw an exception.

The `retry` parameter must be greater than zero when `wait` is greater than zero. Otherwise the client throws a `KahunaException`.

5.4.1 Contention with Multiple Clients

When multiple clients compete for the same lock using `wait` and `retry`, each client retries independently. Kahuna does not queue waiters or guarantee fairness. The first client to attempt acquisition after the lock is released wins. The jitter on the retry interval reduces the chance that all clients retry at exactly the same time.

```
// 10 workers competing for the same lock.
var tasks = Enumerable.Range(0, 10).Select(async i =>
{
    await using KahunaLock lk = await client.GetOrCreateLock(
        "shared/critical-section",
        expiry: TimeSpan.FromSeconds(5),
        wait: TimeSpan.FromSeconds(60),
        retry: TimeSpan.FromMilliseconds(100)
    );

    if (lk.IsAcquired)
    {
        Console.WriteLine($"Worker {i} acquired lock. Token: {lk.FencingToken}");
        await Task.Delay(500); // Simulate work.
    }
});
```

```
    }
});
```

```
await Task.WhenAll(tasks);
```

Each worker eventually acquires the lock because the lease is short (5 seconds) and the wait budget is long (60 seconds).

5.5 Extending a Lease

If your work takes longer than the original lease, extend it before it expires:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    "jobs/long-running",
    TimeSpan.FromSeconds(10)
);

if (lockHandle.IsAcquired)
{
    // Start the work.
    await ProcessFirstBatch();

    // Extend the lease by another 10 seconds.
    (bool extended, long token) = await
lockHandle.TryExtend(TimeSpan.FromSeconds(10));

    if (extended)
    {
        Console.WriteLine($"Lease extended. Fencing token still: {token}");
        await ProcessSecondBatch();
    }
    else
    {
        Console.WriteLine("Extension failed. The lease may have expired.");
        // Stop work. Another process may now hold the lock.
    }
}
```

Key details about extension:

- TryExtend resets the expiration clock from the current time.
- Only the current owner can extend the lock. If the lease already expired and another process acquired it, TryExtend returns false.
- Extending does not change the fencing token. The token remains the same for the duration of a single ownership period.
- There is no automatic renewal. Your application must call TryExtend explicitly.

You can also extend a lock through the client directly, if you have the owner token:

```
(bool success, long token) = await client.TryExtendLock(
    "jobs/long-running",
    lockHandle.Owner,
```

```

    TimeSpan.FromSeconds(10),
    LockDurability.Persistent
);

```

5.6 What Happens When a Lock Holder Crashes

This is the scenario leases are designed for:

1. Process A acquires a lock with a 10-second lease.
2. Process A crashes (power failure, OOM kill, unhandled exception).
3. The `DisposeAsync` call never runs because the process is gone.
4. After 10 seconds, the lease expires.
5. Process B acquires the lock with a new, higher fencing token.

No manual intervention is required. The cluster does not need to detect the crash. The lease simply expires, and the lock becomes available.

This is why lease duration matters:

- **Too short:** The lock expires before the holder finishes its work. The holder must extend frequently, adding network round trips.
- **Too long:** After a crash, other processes wait for the lease to expire before they can make progress.

A good starting point is 10 to 30 seconds for most workloads. Adjust based on how long the protected work takes and how quickly you need recovery after a crash.

5.7 Unlocking

The `await` using pattern releases the lock automatically. You can also unlock manually:

```

KahunaLock lockHandle = await client.GetOrCreateLock(
    "jobs/send-email",
    TimeSpan.FromSeconds(30)
);

if (lockHandle.IsAcquired)
{
    await DoWork();
    bool released = await client.Unlock(
        "jobs/send-email",
        lockHandle.Owner,
        LockDurability.Persistent
    );
    Console.WriteLine($"Released: {released}");
}

```

Only the owner can unlock. If a different process tries to unlock with a different owner token, the call returns false.

If you do not unlock and do not dispose, the lock remains held until the lease expires. The `KahunaLock` finalizer logs a warning if the object is garbage-collected without being disposed.

5.8 Querying Lock State

You can inspect a lock's current state without acquiring it:

```
KahunaLockInfo? info = await client.GetLockInfo(
    "jobs/send-email",
    LockDurability.Persistent
);

if (info != null)
{
    Console.WriteLine($"Owner: {info.Owner}");
    Console.WriteLine($"Fencing token: {info.FencingToken}");
    Console.WriteLine($"Expires: {info.Expires}");
}
```

If no lock exists for the resource, `GetLockInfo` returns null.

You can also query through a `KahunaLock` object, even if you did not acquire the lock:

```
KahunaLock lockHandle = await client.GetOrCreateLock("jobs/send-email",
    TimeSpan.FromSeconds(5));
KahunaLockInfo? info = await lockHandle.GetInfo();
```

Lock info is a diagnostic tool. Do not use it to make decisions about whether to proceed with work. Between the time you read the lock info and the time you act on it, the lock state can change. Use `GetOrCreateLock` for coordination, not `GetLockInfo`.

5.9 Ephemeral Locks

By default, locks are persistent: the acquisition goes through Raft consensus and is replicated to a majority of nodes. This provides strong guarantees but adds latency.

For workloads where speed matters more than durability (coordinating in-memory caches, local-only rate limiting), use ephemeral locks:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    "cache/rebuild",
    expiry: TimeSpan.FromSeconds(5),
    durability: LockDurability.Ephemeral
);
```

Ephemeral locks are stored in memory on the receiving node only. They are faster but do not survive a node restart. If the node crashes, the lock is gone immediately (no need to wait for lease expiry, but also no replication safety).

Use ephemeral locks only when losing the lock on a node failure is acceptable.

5.10 CLI Lock Operations

The command-line client supports lock operations:

Acquire a lock:

```
kahuna-cli --lock "jobs/send-email" --owner "worker-1" --expires 10000
```

Extend a lock:


```
kahuna-cli --extend-lock "jobs/send-email" --owner "worker-1" --expires 10000
```

Release a lock:

```
kahuna-cli --unlock "jobs/send-email" --owner "worker-1"
```

When using the CLI, you supply the owner name yourself. In the C# SDK, the owner is generated automatically as a GUID.

5.11 Common Patterns

5.11.1 Try-Once Pattern

Attempt the lock once. If it fails, skip the work or report the conflict:

```
await using KahunaLock lk = await client.GetOrCreateLock(
    "jobs/daily-report", TimeSpan.FromMinutes(5));

if (!lk.IsAcquired)
{
    Console.WriteLine("Another instance is already generating the report.");
    return;
}

await GenerateDailyReport();
```

This is appropriate when duplicate work is harmless (another instance is already doing the job) or when you have an external retry mechanism (a job scheduler that retries later).

5.11.2 Wait-and-Retry Pattern

Wait for the lock to become available:

```
await using KahunaLock lk = await client.GetOrCreateLock(
    "orders/checkout",
    expiry: TimeSpan.FromSeconds(15),
    wait: TimeSpan.FromSeconds(30),
    retry: TimeSpan.FromMilliseconds(100)
);

if (!lk.IsAcquired)
    throw new TimeoutException("Could not acquire checkout lock within 30 seconds.");

await ProcessCheckout();
```

5.11.3 Extend-While-Working Pattern

For work that may take longer than the initial lease:

```
await using KahunaLock lk = await client.GetOrCreateLock(
    "etl/load-customers", TimeSpan.FromSeconds(30));

if (!lk.IsAcquired) return;

foreach (var batch in customerBatches)
```

```

{
    await LoadBatch(batch);

    // Extend the lease before it expires.
    (bool ok, _) = await lk.TryExtend(TimeSpan.FromSeconds(30));
    if (!ok)
    {
        Console.WriteLine("Lost the lock. Stopping work.");
        return;
    }
}

```

Each iteration resets the lease clock. If the extension fails (because the lease expired between iterations), the code stops immediately to avoid conflicting with a new lock holder.

5.12 Summary

Kahuna's distributed locks combine leases and fencing tokens to solve two problems: a crashed lock holder (the lease expires and frees the lock) and a stale lock holder (the fencing token lets downstream systems reject outdated writes). Locks can wait and retry when contended, extend their lease for long-running work, and be queried for diagnostic purposes. Ephemeral locks trade durability for speed when replication is not needed.

The next chapter covers the distributed sequencer: generating globally unique, monotonically increasing numbers across the cluster.

6 The Distributed Sequencer

Many applications need unique, monotonically increasing numbers: order IDs, invoice numbers, event sequence numbers, log offsets. On a single machine, an auto-incrementing counter solves the problem. In a distributed system, there is no single machine to own the counter. Two nodes incrementing their own local counters will produce duplicates.

Kahuna's distributed sequencer generates globally unique, monotonically increasing numbers across the entire cluster. It does this without making every allocation wait for Raft consensus, thanks to a block-based allocation scheme that amortizes the cost of consensus over many values.

6.1 Creating a Sequence

A sequence is a named, persistent counter. Create one with a name:

```
KahunaSequence seq = await client.CreateSequence("order-ids");
```

The sequence starts at 0 with an increment of 1. You can customize the starting value, the increment, and an optional maximum:

```
KahunaSequence invoiceSeq = await client.CreateSequence(  
    name: "invoice-numbers",  
    initialValue: 1000,  
    increment: 1,  
    maxValue: 999999  
);
```

This sequence starts allocating from 1001 (initialValue + increment) and stops at 999,999. If you try to allocate past the maximum, the client throws a `KahunaException` with a `MaxValueExceeded` response.

Creating a sequence that already exists throws a `KahunaException` with an `AlreadyExists` response. Use `GetSequence` to check whether a sequence exists before creating it.

6.2 Getting the Next Value

The simplest operation is requesting a single value:

```
long id = await client.NextSequenceValue("order-ids");  
Console.WriteLine(id); // 1
```

Each call returns the next value in the sequence. Values are unique across all clients and all nodes. Two clients calling `NextSequenceValue` concurrently on the same sequence will never receive the same number.

6.3 Reserving a Range

If you need many values at once, reserve a range:

```
KahunaSequenceRange range = await client.ReserveSequenceRange("order-ids",  
    count: 100);  
Console.WriteLine($"Start: {range.Start}, End: {range.End}, Count:  
{range.Count}");
```

The returned range contains count consecutive values. Your application can use them locally without making further calls to Kahuna:

```
KahunaSequenceRange range = await client.ReserveSequenceRange("order-ids",
count: 50);

for (long id = range.Start; id <= range.End; id++)
{
    await InsertOrder(id);
}
```

Reserving a range is more efficient than calling `NextSequenceValue` in a loop because it requires only one round trip to the server.

6.4 How Block Allocation Works

Understanding block allocation explains why the sequencer is fast and what trade-offs it makes.

6.4.1 The Problem with Naive Allocation

A naive sequencer would store a counter in Raft and increment it on every request. Each request would require a Raft proposal, replication to a majority of nodes, and a durable write. At high throughput, this becomes a bottleneck: hundreds of Raft round trips per second for a single counter.

6.4.2 Block-Based Allocation

Kahuna uses a different approach. Instead of incrementing the counter by 1 on every request, the sequencer reserves a block of values in a single Raft operation.

Here is how it works:

1. The server stores a high-water mark for each sequence. The high-water mark is the highest value that has been reserved (not necessarily issued to clients).
2. When a client requests a value, the sequencer actor checks if it has any values left in its current block. If it does, it returns the next value from memory. No storage operation is needed.
3. When the current block is exhausted, the actor reserves a new block by bumping the high-water mark in the durable record. This is a single Raft compare-and-swap operation. The new block provides another batch of values to serve from memory.

The default block size is 1,000. This means one Raft commit is amortized over 1,000 values. The throughput of the sequencer is roughly 1,000 times higher than a naive per-value approach.

6.4.3 The Trade-Off: Gaps

Block allocation introduces the possibility of gaps in the sequence. If a node reserves a block of 1,000 values and only issues 50 before a restart or a leadership change, the remaining 950 values in that block are never issued. The next block starts from the new high-water mark.

For example:

1. The sequencer reserves block [1, 1000].
2. It issues values 1 through 50 to clients.
3. The node restarts.
4. The sequencer reserves a new block [1001, 2000].

5. Values 51 through 1000 are never issued.

This is the same trade-off that PostgreSQL makes with CACHE on sequences, or that SQL Server makes with sequence caching. The values are unique and monotonically increasing, but they are not contiguous.

6.4.4 When Gaps Are Not Acceptable

If your application requires gap-free numbering (some financial regulations require contiguous invoice numbers), set the block size to 1:

```
kahuna-server --sequencer-block-size 1
```

With a block size of 1, every allocation requires a Raft round trip. This is slower but produces no gaps. The right choice depends on your requirements.

6.4.5 Block Lease Revalidation

To bound stale-leader exposure, each block has a lease duration (default: 5 seconds). If a block sits in memory longer than the lease, the sequencer revalidates it against the durable record before serving more values. This prevents a stale leader from issuing values that conflict with values issued by a new leader.

6.5 Increment

The increment parameter controls the step size between values. With the default increment of 1, values go 1, 2, 3, 4. With an increment of 5:

```
KahunaSequence seq = await client.CreateSequence(
    "batch-ids", initialValue: 0, increment: 5);

long a = await client.NextSequenceValue("batch-ids"); // 5
long b = await client.NextSequenceValue("batch-ids"); // 10
long c = await client.NextSequenceValue("batch-ids"); // 15
```

The first value issued is `initialValue + increment`. Subsequent values increase by `increment`.

6.6 Maximum Value

Sequences can have an optional maximum. When the sequence reaches its maximum, further allocations fail:

```
KahunaSequence seq = await client.CreateSequence(
    "limited-ids", initialValue: 0, increment: 1, maxValue: 100);

// Allocate values until the sequence is exhausted.
for (int i = 0; i < 200; i++)
{
    try
    {
        long val = await client.NextSequenceValue("limited-ids");
        Console.WriteLine(val);
    }
    catch (KahunaException ex)
```

```

    {
        Console.WriteLine($"Sequence exhausted: {ex.Message}");
        break;
    }
}

```

When the maximum is reached, the client throws a `KahunaException`. The sequence does not wrap around.

If no maximum is set (the default), the sequence can grow until it reaches `long.MaxValue`.

6.7 Idempotent Allocation

In a distributed system, a client might send a request, experience a timeout, and retry. Without protection, the retry allocates new values. The client does not know whether the original request succeeded, so it may end up with two sets of values for the same logical operation.

Kahuna solves this with idempotency keys. Pass a key with your allocation request:

```

long id = await client.NextSequenceValue(
    "order-ids",
    idempotencyKey: "checkout-request-abc123"
);

```

If you retry with the same idempotency key, Kahuna returns the same value that was allocated on the first call. No new value is consumed. This is true even if a leadership change occurred between the first call and the retry, because the idempotent result is stored in the durable record.

Idempotent reserves work the same way:

```

KahunaSequenceRange range = await client.ReserveSequenceRange(
    "order-ids",
    count: 10,
    idempotencyKey: "batch-import-42"
);

```

Retrying with the same key and the same count returns the identical range. If you retry with the same key but a different count, Kahuna returns an `InvalidInput` error.

6.7.1 Retention Limits

Idempotency entries are retained for a limited time and number:

Setting	Default	Description
<code>--sequencer-idempotency-retention-max</code>	256	Maximum idempotency entries per sequence
<code>--sequencer-idempotency-retention-ttl</code>	600 seconds	Time window for idempotent replay

After an entry is evicted (by age or count), a retry with that key allocates fresh values instead of replaying the original allocation. Set these values based on how long your retry windows last.

6.8 Querying a Sequence

Read the metadata of an existing sequence:

```
KahunaSequence? seq = await client.GetSequence("order-ids");

if (seq != null)
{
    Console.WriteLine($"Name: {seq.Name}");
    Console.WriteLine($"Current value: {seq.CurrentValue}");
    Console.WriteLine($"Initial value: {seq.InitialValue}");
    Console.WriteLine($"Increment: {seq.Increment}");
    Console.WriteLine($"Max value: {seq.MaxValue}");
    Console.WriteLine($"Revision: {seq.Revision}");
}
```

The `CurrentValue` property is the high-water mark: the highest value that has been reserved (by block allocation), not the last value issued to a client. It may be higher than the last value any client received.

If the sequence does not exist, `GetSequence` returns null.

6.9 Deleting a Sequence

```
bool deleted = await client.DeleteSequence("order-ids");

if (deleted)
    Console.WriteLine("Sequence deleted.");
else
    Console.WriteLine("Sequence not found.");
```

Deleting a sequence removes its durable record. If you create a sequence with the same name afterward, it starts fresh from its initial value.

6.10 CLI Commands

Operation	Command
Create	kahuna-cli --create-sequence "name"
Create with options	kahuna-cli --create-sequence "name" --initial-value 1000 --increment 5 --max-value 99999
Get metadata	kahuna-cli --get-sequence "name"
Next value	kahuna-cli --next-sequence "name"
Next with idempotency	kahuna-cli --next-sequence "name" --idempotency-key "req-123"
Reserve range	kahuna-cli --reserve-sequence "name" --count 100
Delete	kahuna-cli --delete-sequence "name"

6.11 Server Configuration

These server-side settings control sequencer behavior:

Setting	Default	Description
--sequencer-block-size	1000	Values reserved per Raft commit. 1 = gap-free (one commit per value).
--sequencer-block-lease	5 seconds	Time before a cached block is revalidated against the durable record.
--sequencer-workers	128	Number of sequencer actor workers.
--sequencer-max-sequences-per-actor	10000	Maximum sequences one actor keeps in memory. Least recently used sequences are evicted.
--sequencer-idempotency-retention-max	256	Maximum idempotency entries per sequence record.
--sequencer-idempotency-retention-ttl	600 seconds	How long idempotent replays survive.

6.12 Use Cases

6.12.1 Order Numbers

```
await client.CreateSequence("orders", initialValue: 10000);

// In the order placement handler:
long orderId = await client.NextSequenceValue(
    "orders",
    idempotencyKey: $"place-order-{requestId}"
);
await SaveOrder(orderId, orderDetails);
```

The idempotency key ensures that a retried placement does not consume a second order number.

6.12.2 Event Offsets

```
await client.CreateSequence("event-log-offset");

// When appending an event:
KahunaSequenceRange batch = await client.ReserveSequenceRange("event-log-offset", count: events.Count);

long offset = batch.Start;
foreach (var evt in events)
{
    evt.Offset = offset++;
}
```



```
    await AppendEvent(evt);
}
```

Reserving a range gives each event in the batch a unique, contiguous offset within that batch.

6.12.3 Sharded ID Generation

Multiple sequences can generate IDs for different shards:

```
// One sequence per shard, with non-overlapping ranges via increment and initial value.
await client.CreateSequence("ids-shard-0", initialValue: 0, increment: 4);
await client.CreateSequence("ids-shard-1", initialValue: 1, increment: 4);
await client.CreateSequence("ids-shard-2", initialValue: 2, increment: 4);
await client.CreateSequence("ids-shard-3", initialValue: 3, increment: 4);

// Shard 0 produces: 4, 8, 12, 16, ...
// Shard 1 produces: 5, 9, 13, 17, ...
// Shard 2 produces: 6, 10, 14, 18, ...
// Shard 3 produces: 7, 11, 15, 19, ...
```

Each shard generates IDs independently. The increment of 4 and staggered initial values ensure the ranges never overlap.

6.13 Guarantees and Non-Guarantees

The sequencer provides these guarantees:

- **Uniqueness.** A value is issued at most once for the lifetime of the sequence, across all clients, nodes, and leadership changes.
- **Monotonic increase.** Values always climb. A later allocation returns a higher number than an earlier one (within the same ownership period).
- **Contiguous within a reserve.** A `ReserveSequenceRange(count: N)` call returns exactly N consecutive values.

The sequencer does not guarantee:

- **No gaps.** Block allocation can leave gaps on restart, eviction, or leadership change. Set `--sequencer-block-size 1` for gap-free numbering at the cost of throughput.
- **Strict ordering across leadership changes.** During a leadership transition, the new leader starts from the next block boundary. This preserves uniqueness and monotonicity but may skip values.

6.14 Summary

Kahuna's distributed sequencer generates unique, monotonically increasing numbers across the cluster. Block-based allocation amortizes Raft consensus over many values, trading contiguity for throughput. Idempotency keys protect against duplicate allocation on retries. Sequences support custom increments, maximum values, and range reservations for batch workloads.

The next chapter covers transactions: how to read and write multiple keys atomically, with snapshot isolation and conflict detection.

7 Transactions

The previous chapters covered operations on individual keys, locks, and sequences. Each operation targets a single resource and completes in one round trip. Many real workloads need more than that. Transferring a balance between two accounts requires reading both keys, checking constraints, and writing both keys as a single atomic unit. If the process crashes after writing one key but before writing the other, the data is inconsistent.

Kahuna supports multi-key transactions with snapshot isolation. A transaction groups multiple reads and writes into a unit that either commits entirely or rolls back entirely. No partial results are visible to other clients.

Kahuna offers two transaction models:

- **Script transactions** send a self-contained script to the server. The server parses, plans, and executes the entire script as one atomic unit. This is the simpler model and the better choice when the logic can be expressed as a short sequence of key-value operations.
- **Interactive transactions** open a session between the client and the server. The client sends operations one at a time within the session, reads results, makes decisions in application code, and then commits or rolls back. This model is necessary when the transaction logic depends on intermediate results that only the application can evaluate.

Both models use two-phase commit (2PC) internally when the transaction touches keys on multiple partitions.

In both models, the server is the transaction coordinator. The client sends operations and a commit request, but the server drives the 2PC protocol and applies the writes. If the client crashes after it sends the commit request, the server still completes the transaction. The commit decision does not depend on the client remaining connected. This makes Kahuna transactions resilient to client failures at the critical moment.

7.1 Script Transactions

A script transaction is a string of key-value commands that the server executes atomically. If any command fails, the entire transaction is aborted and no changes are applied.

7.1.1 A Simple Script

```
string script = @"
    SET accounts/alice '950'
    SET accounts/bob '1050'
";

KahunaKeyValueTransactionResult result =
    await client.ExecuteKeyValueTransactionScript(script);

Console.WriteLine(result.Type); // Set
```

This script writes two keys in a single atomic operation. Both writes succeed together or fail together. The `Type` property on the result indicates the outcome of the last command in the script.

7.1.2 Parameters

Hard-coding values into a script string is inconvenient and error-prone. Use parameters to pass values at runtime:

```
string script = "SET @key @value";

var parameters = new List<KeyValueParameter>
{
    new() { Key = "@key", Value = "accounts/alice" },
    new() { Key = "@value", Value = "950" }
};

KahunaKeyValueTransactionResult result =
    await client.ExecuteKeyValueTransactionScript(script, parameters:
parameters);
```

A parameter is a placeholder that starts with @. At execution time, Kahuna replaces each placeholder with the corresponding value from the parameter list.

7.1.3 The Result Object

ExecuteKeyValueTransactionScript returns a KahunaKeyValueTransactionResult. This object contains the outcome of the script:

Property	Type	Description
Type	KeyValueResponseType	The response type of the last command
Values	List<...>	List of values returned by GET commands
FirstValue	byte[]?	The value from the first entry in the list
FirstValueAsString	string?	The first value decoded as UTF-8
FirstRevision	long	The revision from the first entry
TimeElapsedMs	int	Server-side execution time

When a script contains one or more GET commands, the returned values appear in the Values list in order. Each entry carries the key, value, revision, expiration time, and last modification timestamp.

```
string script = @"
    SET users/1 'Alice'
    SET users/2 'Bob'
    GET users/1
";

KahunaKeyValueTransactionResult result =
    await client.ExecuteKeyValueTransactionScript(script);

Console.WriteLine(result.FirstValueAsString); // Alice
```

7.1.4 Pre-Hashing for Reuse

Every time you call `ExecuteKeyValueTransactionScript` with a string script, the server must parse it. If you execute the same script many times (with different parameters), you can avoid repeated parsing by loading the script once:

```
KahunaTransactionScript transferScript = client.LoadTransactionScript(
    "SET @from @fromBalance SET @to @toBalance"
);
```

`LoadTransactionScript` computes a Blake3 hash of the script text. On the first execution, the server parses the script and caches the result keyed by this hash. On subsequent executions, the server finds the cached plan and skips parsing.

Execute the pre-hashed script with `Run`:

```
KahunaKeyValueTransactionResult result = await transferScript.Run(
    parameters: new()
    {
        new() { Key = "@from", Value = "accounts/alice" },
        new() { Key = "@fromBalance", Value = "900" },
        new() { Key = "@to", Value = "accounts/bob" },
        new() { Key = "@toBalance", Value = "1100" }
    }
);
```

7.1.5 Script Priority

By default, script transactions run at Normal priority. When the server is at its concurrency ceiling, you can influence which transactions start first:

```
KahunaKeyValueTransactionResult result = await transferScript.Run(
    priority: TransactionPriority.High,
    parameters: new()
    {
        new() { Key = "@key", Value = "config/critical" },
        new() { Key = "@value", Value = "updated" }
    }
);
```

Priority levels, from lowest to highest: Background, Low, Normal, High, Critical. Priority only affects admission order when the server is saturated. Below the concurrency ceiling, all transactions start immediately regardless of priority.

The next chapter covers the full script language in detail: variables, conditionals, loops, built-in functions, and multi-statement control flow.

7.2 Interactive Transactions

When the transaction logic requires reading a value from the server, making a decision in your application, and then writing based on that decision, use an interactive transaction session.

7.2.1 Starting a Session

```
await using KahunaTransactionSession session = await
client.StartTransactionSession(
```

```

new KahunaTransactionOptions
{
    Timeout = 5000,
    Locking = KeyValueTransactionLocking.Pessimistic
};

```

`StartTransactionSession` opens a session with the server. The session receives a unique transaction ID and is pinned to a coordinator node for its lifetime. All operations within the session are part of the same transaction.

The session implements `IAsyncDisposable`. When the `await using` block ends:

- If the session is still in Pending status (no explicit commit or rollback), it rolls back automatically.
- If the session was committed or rolled back, disposal is a no-op.

7.2.2 Session Operations

Inside a session, you can perform the same key-value operations as the main `KahunaClient`, but they execute within the transaction scope:

```

await using KahunaTransactionSession session = await
client.StartTransactionSession(
    new KahunaTransactionOptions { Timeout = 5000 }
);

KahunaKeyValue alice = await session.GetKeyValue("accounts/alice");
KahunaKeyValue bob = await session.GetKeyValue("accounts/bob");

long aliceBalance = long.Parse(alice.ValueAsString ?? "0");
long bobBalance = long.Parse(bob.ValueAsString ?? "0");

if (aliceBalance >= 100)
{
    await session.SetKeyValue("accounts/alice", (aliceBalance - 100).ToString());
    await session.SetKeyValue("accounts/bob", (bobBalance + 100).ToString());
    await session.Commit();
}
else
{
    await session.Rollback();
}

```

This transaction reads two balances, checks a constraint in application code, and writes updated balances. The reads and writes are part of a single atomic unit. If another client modifies either account between the reads and the commit, the transaction may be aborted (depending on the locking mode and read validation settings).

The session supports these operations:

Operation	Method
Set a key	<code>SetKeyValue(key, value, ...)</code>
Get a key	<code>GetKeyValue(key, ...)</code>
Check existence	<code>ExistsKeyValue(key, ...)</code>
Extend TTL	<code>ExtendKeyValue(key, expiresMs, ...)</code>
Delete a key	<code>DeleteKeyValue(key, ...)</code>
Delete many keys	<code>DeleteManyKeyValues(keys, ...)</code>
Get by prefix	<code>GetByBucket(prefixKey, ...)</code>
Get by range	<code>GetByRange(prefix, startKey, ..., endKey, ...)</code>
Compare-value-and-swap	<code>TryCompareValueAndSetKeyValue(key, value, compareValue, ...)</code>
Compare-revision-and-swap	<code>TryCompareRevisionAndSetKeyValue(key, value, compareRevision, ...)</code>

All operations check that the session is still in Pending status. If you try to operate on a committed, rolled back, or aborted session, the client throws a `KahunaException`.

7.2.3 Committing

Call `Commit` to finalize the transaction:

```
bool committed = await session.Commit();
```

The commit drives a two-phase commit protocol across all partitions that the transaction touched. If the commit succeeds, `committed` is true and the session status moves to `Committed`. All writes become visible to other clients.

If a conflict is detected (another transaction modified a key that this transaction read or wrote), the commit throws a `KahunaException` with an `Aborted` error code. The session moves to the `Aborted` status. This is terminal: you cannot retry a commit on an aborted session. Start a new session instead.

If a transient error occurs (network timeout, temporary leader unavailability), the session returns to `Pending` status. You can retry the commit.

7.2.4 Rolling Back

Call `Rollback` to discard all changes:

```
await session.Rollback();
```

After a rollback, the session status is `Rolledback`. All acquired locks are released and no writes from this session are applied.

If you do not call either `Commit` or `Rollback`, the `await using` block calls `Rollback` automatically when the session is disposed.

7.2.5 Transaction Status

A session moves through these states:

Status	Meaning
Pending	The session is active. Operations, commit, and rollback are allowed.
Finalizing	A commit or rollback is in flight. No new operations are accepted. If the finalize fails, the session returns to Pending.
Committed	The transaction committed. Terminal state.
Rolledback	The transaction rolled back. Terminal state.
Aborted	A commit was definitively rejected (conflict or permanent 2PC failure). Terminal state.

7.3 Locking Modes

Kahuna interactive sessions support two locking strategies: pessimistic (the default) and optimistic.

7.3.1 Pessimistic Locking

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Pessimistic
}
```

In pessimistic mode, the session acquires an exclusive lock on every key it reads or writes. This means:

- `GetKeyValue("k")` acquires an exclusive lock on `k` before reading.
- `SetKeyValue("k", "v")` acquires an exclusive lock on `k` before writing.
- `GetByBucket("prefix/")` acquires an exclusive prefix lock before scanning.
- `GetByRange(...)` acquires an exclusive range lock before scanning.

No other transaction can read or write a locked key until this transaction commits or rolls back. This prevents conflicts at the cost of concurrency: if two transactions touch the same key, one waits (or aborts) while the other holds the lock.

Pessimistic locking is the safer default. Use it when conflicts are common or when you cannot tolerate aborted transactions.

7.3.2 Optimistic Locking

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Optimistic
}
```

In optimistic mode, the session acquires exclusive locks only on writes, not on reads:

- `GetKeyValue("k")` reads without locking.
- `SetKeyValue("k", "v")` acquires an exclusive lock on `k` before writing.

This allows higher concurrency for read-heavy workloads. Multiple transactions can read the same keys simultaneously. Conflicts are detected at commit time: if a key that this transaction read was modified by another transaction after the read, the commit is rejected.

Optimistic locking works best when conflicts are rare. If conflicts are frequent, transactions abort and retry repeatedly, which wastes work.

7.4 Read Validation and Write-Skew Detection

By default (`ReadValidation.None`), the commit does not check whether keys that the transaction read were modified by other transactions. This is efficient but allows a class of anomaly called write-skew.

7.4.1 The Write-Skew Problem

Consider two doctors on call. A business rule says at least one doctor must remain on call. Both doctors check the on-call roster, see the other is on call, and each removes themselves. Both transactions commit because neither wrote to the same key. The result: zero doctors on call.

7.4.2 Enabling Read Validation

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Optimistic,
    ReadValidation = ReadValidation.TrackAndValidate
}
```

With `TrackAndValidate`, the session records every key it reads (the read set). At commit time, the server checks whether any key in the read set was modified after this transaction read it. If so, the commit is aborted. This prevents write-skew at the cost of additional validation work during commit.

`TrackAndValidate` is most useful with optimistic locking. Pessimistic locking already prevents concurrent modifications through exclusive locks, so read validation adds little value in that mode.

7.5 RetryableTransaction

Many applications follow the same pattern: start a session, do work, commit, and retry on conflict. Kahuna provides a helper that encapsulates this pattern:

```
await client.RetryableTransaction(
    new KahunaTransactionOptions
    {
        Timeout = 5000,
        Locking = KeyValueTransactionLocking.Pessimistic
    },
    async (session, ct) =>
    {
        KahunaKeyValue counter = await session.GetKeyValue("stats/visits");
        long count = long.Parse(counter.ValueAsString ?? "0");
        await session.SetKeyValue("stats/visits", (count + 1).ToString());
        await session.Commit(ct);
    }
);
```



```
    }  
);
```

`RetryableTransaction` starts a session, runs your callback, and handles retry logic automatically. If the callback throws a `KahunaException` with an `Aborted`, `MustRetry`, or `AlreadyLocked` error code, the method waits and retries with a new session.

The retry strategy uses decorrelated jitter backoff:

- Up to 10 retries.
- The first retry delay has a median of 50 milliseconds.
- Each subsequent delay grows with jitter but is capped at 500 milliseconds.
- If all 10 retries fail, the method throws a `KahunaException` with an `Aborted` error code.

Any exception that is not `Aborted`, `MustRetry`, or `AlreadyLocked` propagates immediately without retry. This includes application logic errors, cancellation, and permanent failures.

7.6 Transaction Options Reference

Option	Type	Default	Description
Timeout	int	5000 ms	How long the transaction may live once started. The server releases the session and its locks when the timeout elapses.
AdmissionWaitMs	int	0 (server default)	How long the client waits for an admission slot when the server is at its session ceiling. If the wait is exhausted, the call fails with <code>AdmissionRefused</code> .
LockingKeyValueTypeTransactionLocking	Pessimistic		Locking strategy: Pessimistic locks on reads and writes, Optimistic locks on writes only.
AutoCommit	bool	true	Whether the server should auto-commit the transaction on dispose.
ReadValidation	ReadValidation	None	None skips read-set validation. <code>TrackAndValidate</code> checks for write-skew at commit time.
DecisionDurability	DecisionDurability	BestEffort	<code>BestEffort</code> returns the outcome to the client before the 2PC decision record is durably replicated. <code>Durable</code> waits until the decision is committed through Raft.
ReadTimestamp	HLCTimestamp	0 (latest)	A snapshot timestamp for reads. Zero means reads observe the current committed state. A non-zero value pins reads to that point in time.
Priority	TransactionPriority	Normal	Admission priority when the server is saturated. Levels: Background, Low, Normal, High, Critical.

7.6.1 Decision Durability

The `DecisionDurability` option controls how the server handles the 2PC decision record:

- **BestEffort** (default): The server tells the client the outcome as soon as the two-phase commit protocol completes in memory. The decision is persisted asynchronously. This is

faster, but in a narrow crash window the decision could be lost and the transaction outcome becomes unknown.

- **Durable:** The server persists the decision record through Raft consensus before returning the outcome to the client. This adds latency but guarantees that the outcome survives a crash.

For most workloads, `BestEffort` is sufficient. Use `Durable` when you need an external guarantee that a committed transaction will remain committed even if the coordinator node crashes immediately after the commit response.

7.6.2 Snapshot Reads

The `ReadTimestamp` option pins all reads in the session to a specific point in time. This is useful for reporting queries that must see a consistent snapshot while other transactions continue to write:

```
await using KahunaTransactionSession session = await
client.StartTransactionSession(
    new KahunaTransactionOptions
    {
        Timeout = 10000,
        Locking = KeyValueTransactionLocking.Optimistic,
        ReadTimestamp = snapshotTimestamp
    }
);
```

```
KahunaKeyValue balance = await session.GetKeyValue("accounts/alice");
```

When `ReadTimestamp` is non-zero, the server serves the value as of that timestamp without recording a read dependency. This means the read does not participate in conflict detection, because it reads historical data that cannot be changed.

7.7 Common Patterns

7.7.1 Read-Modify-Write

The most common transaction pattern: read a value, compute a new value, and write it back.

```
await client.RetryableTransaction(
    new KahunaTransactionOptions { Timeout = 5000 },
    async (session, ct) =>
    {
        KahunaKeyValue item = await session.GetKeyValue("inventory/widget-a");
        int stock = int.Parse(item.ValueAsString ?? "0");

        if (stock < 1)
            throw new InvalidOperationException("Out of stock.");

        await session.SetKeyValue("inventory/widget-a", (stock - 1).ToString());
        await session.Commit(ct);
    }
);
```

Wrapping this in `RetryableTransaction` ensures that if another client modifies the same key concurrently, the transaction retries with fresh data.

7.7.2 Multi-Key Transfer

Move a value from one key to another atomically:

```
await client.RetryableTransaction(
    new KahunaTransactionOptions { Timeout = 5000 },
    async (session, ct) =>
    {
        KahunaKeyValue from = await session.GetKeyValue("accounts/alice");
        KahunaKeyValue to = await session.GetKeyValue("accounts/bob");

        long fromBalance = long.Parse(from.ValueAsString ?? "0");
        long toBalance = long.Parse(to.ValueAsString ?? "0");
        long amount = 100;

        if (fromBalance < amount)
            throw new InvalidOperationException("Insufficient funds.");

        await session.SetKeyValue("accounts/alice", (fromBalance -
amount).ToString());
        await session.SetKeyValue("accounts/bob", (toBalance +
amount).ToString());
        await session.Commit(ct);
    }
);
```

If `accounts/alice` and `accounts/bob` live on different partitions, Kahuna uses 2PC to commit both writes atomically.

7.7.3 Conditional Insert

Insert a key only if it does not already exist, as part of a larger transaction:

```
await using KahunaTransactionSession session = await
client.StartTransactionSession(
    new KahunaTransactionOptions { Timeout = 5000 }
);

KahunaKeyValue existing = await session.GetKeyValue("users/alice@example.com");

if (existing.Success)
{
    Console.WriteLine("User already exists.");
    await session.Rollback();
}
else
{
    await session.SetKeyValue("users/alice@example.com", "Alice");
    await session.SetKeyValue("user-count", "1");
}
```

```
    await session.Commit();  
}
```

7.8 Summary

Kahuna provides two transaction models. Script transactions send a self-contained script to the server for atomic execution. Interactive transactions open a session and let the client make decisions based on intermediate results. Both models use 2PC for multi-partition atomicity.

Pessimistic locking (the default) acquires exclusive locks on every read and write, preventing conflicts at the cost of concurrency. Optimistic locking acquires locks only on writes, allowing higher concurrency but detecting conflicts at commit time. Read validation with `TrackAndValidate` prevents write-skew anomalies by checking the read set at commit time.

`RetryableTransaction` wraps the common start, work, commit, retry loop with decorrelated jitter backoff.

The next chapter covers the transaction script language in full: variables, conditionals, loops, built-in functions, and error handling.

8 The Script Language

Chapter 6 introduced script transactions: self-contained programs that the server executes atomically. This chapter is the complete reference for the script language. It covers the syntax, data types, commands, control flow, expressions, built-in functions, and transaction options.

8.1 Language Overview

The Kahuna script language is a small, domain-specific language designed for one purpose: reading and writing key-value data inside a transaction. It is not a general-purpose programming language. It has no file I/O, no networking, no user-defined functions, and no classes. What it does have is direct access to Kahuna's key-value operations, conditional logic, loops, and built-in functions for type checking and data manipulation.

Scripts are case-insensitive. `SET`, `set`, and `Set` are the same command. String literals use single quotes (`'hello'`) or double quotes (`"hello"`). Identifiers that collide with reserved words can be escaped with backticks (``delete``).

A script is a sequence of statements. The server parses the script into an abstract syntax tree, executes the statements in order, and returns the result of the last statement (or the value passed to `RETURN`).

8.2 Data Types

The language supports five data types:

Type	Examples	Notes
Integer	42, -7, 0	64-bit signed integer (long)
Float	3.14, -0.5, 1.0	64-bit double-precision floating point
String	'hello', "world"	UTF-8 strings
Boolean	true, false	Case-insensitive
Null	null	Represents the absence of a value

Arrays are created with the range operator (`1..10`) or returned by commands like `GET BY BUCKET` and `SCAN BY PREFIX`. You cannot construct an array literal directly.

8.3 Variables

Use `LET` to assign a value to a variable:

```
LET x = 42
LET name = 'Alice'
LET total = x + 10
LET found = true
```

Variables are dynamically typed. You can reassign a variable to a different type:

```
LET x = 42
LET x = 'now a string'
```

Variable names follow identifier rules: letters, digits, and underscores, starting with a letter or underscore.

8.4 Key-Value Commands

8.4.1 SET

SET writes a value to a key:

```
SET mykey 'hello world'
```

The key can be an identifier, a string literal, or a placeholder:

```
SET 'users/alice' 'active'
SET @key @value
```

8.4.1.1 SET Flags

Flags modify the behavior of SET. You can combine multiple flags on one command.

NX (Not Exists): Write only if the key does not exist yet.

```
SET mykey 'first' NX
```

If mykey already exists, the SET does not modify it.

XX (Exists): Write only if the key already exists.

```
SET mykey 'updated' XX
```

If mykey does not exist, the SET does nothing.

CMP (Compare Value): Write only if the current value equals the given expression.

```
SET counter 'closed' CMP 'open'
```

This sets counter to 'closed' only if its current value is 'open'.

CMPREV (Compare Revision): Write only if the current revision equals the given number.

```
SET config 'new-value' CMPREV 5
```

This sets config only if its revision is exactly 5. This is useful for optimistic concurrency: read the revision, do your work, then write only if nobody else changed the key.

EX (Expires): Set a TTL in milliseconds.

```
SET session 'token-abc' EX 30000
```

The key expires and is deleted after 30 seconds.

NOREV (No Revision): Do not track the revision for this write.

```
SET cache/item 'data' NOREV
```

This is useful for cache-like workloads where you do not need version tracking.

Combining flags:

```
SET mykey 'value' NX EX 60000
```

This creates the key only if it does not exist, with a 60-second TTL.

8.4.2 ESET

ESET is the ephemeral variant of SET. It works the same way but stores the value in memory only, without Raft replication. Ephemeral data does not survive a node restart.

```
ESET cache/user 'data' EX 5000
```

ESET supports all the same flags as SET: NX, XX, CMP, CMPREV, EX, NOREV.

8.4.3 GET

GET reads a value by key:

```
GET mykey
```

To capture the result in a variable, use LET:

```
LET value = GET mykey
```

8.4.3.1 Reading a Specific Revision

Read the value at a specific revision number:

```
LET old = GET mykey AT 3
```

This returns the value that mykey had at revision 3.

8.4.3.2 Snapshot Reads

Read the value as it was at a specific point in time (Unix timestamp in milliseconds):

```
LET snapshot = GET mykey AS 0F 17000000000000
```

This returns the value that mykey had at the given timestamp.

8.4.4 EGET

EGET reads from ephemeral storage:

```
LET cached = EGET cache/user
```

EGET supports the same AT and AS 0F variants as GET.

8.4.5 EXISTS

Check whether a key exists:

```
LET found = EXISTS mykey
```

EXISTS also supports AT (revision) and AS 0F (timestamp) variants.

8.4.6 EEXISTS

Check whether an ephemeral key exists:

```
LET found = EEXISTS cache/user
```

8.4.7 DELETE

Delete a key:

```
DELETE mykey
```

8.4.8 EDELETE

Delete an ephemeral key:


```
EDELETE cache/user
```

8.4.9 EXTEND

Extend the TTL of a key by a given number of milliseconds:

```
EXTEND mykey 30000
```

This resets the expiry clock. The key will live for another 30 seconds from now.

8.4.10 EEXTEND

Extend the TTL of an ephemeral key:

```
EEXTEND cache/user 5000
```

8.4.11 GET BY BUCKET

Retrieve all keys that share a common prefix (bucket). The prefix is everything up to and including the last / in the key name:

```
LET items = GET BY BUCKET 'users/'
```

This returns an array of key-value results for all keys whose names start with users/. You can iterate over the results with a FOR loop.

GET BY BUCKET supports AS OF for snapshot reads:

```
LET items = GET BY BUCKET 'users/' AS OF 1700000000000
```

8.4.12 EGET BY BUCKET

The ephemeral variant:

```
LET items = EGET BY BUCKET 'cache/'
```

8.4.13 SCAN BY PREFIX

Scan all keys that match a given prefix:

```
LET results = SCAN BY PREFIX 'config/'
```

Like GET BY BUCKET, this returns an array. It supports AS OF for snapshot reads.

8.4.14 ESCAN BY PREFIX

The ephemeral variant:

```
LET results = ESCAN BY PREFIX 'cache/config/'
```

8.5 Placeholders

Placeholders let you pass values into a script from the calling application. A placeholder starts with @:

```
SET @key @value EX @ttl
```

In C#, pass placeholders as a list of KeyValueParameter objects:

```
string script = "SET @key @value EX @ttl";

var result = await client.ExecuteKeyValueTransactionScript(
    script,
    parameters: [
```

```

    new() { Key = "@key", Value = "users/alice" },
    new() { Key = "@value", Value = "active" },
    new() { Key = "@ttl", Value = "30000" }
  ]
};

```

Placeholders prevent injection and let you reuse the same compiled script with different values. When you use `LoadTransactionScript` (covered in Chapter 6), the server caches the parsed AST and reuses it across calls. Only the placeholder values change.

8.6 Expressions

The language supports arithmetic, comparison, and logical expressions.

8.6.1 Arithmetic

```

LET a = 10 + 5    // 15
LET b = 10 - 3    // 7
LET c = 4 * 3     // 12
LET d = 10 / 2    // 5

```

Arithmetic works on integers and floats. Mixing types promotes the result to float.

8.6.2 Comparison

```

LET eq  = (x = 5)      // equality (single = or ==)
LET neq = (x != 5)     // not equal (or <>)
LET lt  = (x < 10)     // less than
LET gt  = (x > 0)      // greater than
LET lte = (x <= 100)   // less than or equal
LET gte = (x >= 1)     // greater than or equal

```

Both `=` and `==` test equality. Both `!=` and `<>` test inequality.

8.6.3 Logical Operators

```

LET both = (a > 0) && (b > 0) // AND
LET either = (a > 0) || (b > 0) // OR
LET no = !(a > 0)           // NOT

```

You can also use the word forms: `AND`, `OR`, `NOT`.

```
LET both = (a > 0) AND (b > 0)
```

8.6.4 Ranges

The `..` operator creates an array of integers:

```
LET nums = 1..10
```

This creates an array `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`.

8.6.5 Array Indexing

Access array elements by index (zero-based):

```

LET nums = 1..5
LET first = nums[0] // 1
LET third = nums[2] // 3

```

8.6.6 Special Expressions

NOT SET: Evaluates to a sentinel that indicates a key was never written. Use it to check whether a GET returned no data:

```
LET value = GET mykey
IF value = NOT SET THEN
    SET mykey 'default'
END
```

NOT FOUND: Similar to NOT SET, indicates the key was not found:

```
LET value = GET mykey
IF value = NOT FOUND THEN
    SET mykey 'initialized'
END
```

8.7 Control Flow

8.7.1 IF / THEN / ELSE / END

Conditional execution:

```
LET status = GET account/status

IF status = 'active' THEN
    SET account/last-login current_time()
END
```

With an else branch:

```
LET balance = GET account/balance

IF to_long(balance) >= 100 THEN
    SET account/balance to_string(to_long(balance) - 100)
ELSE
    THROW 'Insufficient balance'
END
```

Conditions can use any expression. Nested IF statements are allowed.

8.7.2 FOR / IN / DO / END

Loop over a range or an array:

```
FOR i IN 1..5 DO
    SET concat('key/', to_string(i)) to_string(i * 10)
END
```

This creates five keys: key/1 through key/5 with values 10 through 50.

You can loop over the results of a bucket query:

```
LET items = GET BY BUCKET 'users/'

FOR item IN items DO
```

```
    SET concat('backup/', item) item
END
```

8.8 Transactions in Scripts

8.8.1 Implicit Transactions

When a script contains multiple statements without a BEGIN block, the server wraps the entire script in an auto-commit transaction. If any statement fails, the entire script rolls back.

```
SET account/a '900'
SET account/b '1100'
```

Both writes succeed together or fail together.

8.8.2 Explicit Transactions

Use BEGIN and END to define a transaction block explicitly. Inside the block, you can use COMMIT or ROLLBACK:

```
BEGIN
    LET balance = GET account/a
    LET amount = 100

    IF to_long(balance) >= amount THEN
        SET account/a to_string(to_long(balance) - amount)

        LET target = GET account/b
        SET account/b to_string(to_long(target) + amount)
        COMMIT
    ELSE
        ROLLBACK
    END
END
```

If you call ROLLBACK, all writes inside the BEGIN block are discarded. If you call COMMIT, the writes become durable. If you reach END without calling either, the transaction commits automatically.

8.8.3 Transaction Options

BEGIN accepts options in parentheses:

```
BEGIN (locking = optimistic, timeout = 5000)
    LET a = GET account/a
    LET b = GET account/b
    SET account/a to_string(to_long(a) - 50)
    SET account/b to_string(to_long(b) + 50)
END
```

Available options:

Option	Values	Default	Description
locking	pessimistic, optimistic	pessimistic	Lock acquisition strategy
autoCommit	true, false	true	Commit automatically when the block ends without an explicit COMMIT or ROLL-BACK
asyncRelease	true, false	false	Release locks asynchronously after commit
timeout	integer (ms)	Server default	Maximum execution time for the transaction
admissionWait	integer (ms)	Server default	Maximum time to wait for a transaction slot
snapshot	integer (Unix ms)	0 (disabled)	Read all keys as of this timestamp
priority	background, low, normal, high, critical	normal	Transaction scheduling priority

8.9 RETURN

RETURN ends the script and returns a value to the caller:

```
LET x = GET mykey

IF x = NOT SET THEN
    RETURN 'not found'
END

RETURN concat('found: ', x)
```

Without a RETURN, the script returns the result of the last executed statement.

RETURN without a value ends the script immediately:

```
LET x = GET mykey
IF x = NOT SET THEN
    RETURN
END
SET mykey to_string(to_long(x) + 1)
```

8.10 SLEEP

SLEEP pauses execution for a given number of milliseconds:

```
SLEEP 1000
```

Use SLEEP with caution. It holds the transaction open during the pause. The primary use case is testing and debugging, not production workloads.

8.11 THROW

THROW raises an error and aborts the script:

```
LET balance = GET account/balance

IF to_long(balance) < 0 THEN
    THROW 'Balance cannot be negative'
END
```

The error message is returned to the caller in the transaction result's Reason field.

8.12 Built-in Functions

The language provides built-in functions for type checking, type conversion, string manipulation, math, and metadata access.

8.12.1 Type Checking Functions

These functions test the type of a value and return a boolean:

Function	Description
<code>is_long(x) / is_int(x) / is_integer(x)</code>	True if x is an integer
<code>is_float(x) / is_double(x)</code>	True if x is a float
<code>is_string(x) / is_str(x)</code>	True if x is a string
<code>is_bool(x) / is_boolean(x)</code>	True if x is a boolean
<code>is_null(x)</code>	True if x is null
<code>is_array(x)</code>	True if x is an array

```
LET x = 42
RETURN is_long(x) // true

LET y = 'hello'
RETURN is_string(y) // true

LET z = 1..10
RETURN is_array(z) // true
```

8.12.2 Type Conversion Functions

These functions convert a value from one type to another:

Function	Description
<code>to_long(x)</code> / <code>to_int(x)</code> / <code>to_integer(x)</code> / <code>to_number(x)</code>	Convert to integer
<code>to_float(x)</code> / <code>to_double(x)</code>	Convert to float
<code>to_string(x)</code> / <code>to_str(x)</code>	Convert to string
<code>to_bool(x)</code> / <code>to_boolean(x)</code>	Convert to boolean
<code>to_json(x)</code>	Serialize to a JSON string

Values stored in Kahuna are byte arrays. When you read a value with GET, the result is a string. To do arithmetic on it, convert it to an integer or float first:

```
LET raw = GET counter
LET n = to_long(raw)
SET counter to_string(n + 1)
```

Conversion rules:

- `to_long(1.7)` truncates to 1.
- `to_long(true)` returns 1. `to_long(false)` returns 0.
- `to_float(true)` returns 1. `to_float(false)` returns 0.
- `to_bool(0)` returns false. `to_bool(1)` returns true.
- `to_string(null)` returns an empty string.
- `to_json` works on integers, floats, strings, and arrays. It produces a JSON representation of the value.

8.12.3 String Functions

Function	Description
<code>upper(x)</code>	Convert string to uppercase
<code>lower(x)</code>	Convert string to lowercase
<code>concat(a, b)</code>	Concatenate two strings
<code>len(x)</code> / <code>length(x)</code>	Return the length of a string

```
LET name = 'Alice'
RETURN upper(name)      // 'ALICE'
RETURN lower(name)      // 'alice'
RETURN concat('Hi, ', name) // 'Hi, Alice'
RETURN len(name)        // 5
```

`concat` takes exactly two string arguments. To concatenate more than two values, nest the calls:

```
LET full = concat(concat(first, ' '), last)
```

8.12.4 Math Functions

Function	Description
<code>abs(x)</code>	Absolute value
<code>pow(x, y)</code>	x raised to the power y
<code>round(x)</code>	Round to nearest integer
<code>ceil(x)</code>	Round up
<code>floor(x)</code>	Round down
<code>min(a, b)</code>	Smaller of two values
<code>max(a, b)</code>	Larger of two values

```
LET a = abs(-5) // 5
LET b = pow(2, 10) // 1024
LET c = round(3.7) // 4
LET d = ceil(3.1) // 4
LET e = floor(3.9) // 3
LET f = min(10, 20) // 10
LET g = max(10, 20) // 20
```

8.12.5 Metadata Functions

Function	Description
<code>revision(x) / rev(x)</code>	Return the revision number of a GET result
<code>expires(x)</code>	Return the expiry timestamp of a GET result
<code>current_time()</code>	Return the current UTC time as Unix milliseconds
<code>count(x)</code>	Return the number of elements in an array

```
LET value = GET mykey
LET r = revision(value)
LET e = expires(value)
LET now = current_time()
```

`revision` and `expires` are useful for conditional logic based on a key's metadata:

```
LET value = GET mykey
IF revision(value) > 10 THEN
  SET mykey 'stable'
END
```

`count` works only on arrays:

```
LET items = GET BY BUCKET 'users/'
LET total = count(items)
RETURN to_string(total)
```


8.13 Complete Example: Balance Transfer

This example combines many features: variables, GET, SET with conditions, type conversion, explicit transactions, and error handling.

```
BEGIN (locking = pessimistic, timeout = 5000)
  LET source_balance = GET @source
  LET target_balance = GET @target

  IF source_balance = NOT SET THEN
    THROW concat('Source account not found: ', @source)
  END

  IF target_balance = NOT SET THEN
    THROW concat('Target account not found: ', @target)
  END

  LET amount = to_long(@amount)
  LET src = to_long(source_balance)
  LET tgt = to_long(target_balance)

  IF src < amount THEN
    THROW 'Insufficient balance'
  END

  SET @source to_string(src - amount)
  SET @target to_string(tgt + amount)
  COMMIT
END
```

Call this script from C#:

```
var result = await client.ExecuteKeyValueTransactionScript(
  script,
  parameters: [
    new() { Key = "@source", Value = "account/alice" },
    new() { Key = "@target", Value = "account/bob" },
    new() { Key = "@amount", Value = "500" }
  ]
);
```

The transaction reads both balances, checks the source has enough funds, updates both accounts, and commits. If any step fails or the source balance is too low, the script throws and no writes take effect.

8.14 Complete Example: Batch Initialization

This example uses a FOR loop to initialize a set of keys:

```
FOR i IN 0..9 DO
  LET key = concat('sensor/', to_string(i))
  LET exists_result = EXISTS key
```

```

    IF exists_result = NOT SET THEN
        SET key '0' EX 60000
    END
END

```

This creates keys `sensor/0` through `sensor/9`, each with a 60-second TTL, but only if the key does not exist yet.

8.15 Complete Example: Snapshot Audit

Read the state of several keys as they were at a specific moment:

```

BEGIN (snapshot = @timestamp)
    LET a = GET account/a
    LET b = GET account/b
    LET c = GET account/c
    LET total = to_long(a) + to_long(b) + to_long(c)
    RETURN to_string(total)
END

```

The snapshot option ensures all three reads see a consistent view of the data at the given timestamp, even if the keys changed after that point.

8.16 Script Caching

When you send a script to the server, the server parses it into an AST (abstract syntax tree). Parsing is fast, but for scripts that run frequently, you can avoid the overhead by using `LoadTransactionScript` (covered in Chapter 6). The server hashes the script with Blake3, caches the AST, and reuses it on subsequent calls with the same hash. Placeholder values change between calls, but the structure stays cached.

The server evicts cached scripts when they have not been used within the cache TTL. The TTL is configured with `--script-cache-expiration` (default: 600 seconds).

8.17 Reserved Words

These words are reserved by the language. You cannot use them as bare key names or variable names without backtick escaping:

```

SET, GET, LET, IF, THEN, ELSE, END, FOR, DO, IN, BEGIN, COMMIT, ROLLBACK, RETURN, SLEEP, DELETE,
DEL, EXTEND, EXISTS, ESET, EGET, EDELETE, EDEL, EEXTEND, EEXISTS, TRUE, FALSE, NULL, NX, XX,
EX, CMP, CMPREV, NOREV, THROW, FOUND, NOT, AT, AS, OF, SCAN, ESCAN, BY, BUCKET, PREFIX, AND, OR

```

If your key name collides with a reserved word, use a string literal or backtick escaping:

```

SET 'delete' 'some value'
SET `delete` 'some value'

```

8.18 Operator Precedence

Operators follow this precedence, from lowest to highest:

Precedence	Operator	Description
1 (lowest)	..	Range
2	OR, \ \	Logical OR
3	AND, &&	Logical AND
4	=, ==, !=, <>	Equality
5	<, >, <=, >=	Comparison
6	+, -	Addition, subtraction
7	*, /	Multiplication, division
8	!, NOT	Logical NOT (right-associative)
9 (highest)	[]	Array indexing

Use parentheses to override the default order:

```
LET result = (a + b) * c
```

8.19 Summary

The Kahuna script language provides the building blocks for server-side transaction logic: key-value commands with conditional flags, variables, arithmetic and logical expressions, control flow with IF and FOR, explicit transaction blocks with configurable options, and built-in functions for type checking, conversion, string manipulation, and math. Placeholders separate data from logic, and script caching amortizes parsing cost.

Each persistent command (SET, GET, EXISTS, DELETE, EXTEND) has an ephemeral counterpart (ESET, EGET, EEXISTS, EDELETE, EEXTEND) for in-memory workloads. Bulk reads use GET BY BUCKET and SCAN BY PREFIX.

The next chapter covers real-world patterns: how to combine these primitives to build distributed coordination workflows, leader election, and idempotent processing.

9 Leader Election and Job Coordination

Many distributed applications need exactly one active process to do a specific job. A scheduler that sends reminder emails, a worker that processes a payment queue, a service that aggregates metrics: each of these must run on one node at a time. If two instances run simultaneously, they send duplicate emails, process payments twice, or produce incorrect aggregates.

On a single machine, this is simple. You run one process. If it crashes, a supervisor restarts it. In a distributed system, you run multiple instances for availability, and you need a way to decide which one is the active leader. The others stand by and take over if the leader fails.

This chapter shows how to build leader election and job coordination patterns with Kahuna's locks, fencing tokens, and key-value operations.

9.1 The Simplest Leader Election

A distributed lock is a natural fit for leader election. The process that holds the lock is the leader. All other processes are followers. When the leader fails, the lock's lease expires and a follower acquires it.

Here is the simplest version:

```
while (!stoppingToken.IsCancellationRequested)
{
    await using KahunaLock lockHandle = await client.GetOrCreateLock(
        resource: "leader/email-scheduler",
        expiry: TimeSpan.FromSeconds(15),
        wait: TimeSpan.FromSeconds(10),
        retry: TimeSpan.FromMilliseconds(500),
        durability: LockDurability.Persistent,
        cancellationToken: stoppingToken
    );

    if (lockHandle.IsAcquired)
    {
        Console.WriteLine($"I am the leader. Fencing token: {lockHandle.FencingToken}");
        await RunSchedulerLoop(lockHandle, stoppingToken);
    }
    else
    {
        Console.WriteLine("Another instance is the leader. Retrying...");
        await Task.Delay(2000, stoppingToken);
    }
}
```

Each instance tries to acquire the lock. The winner becomes the leader and runs the scheduler loop. The losers wait and try again. If the leader crashes, the 15-second lease expires and a follower acquires the lock on the next attempt.

This version works, but it has a problem. The leader does work for up to 15 seconds and then the lock expires. If the work takes longer than 15 seconds, another instance acquires the lock and both instances run simultaneously.

9.2 Heartbeat Renewal

The solution is a heartbeat: the leader renews its lease periodically while it does work. The lease duration is a safety net for crashes, not a cap on how long the leader can run.

```

async Task RunAsLeader(KahunaLock lockHandle, CancellationToken stoppingToken)
{
    using CancellationTokenSource leaderCts =
        CancellationTokenSource.CreateLinkedTokenSource(stoppingToken);

    Task renewalTask = RunHeartbeat(lockHandle, leaderCts);

    try
    {
        await DoLeaderWork(lockHandle.FencingToken, leaderCts.Token);
    }
    finally
    {
        leaderCts.Cancel();
        await renewalTask;
    }
}

async Task RunHeartbeat(KahunaLock lockHandle, CancellationTokenSource leaderCts)
{
    try
    {
        while (!leaderCts.Token.IsCancellationRequested)
        {
            await Task.Delay(5000, leaderCts.Token);

            (bool extended, long token) = await
                lockHandle.TryExtend(TimeSpan.FromSeconds(15));

            if (!extended)
            {
                Console.WriteLine("Lost leadership. Lease renewal failed.");
                leaderCts.Cancel();
                return;
            }
        }
    }
    catch (OperationCanceledException)
    {
    }
}

```

```

        // Normal shutdown.
    }
}

```

The heartbeat task renews the lease every 5 seconds. The lease duration is 15 seconds. This gives the leader three chances to renew before the lease expires. If a single renewal fails (network blip, temporary leader unavailability), the leader has 10 more seconds before it loses the lock.

A good rule of thumb: renew at one-third of the lease duration. This gives you two full renewal attempts as a buffer.

9.2.1 What Happens When Renewal Fails

If `TryExtend` returns false, one of these things occurred:

1. The lease expired before the renewal reached the server (network delay, GC pause, slow disk).
2. Another process acquired the lock after the lease expired.

In both cases, the leader must stop its work immediately. The `CancellationTokenSource` propagates the cancellation to all ongoing work. Any downstream operation that respects the cancellation token will stop.

The leader should not attempt to re-acquire the lock in the same iteration. It should release the lock handle, return to the outer loop, and compete for leadership again from scratch.

9.3 The Leader Election Loop

Here is the complete pattern that combines acquisition, heartbeat renewal, and graceful shutdown:

```

public class LeaderElectionService : BackgroundService
{
    private readonly KahunaClient _client;
    private readonly string _resource;
    private readonly TimeSpan _leaseDuration;
    private readonly TimeSpan _renewalInterval;

    public LeaderElectionService(KahunaClient client, string resource)
    {
        _client = client;
        _resource = resource;
        _leaseDuration = TimeSpan.FromSeconds(15);
        _renewalInterval = TimeSpan.FromSeconds(5);
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await using KahunaLock lockHandle = await _client.GetOrCreateLock(
                resource: _resource,

```

```

        expiry: _leaseDuration,
        wait: TimeSpan.FromSeconds(30),
        retry: TimeSpan.FromMilliseconds(500),
        durability: LockDurability.Persistent,
        cancellationToken: stoppingToken
    );

    if (!lockHandle.IsAcquired)
    {
        await Task.Delay(2000, stoppingToken);
        continue;
    }

    Console.WriteLine($"Elected as leader. Token: {lockHandle.FencingToken}");

    using CancellationTokenSource leaderCts =
        CancellationTokenSource.CreateLinkedTokenSource(stoppingToken);

    Task heartbeat = RunHeartbeat(lockHandle, leaderCts);

    try
    {
        await DoLeaderWork(lockHandle.FencingToken, leaderCts.Token);
    }
    catch (OperationCanceledException) when
        (leaderCts.IsCancellationRequested)
    {
        Console.WriteLine("Leadership ended.");
    }
    finally
    {
        leaderCts.Cancel();
        await heartbeat;
    }
}

private async Task RunHeartbeat(KahunaLock lockHandle,
    CancellationTokenSource leaderCts)
{
    try
    {
        while (!leaderCts.Token.IsCancellationRequested)
        {
            await Task.Delay(_renewalInterval, leaderCts.Token);
            (bool ok, _) = await lockHandle.TryExtend(_leaseDuration);
        }
    }
}

```

```

        if (!ok)
        {
            leaderCts.Cancel();
            return;
        }
    }
}

catch (OperationCanceledException) { }
}

private async Task DoLeaderWork(long fencingToken, CancellationToken ct)
{
    while (!ct.IsCancellationRequested)
    {
        await ProcessNextBatch(fencingToken, ct);
    }
}

private async Task ProcessNextBatch(long fencingToken, CancellationToken ct)
{
    // Application-specific work goes here.
    // Pass fencingToken to downstream systems.
    await Task.Delay(1000, ct);
}
}

```

This pattern uses .NET's `BackgroundService` as a host. You can register it in your dependency injection container. It runs for the lifetime of the application.

9.3.1 State Machine

A process in this pattern moves through three states:

1. **Candidate.** The process tries to acquire the lock. It waits and retries until it succeeds or the application shuts down.
2. **Leader.** The process holds the lock and does work. A background task renews the lease. The fencing token identifies this leadership term.
3. **Follower.** The process lost or failed to acquire the lock. It returns to the candidate state and tries again.

The transitions are:

- Candidate to Leader: `GetOrCreateLock` returns `IsAcquired = true`.
- Leader to Follower: `TryExtend` returns false, or the application requests shutdown.
- Follower to Candidate: the outer loop iterates and calls `GetOrCreateLock` again.

9.4 Why Fencing Tokens Matter

Chapter 4 introduced fencing tokens. In leader election, they are essential. Here is why.

9.4.1 The Stale Leader Problem

Consider this sequence of events:

1. Process A acquires the leadership lock. Fencing token: 5.
2. Process A starts processing jobs from a queue.
3. Process A experiences a long garbage collection pause (20 seconds).
4. The lease (15 seconds) expires during the pause. Process A does not know this.
5. Process B acquires the lock. Fencing token: 6.
6. Process B starts processing jobs from the same queue.
7. Process A resumes. It still believes it is the leader. It processes the same jobs that process B is already handling.

The result: duplicate processing. Both processes act as leader simultaneously.

9.4.2 Leader Election Without Fencing (The Wrong Way)

This implementation looks correct but is broken:

```
// INCORRECT: no fencing token check
async Task ProcessJob(KahunaClient client, string jobId)
{
    KahunaKeyValue job = await client.GetKeyValue($"jobs/{jobId}");

    if (job.ValueAsString == "pending")
    {
        await ExecuteJob(jobId);
        await client.SetKeyValue($"jobs/{jobId}", "completed");
    }
}
```

A stale leader (process A) can execute this code after its lease expired. It reads the job, sees “pending”, executes it, and marks it “completed.” Meanwhile, the new leader (process B) also reads the same job as “pending” before A marks it complete. Both execute the job.

9.4.3 Leader Election With Fencing (The Correct Way)

Pass the fencing token to every operation that the leader performs. The downstream system rejects writes from stale leaders:

```
async Task ProcessJob(KahunaClient client, long fencingToken, string jobId)
{
    KahunaKeyValue job = await client.GetKeyValue($"jobs/{jobId}");

    if (job.ValueAsString == "pending")
    {
        await ExecuteJob(jobId);

        // Store the fencing token alongside the job status.
        await client.SetKeyValue(
            $"jobs/{jobId}",
            $"completed|token={fencingToken}"
        );
    }
}
```

For systems that do not natively support fencing (most databases and message queues), you can implement a fence check at the application level:

```
async Task<bool> TryClaimJob(KahunaClient client, long fencingToken, string
jobId)
{
    // Use compare-and-swap to claim the job atomically.
    KahunaKeyValue result = await client.SetKeyValue(
        $"jobs/{jobId}",
        $"running|token={fencingToken}",
        flags: KeyValueFlags.SetIfEqualToValue,
        durability: KeyValueDurability.Persistent
    );

    return result.Success;
}
```

If a stale leader with token 5 tries to claim a job that the new leader (token 6) already claimed, the compare-and-swap fails because the current value does not match “pending.”

9.5 Publishing Leader Identity

Sometimes other services need to know who the current leader is. You can publish leader identity in a key-value entry alongside the lock:

```
if (lockHandle.IsAcquired)
{
    string identity = JsonSerializer.Serialize(new
    {
        host = Environment.MachineName,
        processId = Environment.ProcessId,
        fencingToken = lockHandle.FencingToken,
        electedAt = DateTimeOffset.UtcNow
    });

    await client.SetKeyValue(
        "leader/email-scheduler/identity",
        identity,
        expiryTime: 20000
    );
}
```

Other services can read this key to discover the current leader. Set a TTL slightly longer than the lease duration so the identity key expires shortly after the lock.

Update the identity key during each heartbeat cycle to keep the TTL fresh:

```
while (!leaderCts.Token.IsCancellationRequested)
{
    await Task.Delay(5000, leaderCts.Token);

    (bool ok, _) = await lockHandle.TryExtend(TimeSpan.FromSeconds(15));
}
```

```

    if (!ok)
    {
        leaderCts.Cancel();
        return;
    }

    // Refresh the identity TTL.
    await client.ExtendKeyValue("leader/email-scheduler/identity", 20000);
}

```

9.6 Distributed Task Queue

Leader election puts one process in charge. A task queue distributes work across many processes. Each worker claims a task, processes it, and marks it complete. No two workers process the same task.

9.6.1 Task Structure

Store tasks as key-value entries. Use a sequence for unique task IDs:

```

await client.CreateSequence("task-ids");

async Task<string> EnqueueTask(KahunaClient client, string payload)
{
    long taskId = await client.NextSequenceValue("task-ids");
    string key = $"tasks/{taskId}";

    await client.SetKeyValue(key, JsonSerializer.Serialize(new
    {
        id = taskId,
        payload,
        status = "pending",
        createdAt = DateTimeOffset.UtcNow
    }));

    return key;
}

```

9.6.2 Claiming a Task

A worker claims a task by acquiring a lock on that task's key. The lock lease acts as a processing timeout:

```

async Task<bool> TryProcessTask(KahunaClient client, string taskKey)
{
    await using KahunaLock taskLock = await client.GetOrCreateLock(
        resource: $"lock/{taskKey}",
        expiry: TimeSpan.FromSeconds(60)
    );

    if (!taskLock.IsAcquired)

```

```

        return false;

        KahunaKeyValue task = await client.GetKeyValue(taskKey);
        var taskData = JsonSerializer.Deserialize<TaskRecord>(task.ValueAsString ??
"{}");

        if (taskData?.Status != "pending")
            return false;

        try
        {
            await ExecuteTask(taskData, taskLock.FencingToken);

            await client.SetKeyValue(taskKey, JsonSerializer.Serialize(taskData with
            {
                Status = "completed",
                CompletedAt = DateTimeOffset.UtcNow
            }));

            return true;
        }
        catch (Exception ex)
        {
            await client.SetKeyValue(taskKey, JsonSerializer.Serialize(taskData with
            {
                Status = "failed",
                Error = ex.Message
            }));

            return false;
        }
    }
}

```

If the worker crashes while processing, the lock lease expires after 60 seconds. Another worker can then claim the task. The task stays in “pending” status because the crashed worker never updated it.

9.6.3 Worker Loop

Each worker scans for pending tasks and claims them:

```

async Task RunWorker(KahunaClient client, CancellationToken ct)
{
    while (!ct.IsCancellationRequested)
    {
        List<KahunaKeyValue> tasks = await client.GetByBucket(
            "tasks/",
            KeyValueDurability.Persistent
        );
    }
}

```

```

bool processed = false;

foreach (KahunaKeyValue task in tasks)
{
    var data = JsonSerializer.Deserialize<TaskRecord>(task.ValueAsString ??
"{}");

    if (data?.Status == "pending")
    {
        bool claimed = await TryProcessTask(client, task.Key);
        if (claimed) processed = true;
    }
}

if (!processed)
{
    // No work available. Wait before scanning again.
    await Task.Delay(2000, ct);
}
}

```

Multiple workers can run this loop concurrently. The lock on each task ensures that only one worker processes it. If two workers try to claim the same task, one gets `IsAcquired = false` and moves on.

9.6.4 Extending Task Leases

For long tasks, extend the lock lease during processing (the same pattern as the leader heartbeat):

```

async Task ExecuteLongTask(KahunaLock taskLock, TaskRecord task,
CancellationTokens ct)
{
    foreach (var step in task.Steps)
    {
        await ProcessStep(step, ct);

        (bool ok, _) = await taskLock.TryExtend(TimeSpan.FromSeconds(60));

        if (!ok)
        {
            Console.WriteLine("Lost task lock. Another worker may take over.");
            return;
        }
    }
}

```

9.7 Failure Scenarios

9.7.1 Leader Crashes

1. The leader process terminates (OOM kill, hardware failure, unhandled exception).
2. The `DisposeAsync` finalizer does not run because the process is gone.
3. The lock lease expires after 15 seconds.
4. A follower acquires the lock on its next attempt.
5. The new leader gets a higher fencing token.

No manual intervention is required. The lease is the recovery mechanism.

9.7.2 Slow Leader

1. The leader encounters a slow database query, a full GC, or a network partition.
2. The heartbeat task cannot renew the lease in time.
3. The lease expires. A follower acquires the lock.
4. The slow leader's heartbeat eventually executes and `TryExtend` returns false.
5. The leader cancels its work through the `CancellationTokenSource`.

The window of danger is the time between the lease expiry and the moment the old leader detects the failure. During this window, both the old and new leaders may act. Fencing tokens prevent the old leader from corrupting shared state.

9.7.3 Network Partition

1. The leader can still reach the work (database, queue) but cannot reach Kahuna.
2. The heartbeat fails. The lease expires on the Kahuna side.
3. A follower on the other side of the partition acquires the lock.
4. The old leader continues doing work because it has not detected the partition.

This is the scenario where fencing is critical. Without fencing, both leaders write to the shared resource. With fencing, the shared resource rejects writes from the old leader's stale token.

9.7.4 Worker Crashes Mid-Task

1. A worker acquires a task lock and starts processing.
2. The worker crashes.
3. The task lock expires after 60 seconds.
4. Another worker scans for pending tasks, finds this one, and claims it.
5. The task is processed to completion by the second worker.

For this to work correctly, the task processing must be idempotent or the task must record progress. If the first worker completed half the work before crashing, the second worker must be able to resume or redo the work safely.

9.8 Tuning Leader Election

9.8.1 Lease Duration

Short leases (5 to 10 seconds) provide fast failover but require frequent renewals. Each renewal is a network round trip to Kahuna. If the network is unreliable, short leases increase the risk of losing leadership during a transient failure.

Long leases (30 to 60 seconds) tolerate network blips but delay failover after a crash. A crashed leader blocks progress for the full lease duration.

9.8.2 Renewal Interval

Renew at one-third of the lease duration. This gives two full buffer attempts:

Lease Duration	Renewal Interval	Buffer Before Expiry
9 seconds	3 seconds	6 seconds (2 retries)
15 seconds	5 seconds	10 seconds (2 retries)
30 seconds	10 seconds	20 seconds (2 retries)

9.8.3 Wait and Retry

When a follower tries to acquire the lock, the wait parameter controls how long it blocks. A long wait (30 to 60 seconds) keeps the follower ready to take over quickly. A short wait (5 to 10 seconds) with external polling gives you more control over shutdown behavior.

The retry interval (the polling frequency during the wait period) should be short enough to detect a released lock quickly but long enough to avoid excessive load. Values of 200 to 500 milliseconds work well for most workloads.

9.9 Combining Leader Election with Task Queues

A common architecture uses both patterns together: a single leader distributes work, and multiple workers process it.

```
async Task DoLeaderWork(long fencingToken, CancellationToken ct)
{
    while (!ct.IsCancellationRequested)
    {
        // The leader generates tasks.
        List<string> pendingItems = await FetchNewItemsFromExternalSource(ct);

        foreach (string item in pendingItems)
        {
            long taskId = await _client.NextSequenceValue("task-ids");

            await _client.SetKeyValue($"tasks/{taskId}",
                JsonSerializer.Serialize(new
                {
                    id = taskId,
                    payload = item,
                    status = "pending",
                    assignedBy = fencingToken
                }));
        }

        await Task.Delay(5000, ct);
    }
}
```

The leader creates tasks. Workers (which do not need to win an election) claim and process tasks independently. If the leader crashes, a new leader takes over task creation. The workers are unaffected because they operate on the task queue, not on the leadership lock.

9.10 Summary

Leader election in Kahuna uses a distributed lock as the coordination primitive. The process that holds the lock is the leader. A background heartbeat task renews the lease to keep leadership alive. When the leader fails, the lease expires and a follower acquires the lock automatically.

Fencing tokens prevent stale leaders from corrupting shared state during the window between lease expiry and failure detection. Every downstream write should carry the fencing token and reject stale values.

The same lock primitive supports task queues: each task gets its own lock, workers claim tasks by acquiring the lock, and crashed workers release their tasks through lease expiry.

The next chapter covers idempotency and duplicate prevention: how to make operations safe to retry without side effects.

10 Idempotency and Duplicate Prevention

In a distributed system, retries are not optional. They are inevitable. A client sends a request to the server. The network drops the response. The client does not know whether the server processed the request or not. It retries.

If the operation was “set the status to completed,” the retry is harmless. The status was already “completed,” and setting it again changes nothing. But if the operation was “add 100 to the balance,” the retry doubles the effect. The balance increases by 200 instead of 100.

An operation is idempotent when executing it multiple times produces the same result as executing it once. This chapter shows how to build idempotent operations with Kahuna’s key-value primitives, conditional writes, fencing tokens, and sequence-based deduplication.

10.1 The Retry Problem

Consider a simple counter increment:

```
// INCORRECT: not idempotent
KahunaKeyValue current = await client.GetKeyValue("stats/page-views");
long count = long.Parse(current.ValueAsString ?? "0");
await client.SetKeyValue("stats/page-views", (count + 1).ToString());
```

This code reads the counter, adds 1, and writes the new value. If the client retries (because the response to the write was lost), it reads the already-incremented counter, adds 1 again, and writes a doubly-incremented value.

The problem is not the retry itself. The problem is that the operation has no way to detect that it already succeeded.

10.2 Delivery Semantics

Distributed systems describe three levels of delivery guarantee:

- **At-most-once.** The operation executes zero or one time. If it fails, the system does not retry. Data may be lost, but it is never duplicated.
- **At-least-once.** The operation executes one or more times. The system retries on failure. Data is never lost, but it may be duplicated.
- **Exactly-once.** The operation executes exactly one time. This is the ideal, but it is impossible to achieve in the general case without additional mechanisms.

Most systems provide at-least-once delivery by default: the client retries until it receives a success response. To get effectively-exactly-once behavior, the operation must be idempotent. A retry of an idempotent operation produces no additional effect, so “at least once” becomes equivalent to “exactly once” in practice.

10.3 Naturally Idempotent Operations

Some operations are idempotent by nature. No extra work is required:

- **Unconditional SET.** `SetKeyValue("config/mode", "maintenance")` can be called any number of times. The result is always the same: the key holds the value “maintenance.”
- **DELETE.** `DeleteKeyValue("temp/session-abc")` succeeds on the first call and returns “not found” on subsequent calls. Either way, the key is gone.

- **SetIfExists.** `SetKeyValue("users/alice", "...", flags: KeyValueFlags.SetIfExists)` creates the key once. Retries see the key already exists and return `Success = false`. No duplicate is created.

These operations are safe to retry without any guarding mechanism.

10.4 Non-Idempotent Operations

These operations are dangerous to retry:

- **Read-modify-write.** Read a value, compute a new value, write it back. A retry reads the already-modified value and modifies it again.
- **Append.** Add an item to a list or log. A retry adds the item twice.
- **Increment.** Add a number to a counter. A retry adds the number twice.
- **Side effects.** Send an email, charge a credit card, enqueue a message. A retry repeats the side effect.

Each of these needs an explicit mechanism to make it safe for retries.

10.5 Revision-Based Idempotency (CRAS)

Every key in Kahuna has a revision number. The revision starts at 0 and increases by 1 on every successful write. You can use this revision as a guard: “write this value, but only if the revision is still what I saw when I read it.”

This is the Compare-Revision-And-Swap (CRAS) operation:

```
// Step 1: Read the current value and its revision.
KahunaKeyValue current = await client.GetKeyValue("stats/page-views");
long count = long.Parse(current.ValueAsString ?? "0");
long revision = current.Revision;

// Step 2: Write the new value, but only if the revision has not changed.
KahunaKeyValue result = await client.TryCompareRevisionAndSetKeyValue(
    "stats/page-views",
    (count + 1).ToString(),
    compareRevision: revision
);

if (result.Success)
    Console.WriteLine("Counter incremented.");
else
    Console.WriteLine("Conflict: another write changed the key. Retry from step 1.");
```

How this prevents double-counting:

1. The client reads revision 5 and value “100.”
2. The client writes “101” with `compareRevision: 5`.
3. The write succeeds. The revision becomes 6.
4. The client retries (response was lost). It writes “101” with `compareRevision: 5` again.
5. The revision is now 6, not 5. The compare fails. The retry is rejected.

The retry does not corrupt the data. The client can re-read the key, see that the value is already “101,” and conclude that the original write succeeded.

10.5.1 When CRAS Is Not Enough

CRAS protects against concurrent writes and duplicate retries from the same read. But if the client crashes after reading and before writing, it restarts with no memory of the previous read. The new read sees the current state, and the new write succeeds normally. This is correct behavior, not a bug, because the first attempt never wrote anything.

The risk arises when the operation has external side effects. If the client charged a credit card, crashed, and retried, the charge happened but the status update did not. CRAS does not help here because the external side effect is outside Kahuna’s control. For that, you need a request-level idempotency key (covered later in this chapter).

10.6 Value-Based Idempotency (CVAS)

Compare-Value-And-Swap (CVAS) writes a new value only if the current value matches an expected value:

```
KahunaKeyValue result = await client.TryCompareValueAndSetKeyValue(
    "orders/abc/status",
    value: "shipped",
    compareValue: "paid"
);

if (result.Success)
    Console.WriteLine("Order marked as shipped.");
else
    Console.WriteLine("Order is not in 'paid' status. No change made.");
```

This is idempotent for state-machine transitions. If the order is already “shipped” (from a previous attempt), the compare against “paid” fails and the retry is a no-op.

CVAS is useful when:

- The key follows a state machine (pending, paid, shipped, delivered).
- Each transition is valid from exactly one source state.
- A retry from the same source state is harmless because the transition already happened.

10.7 SetIfNotExists as an Idempotency Guard

The simplest idempotency guard stores a record of the completed operation. Before doing work, check whether the record exists. If it does, skip the work.

```
async Task<bool> ProcessOrderIdempotently(KahunaClient client, string orderId,
OrderDetails details)
{
    // Try to create a completion record. If it already exists, we already
    // processed this order.
    KahunaKeyValue guard = await client.SetKeyValue(
        $"processed/{orderId}",
        "done",
        flags: KeyValueFlags.SetIfNotExists,
```

```

        durability: KeyValueDurability.Persistent
    );

    if (!guard.Success)
    {
        Console.WriteLine($"Order {orderId} was already processed. Skipping.");
        return false;
    }

    // First time: do the actual work.
    await ChargePayment(details);
    await UpdateInventory(details);
    await SendConfirmationEmail(details);

    return true;
}

```

The `SetIfExists` flag ensures that only one call succeeds. All retries see the existing key and skip the work.

10.7.1 The Crash Window

There is a gap between the guard write and the actual work. If the process crashes after writing the guard but before finishing the work, the retried call sees the guard and skips the work. The result: the work is never completed.

To close this gap, use a two-phase approach:

async Task `ProcessOrderSafely`(KahunaClient client, string orderId, OrderDetails details)

```

{
    string guardKey = $"processed/{orderId}";

    KahunaKeyValue existing = await client.GetKeyValue(guardKey);

    if (existing.ValueAsString == "completed")
        return;

    // Mark as in-progress (not yet completed).
    await client.SetKeyValue(
        guardKey,
        "in-progress",
        flags: KeyValueFlags.SetIfExists
    );

    // Do the work.
    await ChargePayment(details);
    await UpdateInventory(details);

    // Mark as completed.
    await client.TryCompareValueAndSetKeyValue(

```

```

        guardKey,
        value: "completed",
        compareValue: "in-progress"
    );
}

```

On retry:

- If the guard is “completed,” skip. The work finished successfully.
- If the guard is “in-progress,” the previous attempt started but did not finish. You can resume or redo the work (if the work itself is idempotent).
- If the guard does not exist, this is a fresh attempt.

10.8 Fencing-Based Exactly-Once Writes

Chapter 4 and Chapter 8 showed how fencing tokens prevent stale lock holders from corrupting data. The same mechanism provides exactly-once writes.

The pattern:

1. Acquire a lock on the logical operation. Each lock acquisition gets a unique, monotonically increasing fencing token.
2. Perform the work.
3. Write the result along with the fencing token.
4. The downstream system rejects writes with a fencing token lower than or equal to the highest token it has already seen.

async Task **ProcessPaymentWithFencing**(KahunaClient client, string paymentId)

```

{
    await using KahunaLock lockHandle = await client.GetOrCreateLock(
        $"payment/{paymentId}",
        expiry: TimeSpan.FromSeconds(30),
        wait: TimeSpan.FromSeconds(10),
        retry: TimeSpan.FromMilliseconds(200)
    );

    if (!lockHandle.IsAcquired)
        throw new TimeoutException("Could not acquire payment lock.");

    long token = lockHandle.FencingToken;

    // Read the current state.
    KahunaKeyValue state = await client.GetKeyValue($"payment/{paymentId}/state");

    if (state.ValueAsString == "completed")
        return;

    // Charge the payment (external side effect).
    await ChargePaymentGateway(paymentId);
}

```

```

    // Write the result with the fencing token.
    // Only succeeds if no higher token has written to this key.
    await client.SetKeyValue(
        $"payment/{paymentId}/state",
        $"completed|token={token}"
    );
}

```

If the client retries after a timeout:

- If the lock is still held by the original attempt, the retry waits until it expires.
- When the retry acquires the lock, it gets a higher fencing token.
- The retry reads the state. If “completed,” it skips. If not, it processes and writes with the new token.

Fencing tokens are especially valuable when the operation involves an external system (a payment gateway, an email service) that cannot be rolled back. The token guarantees that only the most recent lock holder’s writes persist.

10.9 Sequence-Based Deduplication

Kahuna’s sequencer supports idempotency keys natively (Chapter 5). When you allocate a value with an idempotency key, retrying with the same key returns the same value:

```

long orderId = await client.NextSequenceValue(
    "order-ids",
    idempotencyKey: $"checkout-{requestId}"
);

```

If the response is lost and the client retries with the same requestId, Kahuna returns the same orderId. No new value is consumed. This is server-side deduplication: Kahuna stores a map of recent idempotency keys and their results.

The same applies to range reservations:

```

KahunaSequenceRange batch = await client.ReserveSequenceRange(
    "event-offsets",
    count: 50,
    idempotencyKey: $"import-batch-{batchId}"
);

```

A retry with the same key and count returns the identical range. A retry with the same key but a different count returns an error, because changing the parameters after the first call is ambiguous.

10.9.1 Retention Limits

Idempotency entries are not stored forever. Two server settings control retention:

Setting	Default	Description
--sequencer-idempotency-retention-max	256	Maximum entries per sequence
--sequencer-idempotency-retention-ttl	600 seconds	Time window for replay

After an entry is evicted (by count or age), a retry with that key allocates fresh values. Set these limits based on how long your retry windows last.

10.10 Transaction Operation IDs

Inside an interactive transaction session (Chapter 6), each operation carries a `TransactionOperationId`: a 128-bit random identifier. The client generates one automatically for every operation.

If a network timeout leaves the outcome of an operation unknown, the client can resubmit the same operation with the same `TransactionOperationId`. The server detects the duplicate and replays the original result instead of executing the operation again.

This deduplication is transparent. You do not need to manage operation IDs yourself. The `KahunaTransactionSession` generates a fresh random ID for every call you make. The server uses these IDs internally to handle retransmissions during the session's lifetime.

The IDs also support derived sub-operations. When an operation spans multiple pages (as in a paginated scan), the client derives deterministic sub-IDs from the original. If a page request is retried, the derived ID is the same, and the server replays the result.

10.11 Building an Idempotent Pipeline

Real applications often chain multiple steps. Each step must be idempotent independently, because a failure can happen between any two steps.

Consider an order processing pipeline:

```
async Task ProcessOrder(KahunaClient client, string orderId, OrderDetails order)
{
    // Step 1: Validate and reserve inventory.
    await ReserveInventory(client, orderId, order);

    // Step 2: Charge payment.
    await ChargePayment(client, orderId, order);

    // Step 3: Confirm order.
    await ConfirmOrder(client, orderId, order);
}
```

Each step uses a state key to track progress:

```
async Task ReserveInventory(KahunaClient client, string orderId, OrderDetails order)
{
```

```

string stepKey = $"orders/{orderId}/step/reserve";
KahunaKeyValue status = await client.GetKeyValue(stepKey);

if (status.ValueAsString == "done")
    return;

// Do the reservation.
foreach (var item in order.Items)
{
    await client.RetryableTransaction(
        new KahunaTransactionOptions { Timeout = 5000 },
        async (session, ct) =>
        {
            KahunaKeyValue stock = await session.GetKeyValue($"inventory/{item.Sku}");
            int available = int.Parse(stock.ValueAsString ?? "0");

            if (available < item.Quantity)
                throw new InvalidOperationException($"Insufficient stock for {item.Sku}.");

            await session.SetKeyValue(
                $"inventory/{item.Sku}",
                (available - item.Quantity).ToString()
            );
            await session.Commit(ct);
        }
    );

    await client.SetKeyValue(stepKey, "done");
}

async Task ChargePayment(KahunaClient client, string orderId, OrderDetails order)
{
    string stepKey = $"orders/{orderId}/step/charge";
    KahunaKeyValue status = await client.GetKeyValue(stepKey);

    if (status.ValueAsString == "done")
        return;

    // Acquire a lock with fencing to prevent duplicate charges.
    await using KahunaLock chargeLock = await client.GetOrCreateLock(
        $"charge/{orderId}",
        expiry: TimeSpan.FromSeconds(30)
    );
}

```



```

    if (!chargeLock.IsAcquired)
        throw new InvalidOperationException("Could not acquire charge lock.");

    // Check again after acquiring the lock (another instance may have completed
    it).
    status = await client.GetKeyValue(stepKey);
    if (status.ValueAsString == "done")
        return;

    await CallPaymentGateway(order.PaymentDetails, chargeLock.FencingToken);

    await client.SetKeyValue(stepKey, "done");
}

async Task ConfirmOrder(KahunaClient client, string orderId, OrderDetails order)
{
    // Use CVAS: transition from "processing" to "confirmed."
    await client.TryCompareValueAndSetKeyValue(
        $"orders/{orderId}/status",
        value: "confirmed",
        compareValue: "processing"
    );
}

```

Each step checks a status key before doing work. If the step already completed, it returns immediately. If the process crashes and restarts, it resumes from the last incomplete step. No step executes twice.

10.12 Choosing the Right Mechanism

	Mechanism	Best For	Scope
	Unconditional SET	Values that can be overwritten safely	Single key
	SetIfNotExists	One-time creation (guards, flags)	Single key
CRAS	(Compare-Revision-And-Swap)	Read-modify-write on a single key	Single key
CVAS	(Compare-Value-And-Swap)	State machine transitions	Single key
	Fencing tokens	Protecting external side effects	Lock scope
	Sequence idempotency keys	Preventing duplicate ID allocation	Single sequence
	Transaction operation IDs	Retransmission within a session	Transaction scope
	Step-based progress tracking	Multi-step pipelines	Workflow scope

For simple key-value updates, CRAS or CVAS is enough. For operations with external side effects (payment charges, email sends), use fencing tokens. For multi-step workflows, combine step-based progress tracking with per-step idempotency guards.

10.13 Common Mistakes

10.13.1 Checking Before Writing (Without Atomicity)

```
// INCORRECT: race condition between check and write
KahunaKeyValue existing = await client.GetKeyValue($"processed/{requestId}");
if (existing.Success)
    return;

await DoWork();
await client.SetKeyValue($"processed/{requestId}", "done");
```

Two concurrent requests can both read “not found” and both proceed to do the work. Use `SetIfNotExists` or CRAS to make the check-and-write atomic.

10.13.2 Relying on Client-Side State

```
// INCORRECT: client-side flag does not survive a restart
private bool _processed = false;

async Task Handle(string requestId)
{
    if (_processed) return;
    await DoWork();
    _processed = true;
}
```

Client-side state is lost on crash or restart. Store the idempotency record in Kahuna, where it survives process failures and is visible to all instances.

10.13.3 Ignoring the Gap Between Guard and Work

Writing a guard and then doing work is not atomic. If the process crashes between the two, the guard says “done” but the work did not complete. Use a two-phase status (“in-progress” then “completed”) so a retry can detect and resume incomplete work.

10.14 Summary

Retries are inevitable in distributed systems. Every mutating operation must be safe to retry, either naturally or through an explicit mechanism.

Kahuna provides several building blocks for idempotency. Conditional writes (CRAS, CVAS, `SetIfNotExists`) make single-key updates safe. Fencing tokens protect operations that touch external systems. Sequence idempotency keys prevent duplicate ID allocation at the server level. Transaction operation IDs handle retransmission within a session transparently.

For multi-step workflows, combine step-based progress tracking with per-step idempotency guards. Each step records its completion status. On retry, the pipeline resumes from the last incomplete step.

The next chapter covers globally unique identifier generation: patterns for producing unique, sortable IDs at scale using Kahuna's distributed sequencer.

11 Globally Unique Identifiers

Most distributed applications need unique identifiers. Order numbers, event offsets, invoice IDs, user handles, log sequence numbers: each must be unique across every node, process, and data center that participates in the system.

On a single database, an auto-incrementing column solves the problem. The database serializes all inserts through one counter. In a distributed system, there is no single database. Two services inserting records into two different databases will produce colliding IDs unless they coordinate.

This chapter explores patterns for generating unique IDs with Kahuna's distributed sequencer. It compares Kahuna sequences with UUIDs, Snowflake IDs, and database sequences, and shows how to tune the sequencer for different workloads.

11.1 The ID Generation Problem

A good identifier scheme must satisfy at least two requirements:

1. **Uniqueness.** No two records in the system share the same ID.
2. **Availability.** Generating an ID must not become a bottleneck under high throughput.

Beyond these, different applications value different properties:

- **Sortability.** Can IDs be sorted chronologically? This matters for time-series data, event logs, and database indexes.
- **Human readability.** Can a support engineer read an ID over the phone? Short numeric IDs are easier to communicate than 128-bit hex strings.
- **Contiguity.** Are IDs contiguous (no gaps)? Some financial regulations require contiguous invoice numbers.
- **Predictability.** Should IDs be unpredictable? Public-facing IDs that are sequential can leak information about volume and growth.

No single scheme satisfies all of these. Each scheme makes trade-offs.

11.2 Common ID Schemes

11.2.1 UUIDs (v4)

A UUID v4 is a 128-bit random number. It requires no coordination: any node can generate one independently.

```
string id = Guid.NewGuid().ToString();  
// "3f2504e0-4f89-11d3-9a0c-0305e82c3301"
```

Strengths:

- No coordination. Any process generates one locally.
- Extremely low collision probability.
- Works offline.

Weaknesses:

- Not sortable by time.
- Not human-readable (36 characters).

- Poor database index performance. Random values cause B-tree fragmentation because inserts scatter across the index instead of appending to the end.
- Not contiguous.

11.2.2 UUIDs (v7)

UUID v7 embeds a Unix timestamp in the most significant bits. This makes UUIDs roughly sortable by creation time.

```
string id = Guid.CreateVersion7().ToString();
```

This solves the sortability and B-tree fragmentation problems but does not improve human readability. The IDs are still 36 characters long.

11.2.3 Snowflake IDs

Twitter's Snowflake scheme packs a 64-bit integer from three components: a timestamp (41 bits), a machine ID (10 bits), and a per-machine sequence (12 bits). Each machine generates IDs independently.

Strengths:

- Sortable by time.
- 64-bit integer (fits in a database bigint).
- High throughput (4,096 IDs per millisecond per machine).

Weaknesses:

- Requires machine ID assignment. You must ensure no two machines share the same ID.
- Clock skew can cause non-monotonic IDs across machines.
- Not contiguous.
- Not human-readable (large numbers).

11.2.4 Database Auto-Increment

A single database generates sequential IDs through an auto-incrementing column.

Strengths:

- Contiguous.
- Sortable.
- Human-readable (short numbers).

Weaknesses:

- Single point of failure.
- Throughput bottleneck. Every insert must go through one database.
- Does not work across multiple databases without coordination.

11.2.5 Kahuna Sequences

Kahuna's distributed sequencer generates monotonically increasing 64-bit integers. Block-based allocation amortizes the cost of consensus across many values (Chapter 5).

Strengths:

- Unique across the entire cluster.
- Monotonically increasing (sortable).

- Human-readable (short numbers starting from a configurable initial value).
- High throughput via block allocation.
- Contiguous within a reserved range.
- Idempotent allocation with idempotency keys.

Weaknesses:

- Requires a running Kahuna cluster (not available offline).
- Gaps can occur on node failure or leadership change (unless block size is 1).
- One network round trip per block allocation.

11.3 Comparison Table

Property	UUID v4	UUID v7	Snowflake	DB Auto-Inc	Kahuna Sequence
Coordination required	No	No	Machine ID	Single DB	Kahuna cluster
Sortable by time	No	Yes	Yes	Yes	Yes
Human-readable	No	No	No	Yes	Yes
Contiguous	No	No	No	Yes	Configurable
Throughput	Unlimited	Unlimited	4,096/ms/machine	DB-limited	Tunable (block size)
Size	128 bits	128 bits	64 bits	32 or 64 bits	64 bits
Offline generation	Yes	Yes	Yes	No	No
B-tree friendly	No	Yes	Yes	Yes	Yes

Choose Kahuna sequences when you need human-readable, sortable, unique IDs and you already run a Kahuna cluster. Choose UUIDs when you need offline generation or cannot depend on a central service. Choose Snowflake when you need high throughput without any coordination service and can manage machine IDs.

11.4 Patterns for ID Generation

11.4.1 Single Sequence Per Entity Type

The simplest pattern: one sequence per type of entity.

```
await client.CreateSequence("order-ids", initialValue: 10000);
await client.CreateSequence("invoice-ids", initialValue: 100000);
await client.CreateSequence("user-ids");
```

// In the order service:

```

long orderId = await client.NextSequenceValue("order-ids");

// In the billing service:
long invoiceId = await client.NextSequenceValue("invoice-ids");

// In the user service:
long userId = await client.NextSequenceValue("user-ids");

```

Each entity type gets its own counter. The counters are independent. The order service and the billing service can allocate IDs concurrently without contention because they use different sequences.

Setting `initialValue` to a non-zero number makes early IDs more presentable. Order #10001 looks more professional than order #1.

11.4.2 Sequence Per Tenant

In a multi-tenant system, each tenant can have its own sequence:

```

async Task<long> GetNextOrderId(KahunaClient client, string tenantId)
{
    string sequenceName = $"orders/{tenantId}";

    try
    {
        return await client.NextSequenceValue(sequenceName);
    }
    catch (KahunaException ex) when (ex.Message.Contains("NotFound"))
    {
        await client.CreateSequence(sequenceName, initialValue: 1000);
        return await client.NextSequenceValue(sequenceName);
    }
}

```

Tenant A's order #1001 and tenant B's order #1001 are independent. This is useful when tenants expect their own numbering (invoices, tickets, work orders).

The trade-off is the number of sequences. If you have 10,000 tenants, you have 10,000 sequences. Each sequence occupies a small amount of memory on the sequencer actor that manages it. The server setting `--sequencer-max-sequences-per-actor` (default: 10,000) controls how many sequences one actor keeps in memory. Least recently used sequences are evicted and reloaded on demand.

11.4.3 Prefixed Composite IDs

Combine a sequence number with a prefix to create IDs that carry context:

```

long seq = await client.NextSequenceValue("invoice-ids");
string invoiceId = $"INV-{DateTime.UtcNow:yyyyMM}-{seq:D6}";
// "INV-202608-010042"

```

The prefix makes the ID self-describing. A support engineer can see at a glance that "INV-202608-010042" is an invoice from August 2026. The sequence number provides uniqueness. The zero-padded format (D6) makes IDs sort correctly as strings.

Be careful with this pattern. The date prefix is informational, not a uniqueness guarantee. The sequence number alone guarantees uniqueness. If you reset the sequence or create a new one each month, you must ensure old and new sequences do not overlap.

11.4.4 Batch ID Allocation

When you import data in bulk, allocating one ID at a time is inefficient. Reserve a range and assign IDs locally:

```
async Task ImportCustomers(KahunaClient client, List<CustomerRecord> records)
{
    KahunaSequenceRange range = await client.ReserveSequenceRange(
        "customer-ids",
        count: records.Count,
        idempotencyKey: $"import-{batchId}"
    );

    long id = range.Start;

    foreach (CustomerRecord record in records)
    {
        record.Id = id++;
        await SaveToDatabase(record);
    }
}
```

One network round trip reserves all the IDs. The application assigns them locally. The idempotency key ensures that retrying the import does not consume a second range.

Within a reserved range, IDs are contiguous. This is useful for data imports where contiguous numbering is a requirement.

11.4.5 Sharded ID Spaces

When multiple services need non-overlapping ID ranges, use staggered sequences:

```
int shardCount = 4;

for (int i = 0; i < shardCount; i++)
{
    await client.CreateSequence(
        $"event-ids-shard-{i}",
        initialValue: i,
        increment: shardCount
    );
}

// Shard 0 produces: 4, 8, 12, 16, ...
// Shard 1 produces: 5, 9, 13, 17, ...
// Shard 2 produces: 6, 10, 14, 18, ...
// Shard 3 produces: 7, 11, 15, 19, ...
```


Each shard generates IDs independently. The increment ensures the ranges never overlap. The shard number is encoded in the last bits of every ID (`id % shardCount == shardIndex`).

This pattern is useful when each shard writes to its own database partition and you need globally unique IDs without cross-shard coordination at write time.

11.4.6 Gap-Free Invoice Numbers

Some jurisdictions require contiguous invoice numbers with no gaps. Set the block size to 1:

```
kahuna-server --sequencer-block-size 1
```

With a block size of 1, every allocation requires a Raft round trip. This is slower (roughly 1,000 times slower than the default block size of 1,000) but guarantees no gaps from unused block remainders.

Even with block size 1, an idempotent allocation with a lost response and no retry can appear as a gap to the application. The sequence advanced, but the application never used the value. To close this gap, use an idempotency key:

```
long invoiceNumber = await client.NextSequenceValue(
    "invoice-numbers",
    idempotencyKey: $"create-invoice-{requestId}"
);
```

If the response is lost, the retry returns the same number. No value is wasted.

11.5 Tuning Block Size

The `--sequencer-block-size` setting controls the trade-off between throughput and gap size:

Block Size	Raft Commits Per 10,000 IDs	Maximum Gap on Failure	Use Case
1	10,000	0	Gap-free numbering (invoices, legal documents)
10	1,000	9	Low-gap tolerance with moderate throughput
100	100	99	Balanced for most workloads
1,000 (default)	10	999	High throughput (event offsets, log entries)
10,000	1	9,999	Maximum throughput (analytics, telemetry)

The “maximum gap on failure” column shows the worst case: a node reserves a full block and crashes before issuing any value from it. In practice, gaps are smaller because the node usually issues some values before failing.

11.5.1 Block Lease Revalidation

Each block has a lease (default: 5 seconds, configurable with `--sequencer-block-lease`). If a block sits in memory longer than its lease without being revalidated, the sequencer checks the durable record before serving more values. This prevents a stale leader from issuing values that conflict with a new leader’s allocations.

Shorter leases reduce stale-leader exposure but increase revalidation overhead. For most workloads, the 5-second default is appropriate.

11.6 Failure Scenarios

11.6.1 Node Failure Mid-Block

1. A sequencer actor reserves block [5001, 6000].
2. It issues values 5001 through 5042 to clients.
3. The node crashes.
4. A new leader takes over. It reserves block [6001, 7000].
5. Values 5043 through 6000 are never issued. The sequence jumps from 5042 to 6001.

This is the fundamental gap trade-off of block allocation. The gap is bounded by the block size. With the default block size of 1,000, the maximum gap from a single failure is 999 values.

11.6.2 Sequence Exhaustion

If a sequence has a maximum value and the high-water mark reaches it, further allocations fail:

```
try
{
    long id = await client.NextSequenceValue("limited-ids");
}
catch (KahunaException ex)
{
    Console.WriteLine($"Sequence exhausted: {ex.Message}");
}
```

Plan for exhaustion in advance. Monitor the current high-water mark with `GetSequence` and alert when it approaches the maximum:

```
KahunaSequence? seq = await client.GetSequence("limited-ids");

if (seq != null && seq.MaxValue > 0)
{
    double usedPercent = (double)seq.CurrentValue / seq.MaxValue * 100;

    if (usedPercent > 90)
        Console.WriteLine($"WARNING: Sequence is {usedPercent:F1}% exhausted.");
}
```

The `CurrentValue` property is the high-water mark (the highest reserved value), not the last value issued. It may be higher than the last value a client received.

11.6.3 Leadership Change

During a Raft leadership change, the old leader's in-memory block is discarded. The new leader starts fresh by reserving a new block. This causes a gap equal to the unused portion of the old block.

The gap preserves correctness: the new leader cannot reuse values from the old block because it does not know which values were already issued. Starting from the next block boundary guarantees uniqueness.

11.7 When Not to Use Kahuna Sequences

Kahuna sequences are a poor fit in some scenarios:

- **Offline ID generation.** If the client cannot reach a Kahuna cluster, it cannot allocate IDs. Use UUIDs or Snowflake IDs for offline-capable systems.
- **Unpredictable IDs.** Sequential IDs leak information. If a user sees order #10042, they know roughly how many orders exist. For public-facing IDs where this matters, generate a random ID or hash the sequential one.
- **Extremely high throughput with zero coordination.** If each node needs millions of IDs per second and cannot tolerate a network round trip per block, use local Snowflake-style generation.
- **Cross-cluster uniqueness.** Kahuna sequences are unique within one cluster. If you run multiple independent clusters, sequences can collide. Use a cluster prefix or a UUID for cross-cluster uniqueness.

11.8 A Complete Example

This example creates a service that generates formatted order IDs with a prefix, using batch allocation for throughput:

```
public class OrderIdGenerator
{
    private readonly KahunaClient _client;
    private readonly string _sequenceName = "order-ids";
    private readonly int _batchSize = 100;
    private long _nextId;
    private long _maxId;
    private readonly SemaphoreSlim _lock = new(1, 1);

    public OrderIdGenerator(KahunaClient client)
    {
        _client = client;
    }

    public async Task Initialize()
    {
        try
        {
            await _client.CreateSequence(_sequenceName, initialValue: 100000);
        }
        catch (KahunaException)
        {
            // Sequence already exists.
        }
    }

    public async Task<string> NextOrderId()
    {
        await _lock.WaitAsync();
        try
        {
            if (_nextId >= _maxId)
```

```

        {
            KahunaSequenceRange range = await _client.ReserveSequenceRange(
                _sequenceName,
                count: _batchSize
            );

            _nextId = range.Start;
            _maxId = range.End + 1;
        }

        long id = _nextId++;
        return $"ORD-{id}";
    }
    finally
    {
        _lock.Release();
    }
}
}

```

Usage:

```

var generator = new OrderIdGenerator(client);
await generator.Initialize();

string id1 = await generator.NextOrderId(); // "ORD-100001"
string id2 = await generator.NextOrderId(); // "ORD-100002"

```

The generator reserves 100 IDs at a time and serves them from memory. After 100 IDs, it reserves another batch. This reduces network round trips to one per 100 IDs.

The SemaphoreSlim protects the in-memory range from concurrent access. If multiple threads call NextOrderId simultaneously, only one reserves a new batch while the others wait.

If the process crashes with unused IDs in the batch, those IDs are lost (a gap of at most 99 values). For most order numbering, this is acceptable.

11.9 Summary

Kahuna's distributed sequencer provides human-readable, sortable, unique IDs without a single-database bottleneck. Block allocation delivers high throughput by amortizing Raft consensus across many values.

Choose the block size based on your tolerance for gaps: block size 1 for gap-free numbering, larger blocks for higher throughput. Use range reservation for batch imports. Use staggered sequences for sharded ID spaces. Use idempotency keys to prevent duplicate allocation on retries.

For workloads that need offline generation, unpredictable IDs, or cross-cluster uniqueness, consider UUIDs or Snowflake IDs instead.

The next chapter covers service coordination: distributed configuration, service presence detection, and session management.

12 Coordinating Services

Backend services in a distributed system need shared state. A feature flag must be visible to every instance of the API. A scheduler must know which workers are alive. A game server must track which players are in a lobby. Each of these problems requires state that is consistent, fault-tolerant, and observable across multiple processes.

This chapter shows four coordination patterns built on Kahuna's key-value store: distributed configuration, service presence detection, session management, and game lobby coordination.

12.1 Distributed Configuration

Many applications store configuration in a file or environment variable. This works until you need to change a value without restarting the application. A distributed configuration registry stores configuration in Kahuna, where any service instance can read the current value and where updates take effect immediately.

12.1.1 Storing Configuration

Use a key prefix to organize configuration by domain:

```
await client.SetKeyValue("config/api/rate-limit", "1000");
await client.SetKeyValue("config/api/timeout-ms", "5000");
await client.SetKeyValue("config/features/dark-mode", "true");
await client.SetKeyValue("config/features/beta-signup", "false");
```

Each configuration entry is a key-value pair. The prefix (config/api/, config/features/) groups related entries.

12.1.2 Reading Configuration

Read a single value:

```
KahunaKeyValue entry = await client.GetKeyValue("config/api/rate-limit");
int rateLimit = int.Parse(entry.ValueAsString ?? "500");
```

Read all entries in a group:

```
List<KahunaKeyValue> apiConfig = await client.GetByBucket(
    "config/api/",
    KeyValueDurability.Persistent
);

foreach (KahunaKeyValue kv in apiConfig)
{
    Console.WriteLine($"{kv.Key} = {kv.ValueAsString}");
}
```

GetByBucket returns up to 4,096 entries that match the prefix. For configuration registries, this limit is rarely a concern. If you have more than 4,096 configuration entries, use ScanAllByPrefix instead, which streams all matching entries:

```
await foreach (KahunaKeyValue kv in client.ScanAllByPrefix("config/"))
{
    Console.WriteLine($"{kv.Key} = {kv.ValueAsString}");
}
```

12.1.3 Updating Configuration Safely

When two operators update the same configuration key at the same time, one update can overwrite the other. Use Compare-Revision-And-Swap (CRAS) to prevent lost updates:

```
async Task<bool> UpdateConfig(KahunaClient client, string key, string newValue)
{
    KahunaKeyValue current = await client.GetKeyValue(key);

    KahunaKeyValue result = await client.TryCompareRevisionAndSetKeyValue(
        key,
        newValue,
        compareRevision: current.Revision
    );

    return result.Success;
}
```

If another update changed the key between the read and the write, the revision does not match and the write fails. The caller can retry with the fresh value.

12.1.4 Polling for Changes

Kahuna does not provide a push-based watch mechanism (unlike etcd's watch API). Services must poll for changes. A simple polling loop reads configuration periodically and applies changes:

```
public class ConfigPoller
{
    private readonly KahunaClient _client;
    private readonly string _prefix;
    private readonly Dictionary<string, (string Value, long Revision)> _cache
= new();

    public ConfigPoller(KahunaClient client, string prefix)
    {
        _client = client;
        _prefix = prefix;
    }

    public async Task PollOnce()
    {
        List<KahunaKeyValue> entries = await _client.GetByBucket(
            _prefix,
            KeyValueDurability.Persistent
        );

        foreach (KahunaKeyValue entry in entries)
        {
            if (_cache.TryGetValue(entry.Key, out var cached) && cached.Revision
== entry.Revision)
```

```

        continue;

        _cache[entry.Key] = (entry.ValueAsString ?? "", entry.Revision);
        OnConfigChanged(entry.Key, entry.ValueAsString);
    }
}

private void OnConfigChanged(string key, string? value)
{
    Console.WriteLine($"Config changed: {key} = {value}");
}
}

```

The poller compares revisions to detect changes. If the revision for a key has not changed since the last poll, the value is the same and no action is needed. This avoids unnecessary processing.

A poll interval of 5 to 30 seconds is appropriate for most configuration use cases. Critical settings (rate limits, circuit breaker thresholds) benefit from shorter intervals. Static settings (feature flags, display text) can tolerate longer intervals.

12.1.5 Configuration with TTL

Some configuration entries should expire automatically. A maintenance window flag, for example, should turn itself off after a set duration:

```

await client.SetKeyValue(
    "config/maintenance-mode",
    "true",
    expiryTime: 3600000 // 1 hour in milliseconds
);

```

After one hour, the key expires. Services that poll for this key will see it disappear and exit maintenance mode. No manual cleanup is required.

12.2 Service Presence Detection

In a microservice architecture, services need to know which instances of other services are alive. A load balancer needs a list of healthy backends. A job scheduler needs to know which workers are available.

Kahuna's TTL-based keys provide a simple presence mechanism. Each service instance writes an ephemeral key with a short TTL. The key acts as a heartbeat. If the instance crashes, the key expires and the instance disappears from the registry.

12.2.1 Registering Presence

Each service instance registers itself on startup and renews its registration periodically:

```

public class PresenceService : BackgroundService
{
    private readonly KahunaClient _client;
    private readonly string _serviceName;
    private readonly string _instanceId;
}

```



```

private readonly int _ttlMs = 15000;
private readonly int _renewalMs = 5000;

public PresenceService(KahunaClient client, string serviceName)
{
    _client = client;
    _serviceName = serviceName;
    _instanceId = Guid.NewGuid().ToString("N")[..8];
}

protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    string key = $"presence/{_serviceName}/{_instanceId}";

    string info = JsonSerializer.Serialize(new
    {
        instanceId = _instanceId,
        host = Environment.MachineName,
        port = 8080,
        startedAt = DateTimeOffset.UtcNow
    });

    while (!stoppingToken.IsCancellationRequested)
    {
        await _client.SetKeyValue(key, info, expiryTime: _ttlMs);
        await Task.Delay(_renewalMs, stoppingToken);
    }
}

```

The key includes the service name and a unique instance ID. The value contains connection details that other services need (host, port). The TTL is 15 seconds. The renewal runs every 5 seconds (one-third of the TTL), which gives two buffer attempts before the key expires.

12.2.2 Discovering Instances

Other services query the presence registry to find healthy instances:

```

async Task<List<ServiceInstance>> DiscoverInstances(KahunaClient client, string
serviceName)
{
    List<KahunaKeyValue> entries = await client.GetByBucket(
        $"presence/{serviceName}/",
        KeyValueDurability.Persistent
    );

    List<ServiceInstance> instances = new();

    foreach (KahunaKeyValue entry in entries)
    {

```

```

var instance =
JsonSerializer.Deserialize<ServiceInstance>(entry.ValueAsString ?? "{}");
    if (instance != null)
        instances.Add(instance);
}

return instances;
}

```

Only instances with unexpired keys appear in the result. A crashed instance's key expires after 15 seconds. The discovery query returns only live instances.

12.2.3 Ephemeral Keys for Presence

For presence data that does not need Raft replication, use ephemeral durability:

```

await client.SetKeyValue(
    $"presence/{serviceName}/{instanceId}",
    info,
    expiryTime: 15000,
    durability: KeyValueDurability.Ephemeral
);

```

Ephemeral keys are stored in memory on the receiving node. They are faster to write (no Raft round trip) but do not survive a node restart. If the Kahuna node holding the ephemeral key crashes, all presence records on that node vanish instantly.

Use ephemeral keys when:

- The presence data is recreated quickly (instances re-register on their next heartbeat cycle).
- Speed matters more than durability.
- Losing presence records temporarily is acceptable (the system recovers within one heartbeat cycle).

Use persistent keys when:

- The presence data is expensive to recreate.
- You need the registry to survive a Kahuna node restart.
- Accuracy during Kahuna cluster maintenance is important.

12.2.4 Graceful Deregistration

When a service shuts down gracefully, it should delete its presence key immediately instead of waiting for the TTL:

```

public override async Task StopAsync(CancellationToken cancellationToken)
{
    string key = $"presence/{_serviceName}/{_instanceId}";
    await _client.DeleteKeyValue(key);
    await base.StopAsync(cancellationToken);
}

```

This makes the instance disappear from the registry immediately. Other services see the updated list on their next discovery query. Without graceful deregistration, the stale key persists for up to 15 seconds after the process exits.

12.3 Session Management

Web applications and APIs often need server-side session state: a shopping cart, user preferences, or authentication tokens. Storing session state in Kahuna provides fault tolerance (the session survives a process restart) and consistency (multiple API instances can read the same session).

12.3.1 Creating a Session

```
async Task<string> CreateSession(KahunaClient client, string userId)
{
    string sessionId = Guid.NewGuid().ToString("N");
    string key = $"sessions/{sessionId}";

    string sessionData = JsonSerializer.Serialize(new
    {
        userId,
        createdAt = DateTimeOffset.UtcNow,
        cart = new List<string>()
    });

    await client.SetKeyValue(key, sessionData, expiryTime: 1800000); // 30
minutes

    return sessionId;
}
```

The session key includes a random ID. The TTL (30 minutes) defines the session timeout. If the user does not interact within 30 minutes, the session expires automatically.

12.3.2 Reading and Updating a Session

```
async Task<SessionData?> GetSession(KahunaClient client, string sessionId)
{
    KahunaKeyValue entry = await client.GetKeyValue($"sessions/{sessionId}");

    if (!entry.Success)
        return null;

    // Extend the session TTL on each access (sliding expiration).
    await client.ExtendKeyValue($"sessions/{sessionId}", 1800000);

    return JsonSerializer.Deserialize<SessionData>(entry.ValueAsString ?? "{}");
}

async Task UpdateSession(KahunaClient client, string sessionId, SessionData
data)
{
    string key = $"sessions/{sessionId}";

    KahunaKeyValue current = await client.GetKeyValue(key);
```

```

if (!current.Success)
    throw new InvalidOperationException("Session not found.");

KahunaKeyValue result = await client.TryCompareRevisionAndSetKeyValue(
    key,
    JsonSerializer.Serialize(data),
    compareRevision: current.Revision
);

if (!result.Success)
    throw new InvalidOperationException("Session was modified concurrently.
Retry.");

await client.ExtendKeyValue(key, 1800000);
}

```

Key details:

- ExtendKeyValue resets the TTL from the current time. This provides sliding expiration: the session stays alive as long as the user keeps interacting.
- TryCompareRevisionAndSetKeyValue prevents lost updates. If two API instances update the same session simultaneously, one succeeds and the other gets Success = false.

12.3.3 Session Cleanup

Sessions expire automatically through TTL. No background cleanup is needed. When the TTL elapses, Kahuna removes the key.

For an explicit logout:

```

async Task DestroySession(KahunaClient client, string sessionId)
{
    await client.DeleteKeyValue($"sessions/{sessionId}");
}

```

12.3.4 Listing Active Sessions

To list all active sessions (for an admin dashboard, for example):

```

await foreach (KahunaKeyValue session in client.ScanAllByPrefix("sessions/"))
{
    var data = JsonSerializer.Deserialize<SessionData>(session.ValueAsString ??
    "{}");
    Console.WriteLine($"Session: {session.Key}, User: {data?.UserId}");
}

```

ScanAllByPrefix streams results with no upper limit, so it handles large numbers of active sessions.

12.4 Game Lobby Coordination

Online multiplayer games need coordination infrastructure: creating game rooms, matching players, tracking who is in a lobby, and managing game state transitions. Kahuna's primitives (locks, TTL keys, prefix queries, transactions) combine to support these patterns.

12.4.1 Room Creation

Use a lock to ensure that room creation is exclusive:

```
async Task<string> CreateRoom(KahunaClient client, string gameMode, string
hostPlayer)
{
    long roomId = await client.NextSequenceValue("room-ids");
    string roomKey = $"rooms/{roomId}";

    string roomData = JsonSerializer.Serialize(new
    {
        id = roomId,
        gameMode,
        host = hostPlayer,
        status = "waiting",
        maxPlayers = 4,
        createdAt = DateTimeOffset.UtcNow
    });

    await client.SetKeyValue(roomKey, roomData);

    return roomKey;
}
```

The sequencer generates a unique room ID. The room starts in the “waiting” status.

12.4.2 Player Presence in a Room

Each player registers their presence in the room with a TTL key:

```
async Task JoinRoom(KahunaClient client, string roomKey, string playerId)
{
    string presenceKey = $"{roomKey}/players/{playerId}";

    string playerInfo = JsonSerializer.Serialize(new
    {
        playerId,
        joinedAt = DateTimeOffset.UtcNow,
        ready = false
    });

    await client.SetKeyValue(presenceKey, playerInfo, expiryTime: 10000);
}

async Task SendHeartbeat(KahunaClient client, string roomKey, string playerId)
{
}
```

```

        await client.ExtendKeyValue("{roomKey}/players/{playerId}", 10000);
    }

```

The player's presence key has a 10-second TTL. A heartbeat task renews it every 3 seconds. If the player disconnects (closes the browser, loses network), the key expires and the player disappears from the room.

12.4.3 Listing Players in a Room

```

async Task<List<PlayerInfo>> GetPlayersInRoom(KahunaClient client, string
roomKey)
{
    List<KahunaKeyValue> entries = await client.GetByBucket(
        $"{roomKey}/players/",
        KeyValueDurability.Persistent
    );

    return entries
        .Select(e => JsonSerializer.Deserialize<PlayerInfo>(e.ValueAsString ??
"{}"))
        .Where(p => p != null)
        .ToList()!;
}

```

Only players with unexpired keys appear. A player who disconnected 10 seconds ago is already gone from the list.

12.4.4 Room State Transitions

A game room moves through states: waiting, matching, playing, finished. Use CVAS (Compare-Value-And-Swap) to ensure state transitions are valid:

```

async Task<bool> StartGame(KahunaClient client, string roomKey)
{
    // Only transition from "waiting" to "playing."
    KahunaKeyValue room = await client.GetKeyValue(roomKey);
    var roomData = JsonSerializer.Deserialize<RoomData>(room.ValueAsString ??
"{}");

    if (roomData?.Status != "waiting")
        return false;

    var updated = roomData with { Status = "playing", StartedAt =
DateTimeOffset.UtcNow };

    KahunaKeyValue result = await client.TryCompareValueAndSetKeyValue(
        roomKey,
        JsonSerializer.Serialize(updated),
        compareValue: room.ValueAsString!
    );
}

```

```

    return result.Success;
}

```

If two players press “Start” at the same moment, only one CVAS succeeds. The other gets Success = false. The room transitions exactly once.

12.4.5 Matchmaking

A simple matchmaking system scans for rooms in the “waiting” state and adds the player to the first room with available slots:

```

async Task<string?> FindMatch(KahunaClient client, string playerId, string
gameMode)
{
    List<KahunaKeyValue> rooms = await client.GetByBucket(
        "rooms/",
        KeyValueDurability.Persistent
    );

    foreach (KahunaKeyValue entry in rooms)
    {
        var room = JsonSerializer.Deserialize<RoomData>(entry.ValueAsString ??
"{}");

        if (room?.Status != "waiting" || room.GameMode != gameMode)
            continue;

        List<PlayerInfo> players = await GetPlayersInRoom(client, entry.Key);

        if (players.Count >= room.MaxPlayers)
            continue;

        // Try to join. The TTL-based presence handles the race:
        // if two players join and exceed the limit, the host can
        // enforce the cap before starting the game.
        await JoinRoom(client, entry.Key, playerId);
        return entry.Key;
    }

    return null;
}

```

This is a simple linear scan. For production systems with thousands of rooms, partition rooms by game mode (use a prefix like rooms/{gameMode}/) and scan only the relevant partition.

12.4.6 Room Cleanup

Finished game rooms can be deleted explicitly:

```

async Task CleanupRoom(KahunaClient client, string roomKey)
{
    // Delete all player presence keys.
}

```

```

List<KahunaKeyValue> players = await client.GetByBucket(
    "${roomKey}/players/",
    KeyValueDurability.Persistent
);

foreach (KahunaKeyValue player in players)
{
    await client.DeleteKeyValue(player.Key);
}

// Delete the room itself.
await client.DeleteKeyValue(roomKey);
}

```

Alternatively, create rooms with a TTL so they expire automatically after the maximum game duration:

```
await client.SetKeyValue(roomKey, roomData, expiryTime: 3600000); // 1 hour
```

12.5 Failure Scenarios

12.5.1 Service Crash

A service instance crashes. Its presence key has a 15-second TTL. After 15 seconds, the key expires. The next discovery query omits the crashed instance. Other instances continue to serve traffic. When the crashed instance restarts, it re-registers with a new presence key.

12.5.2 Configuration Update Race

Two operators update the same configuration key at the same moment.

1. Operator A reads the key at revision 5.
2. Operator B reads the key at revision 5.
3. Operator A writes a new value with `compareRevision: 5`. The write succeeds. The revision becomes 6.
4. Operator B writes a different value with `compareRevision: 5`. The compare fails because the revision is now 6.

Operator B must re-read the key and decide whether to retry. No data is lost and no update is silently overwritten.

12.5.3 Player Disconnect

A player's network connection drops. The player's heartbeat stops. After 10 seconds, the presence key expires. The room's player list no longer includes the disconnected player. The host (or the game logic) can detect the change on the next player list query and handle it: pause the game, assign the player's slot to an AI, or end the match.

If the player reconnects within 10 seconds and resumes heartbeats, the key is refreshed and the player remains in the room.

12.6 Choosing Between Persistent and Ephemeral

Concern	Persistent	Ephemeral
Raft consensus	Yes (durable, replicated)	No (in-memory, single node)
Write latency	Higher	Lower
Survives node restart	Yes	No
Use case	Configuration, sessions, game state	Presence heartbeats, rate counters
Consistency	Strong (linearizable)	Best-effort (single node)

A good default: use persistent durability for data that matters (configuration, session state, room records) and ephemeral durability for data that is rebuilt quickly (presence heartbeats, temporary counters).

12.7 Summary

Kahuna's key-value primitives support four common coordination patterns. A configuration registry stores shared settings that all service instances can read. Prefix queries (`GetByBucket`, `ScanAllByPrefix`) retrieve groups of related entries. CRAS prevents lost updates.

Presence detection uses TTL-based keys as heartbeats. A service instance writes a key with a short TTL and renews it periodically. When the instance crashes, the key expires and the instance disappears from the registry.

Session management stores user state in keyed entries with sliding expiration. `ExtendKeyValue` resets the TTL on each access. CRAS prevents concurrent session updates from overwriting each other.

Game lobby coordination combines sequences (room IDs), TTL keys (player presence), prefix queries (player lists), and CVAS (state transitions) into a cohesive system.

The next chapter covers transactional workflows: how to decompose multi-step business operations into atomic transactions.

13 Transactional Workflows

Chapter 6 introduced transactions: script transactions that execute atomically on the server, and interactive sessions that let the client make decisions between operations. Chapter 9 showed how to make individual operations idempotent.

This chapter brings both ideas together. Real business operations span multiple keys, multiple decisions, and sometimes multiple transactions. An order that reserves inventory, charges a payment, and sends a confirmation must either complete fully or leave no partial trace. This chapter shows how to design these workflows, where to draw transaction boundaries, and how to handle failures at each stage.

13.1 The Problem with Separate Writes

Consider this order placement code:

```
// INCORRECT: two separate writes, no transaction
await client.SetKeyValue("inventory/widget-a", (stock - 1).ToString());
await client.SetKeyValue("orders/123/status", "confirmed");
```

If the process crashes between the two writes, the inventory is decremented but the order is never confirmed. The customer does not get their order. The inventory count is wrong.

Making both writes unconditional does not help either. If the second write fails (network timeout, server error), retrying the entire operation decrements inventory a second time.

The solution is a transaction. Both writes succeed together or fail together. No intermediate state is visible to other clients.

13.2 Single-Transaction Workflows

13.2.1 Script Transaction

When the workflow logic is a simple sequence of reads and writes with straightforward conditions, a script transaction is the best fit:

```
string script = @"
    LET stock = GET inventory/widget-a
    IF stock = NOT FOUND THEN
        THROW 'Item not found'
    END

    LET count = TO_INT(stock)
    IF count < 1 THEN
        THROW 'Out of stock'
    END

    SET inventory/widget-a TO_STRING(count - 1)
    SET orders/@orderId/status 'confirmed'
    SET orders/@orderId/item 'widget-a'
";

KahunaKeyValueTransactionResult result =
await client.ExecuteKeyValueTransactionScript(
```

```

script,
parameters: new()
{
    new() { Key = "@orderId", Value = orderId }
}
);

```

The server executes the entire script as one atomic unit. If the stock check fails, no writes happen. If both writes succeed, they are visible together. No other client can see the inventory decremented without the order confirmed.

Script transactions are best when:

- All the data needed for decisions is inside Kahuna.
- The logic is expressible with the script language (conditions, loops, string operations).
- No external service calls are needed during the transaction.

13.2.2 Interactive Session

When the workflow requires application logic that the script language cannot express (calling an external API, running a complex computation, making a decision based on business rules), use an interactive session:

```

await client.RetryableTransaction(
    new KahunaTransactionOptions
    {
        Timeout = 5000,
        Locking = KeyValueTransactionLocking.Pessimistic
    },
    async (session, ct) =>
    {
        KahunaKeyValue stockEntry = await session.GetKeyValue("inventory/
widget-a");
        int stock = int.Parse(stockEntry.ValueAsString ?? "0");

        if (stock < 1)
            throw new InvalidOperationException("Out of stock.");

        decimal price = await GetCurrentPrice("widget-a");

        if (price > maxBudget)
            throw new InvalidOperationException("Price exceeds budget.");

        await session.SetKeyValue("inventory/widget-a", (stock - 1).ToString());
        await session.SetKeyValue($"orders/{orderId}/status", "confirmed");
        await session.SetKeyValue($"orders/{orderId}/price", price.ToString());
        await session.Commit(ct);
    }
);

```

The `GetCurrentPrice` call is application logic that cannot run inside a script. The interactive session lets you interleave reads from Kahuna, external calls, and writes within one atomic boundary.

`RetryableTransaction` wraps the session in a retry loop. If a concurrent transaction modifies the same keys, the commit fails and the entire callback runs again with a fresh session and fresh reads.

13.3 Choosing a Locking Mode

The locking mode determines how the transaction handles concurrent access.

13.3.1 Pessimistic Locking

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Pessimistic
}
```

Pessimistic locking acquires an exclusive lock on every key the session reads or writes. Other transactions that touch the same keys wait until this transaction commits or rolls back.

Use pessimistic locking when:

- Conflicts are frequent. Multiple workers process the same queue. Multiple users update the same account.
- The transaction does expensive work (external API calls, long computations). Aborting and retrying wastes that work.
- You prefer to wait for access rather than retry on conflict.

13.3.2 Optimistic Locking

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Optimistic
}
```

Optimistic locking acquires locks only on writes. Reads are lock-free. Conflicts are detected at commit time: if a key that this transaction read was modified by another transaction, the commit is rejected.

Use optimistic locking when:

- Conflicts are rare. Most transactions touch different keys.
- The transaction is short (a few reads and writes, no external calls).
- You prefer higher concurrency over guaranteed success.

13.3.3 Adding Read Validation

With optimistic locking, enable `TrackAndValidate` to detect write-skew anomalies:

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Optimistic,
    ReadValidation = ReadValidation.TrackAndValidate
}
```

The server records every key the session reads. At commit time, it checks whether any read key was modified after the read. If so, the commit is rejected. This prevents a class of bug where two transactions read overlapping data, make independent decisions, and both commit without conflict.

13.4 Multi-Transaction Workflows

Not every workflow fits in a single transaction. A workflow that calls an external payment gateway cannot roll back the payment if a later step fails. The payment gateway does not participate in Kahuna's transaction protocol.

For these workflows, decompose the work into multiple transactions with explicit progress tracking between them.

13.4.1 The Saga Pattern

A saga is a sequence of transactions, each with a compensating action. If a step fails, the saga runs the compensating actions for all previously completed steps.

Here is an order placement saga with three steps:

```
async Task PlaceOrder(KahunaClient client, string orderId, OrderDetails order)
{
    // Step 1: Reserve inventory (within a Kahuna transaction).
    await ReserveInventory(client, orderId, order);

    try
    {
        // Step 2: Charge payment (external API call, guarded by a lock).
        await ChargePayment(client, orderId, order);
    }
    catch
    {
        // Compensate step 1: release the reserved inventory.
        await ReleaseInventory(client, orderId, order);
        throw;
    }

    // Step 3: Confirm the order (within a Kahuna transaction).
    await ConfirmOrder(client, orderId);
}
```

Each step is a separate transaction (or a lock-protected external call). If the payment fails, the compensation releases the inventory. The order is never left in an inconsistent state.

13.4.2 Step 1: Reserve Inventory

```
async Task ReserveInventory(KahunaClient client, string orderId, OrderDetails order)
{
    await client.RetryableTransaction(
        new KahunaTransactionOptions { Timeout = 5000 },
        async (session, ct) =>
```

```

    {
        foreach (var item in order.Items)
        {
            KahunaKeyValue stock = await session.GetKeyValue($"inventory/{item.Sku}");
            int available = int.Parse(stock.ValueAsString ?? "0");

            if (available < item.Quantity)
                throw new InvalidOperationException($"Insufficient stock for {item.Sku}.");

            await session.SetKeyValue(
                $"inventory/{item.Sku}",
                (available - item.Quantity).ToString()
            );
        }

        await session.SetKeyValue($"orders/{orderId}/step", "inventory-reserved");
        await session.Commit(ct);
    }
};
}

```

13.4.3 Step 2: Charge Payment

```

async Task ChargePayment(KahunaClient client, string orderId, OrderDetails order)
{
    await using KahunaLock chargeLock = await client.GetOrCreateLock(
        $"charge/{orderId}",
        expiry: TimeSpan.FromSeconds(30)
    );

    if (!chargeLock.IsAcquired)
        throw new InvalidOperationException("Could not acquire charge lock.");

    KahunaKeyValue stepCheck = await client.GetKeyValue($"orders/{orderId}/step");

    if (stepCheck.ValueAsString == "payment-charged")
        return;

    await CallPaymentGateway(order.PaymentDetails, chargeLock.FencingToken);

    await client.SetKeyValue($"orders/{orderId}/step", "payment-charged");
}

```

The lock prevents duplicate charges on retry. The step check provides idempotency: if the step already completed, the method returns without charging again. The fencing token protects against stale lock holders (Chapter 9).

13.4.4 Step 3: Confirm Order

```
async Task ConfirmOrder(KahunaClient client, string orderId)
{
    await client.RetryableTransaction(
        new KahunaTransactionOptions { Timeout = 5000 },
        async (session, ct) =>
        {
            await session.SetKeyValue($"orders/{orderId}/status", "confirmed");
            await session.SetKeyValue($"orders/{orderId}/step", "completed");
            await session.Commit(ct);
        }
    );
}
```

13.4.5 Compensation: Release Inventory

```
async Task ReleaseInventory(KahunaClient client, string orderId, OrderDetails
order)
{
    await client.RetryableTransaction(
        new KahunaTransactionOptions { Timeout = 5000 },
        async (session, ct) =>
        {
            foreach (var item in order.Items)
            {
                KahunaKeyValue stock = await session.GetKeyValue($"inventory/
{item.Sku}");
                int current = int.Parse(stock.ValueAsString ?? "0");

                await session.SetKeyValue(
                    $"inventory/{item.Sku}",
                    (current + item.Quantity).ToString()
                );
            }

            await session.SetKeyValue($"orders/{orderId}/step", "cancelled");
            await session.Commit(ct);
        }
    );
}
```

13.5 Transaction Boundaries

Choosing where to draw transaction boundaries is a design decision. Two principles guide it.

13.5.1 Principle 1: Keep Transactions Short

A transaction holds locks (in pessimistic mode) or risks conflicts (in optimistic mode) for its entire duration. The longer the transaction, the more contention it creates.

Do not put external calls (HTTP requests, database queries, message publishes) inside a Kahuna transaction. These calls add latency and can time out, which extends the lock duration and increases the chance of a transaction timeout.

Instead, separate the external call from the transaction:

```
// Read data in a transaction.
string price;
await client.RetryableTransaction(
    new KahunaTransactionOptions { Timeout = 5000 },
    async (session, ct) =>
    {
        KahunaKeyValue entry = await session.GetKeyValue("products/widget-a");
        price = entry.ValueAsString ?? "0";
        await session.Commit(ct);
    }
);

// External call outside the transaction.
bool approved = await PaymentGateway.Charge(price);

// Write the result in a second transaction.
if (approved)
{
    await client.RetryableTransaction(
        new KahunaTransactionOptions { Timeout = 5000 },
        async (session, ct) =>
        {
            await session.SetKeyValue("orders/123/status", "paid");
            await session.Commit(ct);
        }
    );
}
```

13.5.2 Principle 2: Group Related Writes

Writes that must be consistent with each other belong in the same transaction. If decrementing inventory and recording the order must happen together, they go in one transaction. If they can tolerate temporary inconsistency (inventory decremented but order not yet recorded), they can go in separate transactions.

Ask this question: “If the process crashes between these two operations, is the data in a valid state?” If the answer is no, the operations belong in the same transaction.

13.6 Handling Conflicts

When two transactions modify the same keys concurrently, one succeeds and the other is rejected.

13.6.1 With Pessimistic Locking

The second transaction waits for the first to release its locks. If the wait exceeds the transaction timeout, the second transaction is aborted with a timeout error. The client can start a new session and retry.

13.6.2 With Optimistic Locking

Both transactions proceed without waiting. At commit time, the server detects that one transaction read a key that the other modified. The second transaction to commit is rejected with an Aborted error. The client starts a new session and retries with fresh reads.

13.6.3 The Retry Loop

RetryableTransaction handles both cases automatically. It catches Aborted, MustRetry, and AlreadyLocked errors, waits with decorrelated jitter, and retries with a new session:

```
await client.RetryableTransaction(
    new KahunaTransactionOptions
    {
        Timeout = 5000,
        Locking = KeyValueTransactionLocking.Optimistic,
        ReadValidation = ReadValidation.TrackAndValidate
    },
    async (session, ct) =>
    {
        KahunaKeyValue a = await session.GetKeyValue("accounts/alice");
        KahunaKeyValue b = await session.GetKeyValue("accounts/bob");

        long balanceA = long.Parse(a.ValueAsString ?? "0");
        long balanceB = long.Parse(b.ValueAsString ?? "0");

        await session.SetKeyValue("accounts/alice", (balanceA - 50).ToString());
        await session.SetKeyValue("accounts/bob", (balanceB + 50).ToString());
        await session.Commit(ct);
    }
);
```

If another transaction modifies Alice's or Bob's balance between the reads and the commit, this transaction is rejected and retried. The retry reads fresh balances and recomputes the transfer.

The retry strategy uses up to 10 attempts with a jitter cap of 500 milliseconds. If all 10 attempts fail, the method throws a KahunaException.

13.7 Script Transactions for Complex Logic

Some workflows fit entirely in a script with no external calls. Script transactions are faster (one round trip instead of many) and simpler (no session management, no retry loop).

13.7.1 Conditional Batch Update

```
string script = @"
    LET total = 0
```

```

FOR key IN 'cart/user-42/' DO
    LET item = GET key
    IF item != NOT FOUND THEN
        LET price = TO_INT(item)
        LET total = total + price
    END
END

IF total > 10000 THEN
    SET orders/user-42/discount '10'
ELSE
    SET orders/user-42/discount '0'
END

SET orders/user-42/total TO_STRING(total)
";

await client.ExecuteKeyValueTransactionScript(script);

```

This script scans all items in a user's cart, computes the total, applies a discount rule, and writes the result. The entire operation is atomic.

13.7.2 Multi-Key Transfer with Validation

```

string script = @"
    LET from_balance = TO_INT(GET accounts/@from)
    LET to_balance = TO_INT(GET accounts/@to)
    LET amount = TO_INT(@amount)

    IF from_balance < amount THEN
        THROW 'Insufficient funds'
    END

    SET accounts/@from TO_STRING(from_balance - amount)
    SET accounts/@to TO_STRING(to_balance + amount)
";

KahunaTransactionScript transferScript = client.LoadTransactionScript(script);

await transferScript.Run(parameters: new()
{
    new() { Key = "@from", Value = "alice" },
    new() { Key = "@to", Value = "bob" },
    new() { Key = "@amount", Value = "100" }
});

```

The pre-hashed script caches the parsed plan on the server. Subsequent calls with different parameters skip parsing.

13.8 Failure Scenarios

13.8.1 Partial Failure Mid-Workflow

1. The saga reserves inventory (step 1 commits).
2. The payment gateway call times out (step 2 fails).
3. The saga runs compensation: releases the reserved inventory.
4. The order is not created. Inventory is back to its original state.

Without compensation, the inventory stays decremented and the order does not exist. The system is inconsistent.

13.8.2 Concurrent Workflow Conflict

1. Two users try to buy the last unit of the same item.
2. User A's transaction reads stock = 1 and decrements to 0.
3. User B's transaction reads stock = 1 (before A commits, in optimistic mode) and tries to decrement to 0.
4. User A commits first.
5. User B's commit is rejected (the stock key was modified after the read).
6. User B retries. The fresh read sees stock = 0. The transaction throws "Out of stock."

The conflict is detected and resolved correctly. No stock is oversold.

13.8.3 Timeout During Commit

1. The client sends a commit request.
2. The network drops the response.
3. The client does not know whether the commit succeeded.

What happens next depends on the transaction model:

- **Script transactions.** The server executes the script atomically. The client can retry with the same idempotency key (if the script is designed for it) or check the state by reading the keys.
- **Interactive sessions.** The server is the transaction coordinator. If the commit request reached the server and the server decided to commit, the writes are applied regardless of the client's network failure. The client should read the affected keys to determine the outcome.

In both models, the server drives the commit decision. A client crash after sending the commit request does not prevent the transaction from completing.

13.9 Workflow Design Checklist

Before building a multi-step workflow, answer these questions:

1. **Can the entire workflow fit in one transaction?** If yes, use a single script or session transaction. This is simpler and avoids partial failure.
2. **Does the workflow call external services?** If yes, those calls cannot be inside a Kahuna transaction. Split the workflow into multiple transactions with progress tracking between them.
3. **Does each step need a compensating action?** If a later step fails, can you undo the earlier steps? Define compensation for each step.

4. **Is each step idempotent?** On retry, does the step produce the same result? Use step-tracking keys (Chapter 9) to guard against duplicate execution.
5. **What locking mode fits?** High contention favors pessimistic. Rare conflicts favor optimistic. When unsure, start with pessimistic and measure.
6. **How long is the transaction?** Keep each transaction under 5 seconds. If work takes longer, split it into shorter transactions.

13.10 Summary

Transactional workflows ensure that multi-key operations are atomic: all writes commit or none do. Script transactions handle workflows where all logic runs on the server. Interactive sessions handle workflows that need application logic between reads and writes.

For workflows that involve external services, decompose the work into multiple transactions with the saga pattern. Each step records its progress. Compensation actions undo completed steps if a later step fails.

Keep transactions short. Do not put external calls inside a transaction. Group writes that must be consistent in the same transaction. Use `RetryableTransaction` to handle conflicts automatically.

The next chapter covers failure handling and retries: how to classify errors, configure timeouts, and build resilient applications.

14 Handling Failures and Retries

Every distributed system fails. Networks drop packets. Nodes crash. Leaders change. Disks fill up. An application that ignores these failures will eventually lose data, hang indefinitely, or corrupt state.

This chapter catalogs every failure mode a Kahuna client can encounter. For each failure, it explains what happened, whether the operation is safe to retry, and what the application should do.

14.1 Error Classification

Kahuna operations return a response type that indicates the outcome. When an operation fails, the client throws a `KahunaException`. The exception carries an error code that tells you what went wrong and whether you can retry.

The error codes fall into five categories:

14.1.1 Transient Errors (Retry Immediately)

MustRetry means the operation did not execute and is safe to retry. Common causes:

- A leader change occurred while the operation was in flight. The request reached a node that is no longer the leader for the target partition.
- The node has not yet reached a safe timestamp for the requested read.
- The routing table is stale. The client sent the request to the wrong partition.

The operation had no effect. Retry it immediately or after a short delay. The client's connection layer will route the retry to the current leader.

14.1.2 Conflict Errors (Retry with Fresh State)

Aborted means the operation executed partially but was rejected at commit time. The most common cause is a transaction conflict: another transaction modified a key that this transaction read or wrote.

The transaction had no effect (all writes were rolled back). You can retry, but you must start a new transaction session and re-read all keys. Do not retry the commit on the same session.

AlreadyLocked means a key the transaction tried to lock is held by another transaction. In pessimistic mode, this happens when the lock wait exceeds the transaction timeout. You can retry with a new session.

14.1.3 Overload Errors (Back Off and Retry)

AdmissionRefused means the server rejected the request because it is at its concurrency ceiling. The server is not broken. It is protecting itself from overload.

Wait before retrying. Use exponential backoff with jitter. If `AdmissionRefused` errors persist, the server is saturated and needs more capacity or the workload needs throttling.

You can influence how long the client waits for an admission slot with the `AdmissionWaitMs` transaction option. A non-zero value tells the client to wait up to that many milliseconds for a slot before failing.

14.1.4 Permanent Errors (Do Not Retry)

Errored means the operation failed permanently. Something is wrong at the server level (storage corruption, internal bug, unrecoverable state). Retrying will produce the same error.

Log the error and alert. Do not retry.

InvalidInput means the client sent a malformed request: an empty key, an invalid flag combination, a script syntax error. This is a bug in the application code. Fix the input.

14.1.5 Not-Found Responses

DoesNotExist (for keys, locks, and sequences) and **NotSet** (for conditional writes that failed their condition) are not errors. They are normal outcomes that indicate the resource does not exist or the condition was not met. The application should handle these in its logic, not in its error handler.

14.2 The Error Decision Table

Error Code	Meaning	Safe to Retry?	Action
MustRetry	Transient (leader change, stale route)	Yes, immediately	Retry the same operation
Aborted	Transaction conflict	Yes, with fresh state	Start a new session, re-read, retry
AlreadyLocked	Key locked by another transaction	Yes, with fresh state	Start a new session, retry
AdmissionRefused	Server overloaded	Yes, after backoff	Wait with exponential backoff, then retry
Errored	Permanent server error	No	Log and alert
InvalidInput	Malformed request	No	Fix the application code
DoesNotExist	Resource not found	N/A	Handle in application logic
NotSet	Conditional write failed	N/A	Handle in application logic

14.3 Retry Strategies

14.3.1 Simple Retry with Backoff

For standalone key-value operations (not inside a transaction), implement a retry loop with exponential backoff and jitter:

```

async Task<KahunaKeyValue> SetWithRetry(
    KahunaClient client, string key, string value, int maxRetries = 5)
{
    int attempt = 0;

    while (true)
    {
        try
        {
            return await client.SetKeyValue(key, value);
        }
        catch (KahunaException ex) when (
            ex.KeyValueErrorCode == KeyValueResponseType.MustRetry ||
            ex.KeyValueErrorCode == KeyValueResponseType.AdmissionRefused)
        {
            attempt++;

            if (attempt >= maxRetries)
                throw;

            int delayMs = Math.Min(50 * (1 << attempt), 2000);
            int jitter = Random.Shared.Next(0, delayMs / 2);
            await Task.Delay(delayMs + jitter);
        }
    }
}

```

The delay doubles on each attempt (50, 100, 200, 400, 800 ms) with a cap of 2 seconds. The jitter prevents multiple clients from retrying at the same instant.

For `MustRetry`, the delay can be shorter (the error is transient and the next attempt often succeeds immediately). For `AdmissionRefused`, a longer delay gives the server time to recover.

14.3.2 RetryableTransaction

For transactions, use the built-in `RetryableTransaction` helper (Chapter 6). It handles retry logic automatically:

```

await client.RetryableTransaction(
    new KahunaTransactionOptions
    {
        Timeout = 5000,
        Locking = KeyValueTransactionLocking.Pessimistic
    },
    async (session, ct) =>
    {
        KahunaKeyValue counter = await session.GetKeyValue("stats/visits");
        long count = long.Parse(counter.ValueAsString ?? "0");
        await session.SetKeyValue("stats/visits", (count + 1).ToString());
        await session.Commit(ct);
    }
)

```

```
    }
};
```

RetryableTransaction retries on Aborted, MustRetry, and AlreadyLocked. Its retry strategy:

- Up to 10 retries.
- Decorrelated jitter backoff with a median first delay of 50 milliseconds.
- Each delay is capped at 500 milliseconds.
- After 10 failed attempts, it throws a KahunaException with Aborted.

Any exception that is not Aborted, MustRetry, or AlreadyLocked propagates immediately. This includes InvalidInput, Errored, application logic exceptions, and cancellation.

14.3.3 When Not to Retry

Do not retry when:

- The error is InvalidInput. The request is malformed. Retrying sends the same bad request.
- The error is Errored. The server has a permanent problem. Retrying will not fix it.
- The operation has non-reversible side effects. If you called an external payment gateway before the Kahuna operation failed, retrying the entire workflow may charge the customer twice. Use idempotency guards (Chapter 9).

14.4 Timeout Configuration

Three layers of timeout apply to a Kahuna operation:

14.4.1 Operation Timeout

The DefaultOperationTimeout on the KahunaOptions client configuration sets a deadline for every operation that does not supply its own CancellationToken. The default is 30 seconds.

```
var client = new KahunaClient("https://localhost:2070", new KahunaOptions
{
    DefaultOperationTimeout = TimeSpan.FromSeconds(10)
});
```

If the operation does not complete within this timeout, the client throws an OperationCanceledException. The server may still process the request. The client does not know whether the operation succeeded.

14.4.2 Transaction Timeout

The Timeout option on KahunaTransactionOptions controls how long a transaction session can live on the server:

```
new KahunaTransactionOptions { Timeout = 5000 } // 5 seconds
```

When the timeout elapses, the server releases all locks held by the session and aborts any uncommitted transaction. This prevents a slow or crashed client from holding locks indefinitely.

Set the transaction timeout based on how long the transaction's work takes. A 5-second timeout is appropriate for most read-modify-write patterns. Long-running transactions (batch imports, multi-step workflows) may need 10 to 30 seconds.

14.4.3 Lock Wait Timeout

The wait parameter on `GetOrCreateLock` controls how long the client retries before giving up:

```
await client.GetOrCreateLock(  
    "jobs/send-email",  
    expiry: TimeSpan.FromSeconds(15),  
    wait: TimeSpan.FromSeconds(10),  
    retry: TimeSpan.FromMilliseconds(200)  
);
```

If the lock is not acquired within 10 seconds, `IsAcquired` is false. No exception is thrown.

14.4.4 Timeout Hierarchy

Timeout	Scope	Default	Set By
Operation timeout	Single RPC call	30 seconds	<code>KahunaOptions.DefaultOperationTimeout</code>
Transaction timeout	Entire session	5,000 ms (typical)	<code>KahunaTransactionOptions.Timeout</code>
Lock wait	Lock acquisition	0 (no wait)	<code>GetOrCreateLock</code> wait parameter
Admission wait	Transaction admission	0 (server default)	<code>KahunaTransactionOptions.AdmissionWaitMs</code>

14.5 Failure Scenarios

14.5.1 Leader Change During a KV Operation

1. The client sends a `SetKeyValue` request to node A (the current leader for the target partition).
2. A leader election starts. Node B becomes the new leader.
3. Node A receives the request but is no longer the leader. It rejects the request with `MustRetry`.
4. The client retries. The routing layer discovers the new leader (node B) and sends the request there.
5. The operation succeeds on node B.

The key point: `MustRetry` means the operation did not execute. The retry is safe. No data was written by the failed attempt.

14.5.2 Leader Change During a Transaction

1. The client opens a transaction session on node A.
2. The client sends several operations (reads and writes) within the session.
3. A leader change occurs. Node A is no longer the leader.
4. The next operation in the session returns `MustRetry` or `Aborted`.
5. The session is no longer valid. The client must start a new session.

When wrapped in `RetryableTransaction`, this happens automatically: the callback is invoked again with a fresh session, and all reads and writes are repeated from scratch.

14.5.3 Network Partition

1. The client cannot reach any Kahuna node.

2. Every operation times out with an `OperationCanceledException` or a gRPC `Unavailable` error.
3. The client should back off and retry. If the partition persists, the application must decide how to degrade.

Options for degradation:

- Return a cached value (if staleness is acceptable).
- Queue the operation for later retry.
- Return an error to the caller.
- Switch to a fallback data source.

14.5.4 Node Crash

1. The client is connected to node A, which crashes.
2. The gRPC connection drops. The client receives a transport-level error.
3. The client reconnects to another node in its URL list.
4. Operations resume on the surviving nodes.

If the client was constructed with multiple URLs, it distributes requests across available nodes automatically:

```
var client = new KahunaClient(new[]
{
    "https://node1:2070",
    "https://node2:2070",
    "https://node3:2070"
});
```

If a node is unreachable, the client skips it and sends the request to the next node in the round-robin rotation.

14.5.5 Transaction Conflict Under High Contention

1. Ten workers update the same counter concurrently.
2. Each starts a transaction, reads the counter, increments it, and commits.
3. Only one commit succeeds per round. The other nine receive `Aborted`.
4. Each retries with `RetryableTransaction`. The jitter spreads retries over time.
5. Eventually all ten updates complete.

High contention causes many retries. Two strategies reduce contention:

- **Pessimistic locking.** The first transaction locks the key. Others wait instead of reading and aborting. Less wasted work, but lower concurrency.
- **Reduce the conflict window.** Keep transactions short. Read, compute, write, commit. Do not hold a session open while doing unrelated work.

14.5.6 Admission Refused Under Load

1. The server has reached its maximum concurrent transaction count.
2. A new transaction request arrives.
3. The server rejects it with `AdmissionRefused`.
4. The client waits and retries.

To reduce `AdmissionRefused` errors:

- Set `AdmissionWaitMs` to a non-zero value. The server queues the request for up to that many milliseconds before rejecting it.
- Increase the server's admission ceiling (if the hardware supports it).
- Reduce the number of concurrent transactions from the application side (use a semaphore or work queue).

14.5.7 Timeout with Unknown Outcome

1. The client sends a `Commit` request.
2. The network drops the response.
3. The client's operation timeout fires. The client does not know whether the commit succeeded.

This is the most difficult failure mode. The server may have committed the transaction. The client cannot know without checking.

What to do:

- Read the keys that the transaction wrote. If they have the expected values, the commit succeeded.
- If the operation is idempotent (Chapter 9), retry the entire workflow. The idempotency guard prevents duplicate effects.
- If the operation is not idempotent and you cannot determine the outcome, log the uncertainty and alert. A human or automated reconciliation process must resolve the ambiguity.

The server is the transaction coordinator. If the commit request reached the server and the server decided to commit, the writes are applied even if the client never receives the response. The data is consistent. The only problem is that the client does not know it.

14.6 Application-Level Circuit Breaker

When Kahuna is unreachable for an extended period, retrying every operation wastes resources and adds latency. A circuit breaker stops sending requests after a threshold of consecutive failures and periodically probes to detect recovery.

```
public class KahunaCircuitBreaker
{
    private int _failureCount;
    private DateTime _openUntil = DateTime.MinValue;
    private readonly int _threshold;
    private readonly TimeSpan _openDuration;

    public KahunaCircuitBreaker(int threshold = 5, int openSeconds = 30)
    {
        _threshold = threshold;
        _openDuration = TimeSpan.FromSeconds(openSeconds);
    }

    public bool IsOpen => DateTime.UtcNow < _openUntil;

    public void RecordSuccess()
    {

```

```

        _failureCount = 0;
        _openUntil = DateTime.MinValue;
    }

    public void RecordFailure()
    {
        _failureCount++;

        if (_failureCount >= _threshold)
            _openUntil = DateTime.UtcNow + _openDuration;
    }
}

```

Usage:

```

if (circuitBreaker.IsOpen)
{
    // Fail fast. Do not send the request to Kahuna.
    return GetCachedValue(key);
}

try
{
    KahunaKeyValue result = await client.GetKeyValue(key);
    circuitBreaker.RecordSuccess();
    return result.ValueAsString;
}
catch (Exception)
{
    circuitBreaker.RecordFailure();
    return GetCachedValue(key);
}

```

After 5 consecutive failures, the circuit opens for 30 seconds. During that window, requests bypass Kahuna and use a fallback. After 30 seconds, the next request probes Kahuna. If it succeeds, the circuit closes. If it fails, the circuit stays open for another 30 seconds.

14.7 Error Handling Checklist

1. **Catch `KahunaException` and check the error code.** Do not catch the base `Exception` type and treat all errors the same.
2. **Retry `MustRetry` immediately.** The operation did not execute. The next attempt will likely succeed.
3. **Retry `Aborted` and `AlreadyLocked` with a new session.** Re-read all data. Do not reuse the old session.
4. **Back off on `AdmissionRefused`.** The server needs time to catch up.
5. **Do not retry `Errored` or `InvalidInput`.** These indicate a permanent problem.
6. **Use `RetryableTransaction` for transactions.** It handles retry, backoff, and session management.

7. **Set appropriate timeouts.** Operation timeout for individual calls, transaction timeout for sessions, admission wait for overloaded servers.
8. **Handle unknown outcomes explicitly.** When a timeout leaves the result unknown, check the state or use idempotency guards.

14.8 Summary

Kahuna errors fall into five categories: transient (`MustRetry`), conflict (`Aborted`, `AlreadyLocked`), overload (`AdmissionRefused`), permanent (`Errored`), and client bugs (`InvalidInput`). Each category demands a different response.

For standalone operations, implement a retry loop with exponential backoff and jitter. For transactions, use `RetryableTransaction`, which retries up to 10 times with decorrelated jitter backoff.

Configure timeouts at three levels: operation timeout (30 seconds default), transaction timeout (set per session), and lock wait (set per acquisition). A timeout with unknown outcome is the hardest failure mode: check the state or rely on idempotency guards.

A circuit breaker protects the application when Kahuna is unreachable for an extended period. It fails fast during the outage and probes periodically for recovery.

This chapter concludes Part II. The next chapter begins Part III (Understanding Kahuna Internals) with an architecture overview: the project layout, the component hierarchy, and how the three subsystems share a common structure.

15 Architecture Overview

Part II showed how to use Kahuna from the outside: keys, locks, sequences, transactions, and patterns for building distributed applications. Part III moves inside. The next ten chapters trace requests through the system, explain how data is partitioned and replicated, and describe how each subsystem works at the implementation level.

This chapter provides the map. It introduces the project layout, the component hierarchy, the shared patterns across subsystems, and the external libraries that Kahuna depends on. Every subsequent internal chapter zooms in on one part of this map.

15.1 Project Layout

The Kahuna repository contains several projects. Each has a distinct role:

Project	Description
<code>Kahuna.Core</code>	The embeddable engine. Contains all subsystem managers (key-value, locks, sequencer), persistence backends, the actor-based processing pipeline, composition, and inter-node communication.
<code>Kahuna.Server</code>	The server executable (<code>kahuna-server</code>). Hosts gRPC and REST endpoints, parses command-line options, and wires <code>Kahuna.Core</code> into an ASP.NET host.
<code>Kahuna.Client</code>	The .NET client library. Provides <code>KahunaClient</code> , gRPC and REST transport, request batching, and session management.
<code>Kahuna.Shared</code>	Types shared between client and server: enums (<code>KeyValueResponseType</code> , <code>LockResponseType</code> , <code>SequenceResponseType</code>), request and response DTOs, and protocol types.
<code>Kahuna.Control</code>	The command-line client (<code>kahuna-cli</code>). A thin wrapper over <code>Kahuna.Client</code> for interactive use.

Test and benchmark projects (`Kahuna.Client.Tests`, `Kahuna.Server.Tests`, `Kahuna.Benchmark`, `Kahuna.Microbenchmarks`) round out the repository.

The two most important projects for understanding internals are `Kahuna.Core` (where the engine lives) and `Kahuna.Server` (where the engine is hosted).

15.2 External Libraries

Kahuna depends on two libraries that are central to its architecture:

15.2.1 Kommander (Raft Consensus)

Kommander is the Raft consensus library. It provides:

- **IRaft**: the interface for proposing log entries, querying leader status, and registering replication callbacks.
- **Leader election**: heartbeat-based leader election within each Raft group.
- **Log replication**: append-only log with majority-based commit.

- **Hybrid Logical Clocks (HLC)**: timestamps that combine physical time with a logical counter for causal ordering.
- **Write-ahead log (WAL)**: durable log storage that Raft entries are written to before replication.
- **State transfer**: snapshot-based catch-up for followers that fall behind log compaction.

Kahuna does not implement consensus itself. Every durable write goes through Kommander's Raft protocol. Kommander handles leader election, log replication, and commit notification. Kahuna registers callbacks to apply committed entries to its own state.

15.2.2 Nixie (Actor System)

Nixie is the actor framework. It provides:

- **ActorSystem**: the container that manages actor lifecycles.
- **IActor<TRequest, TResponse>**: the interface that all Kahuna actors implement. Each actor processes messages one at a time in a single-threaded loop.
- **IActorRef<TActor, TRequest>**: a handle for sending messages to an actor.

Actors are Kahuna's concurrency model. Each key-value partition, lock partition, and sequencer partition has its own set of actors. Because each actor processes one message at a time, there is no need for locks on the in-memory state within an actor. Concurrency comes from having many actors, not from shared-memory parallelism within one.

15.3 The IKahuna Facade

The IKahuna interface is the contract between the server layer and the core engine. It declares every operation that Kahuna supports, organized into categories:

- **Key-value operations**: set, get, exists, delete, extend, batch variants, prefix queries, range scans.
- **Lock operations**: acquire, extend, release, query.
- **Sequence operations**: create, next value, reserve range, get, delete.
- **Transaction lifecycle**: start session, commit, rollback, close.
- **Transaction internals**: acquire exclusive locks, prepare mutations (2PC phase 1), commit mutations (2PC phase 2), rollback mutations, write-intent checks.
- **Script execution**: parse and execute transaction scripts.
- **Replication callbacks**: log restored, replication received, replication error, leader changed.
- **Range management**: register and unregister key ranges, state transfer for range splits.
- **Persistence**: flush pending writes, bootstrap from backup.

Every routed operation has two variants. The `LocateAnd*` variant (for example, `LocateAndTrySetKeyValue`) resolves which partition owns the key, checks whether this node is the leader for that partition, and either dispatches locally or forwards to the correct node. The direct `Try*` variant (for example, `TrySetKeyValue`) operates locally when the caller already knows that this node is the correct leader.

15.4 KahunaManager

`KahunaManager` is the class that implements `IKahuna`. It is the root of the component hierarchy. Its constructor calls `KahunaNodeComposer.Build()`, which assembles all internal components in a specific order:

1. Wrap the raw persistence backend in an `UnflushedOverlayPersistenceBackend`. This overlay caches writes that are committed in Raft but not yet flushed to disk, so reads do not miss recently committed values.
2. Create I/O schedulers (`FairReadScheduler`) for backend reads and writes.
3. Create internal stores: `SnapshotFloorStore`, `CompletionReceiptStore`, `TransactionRecordStore`, `PreparedIntentStore`.
4. Spawn the `BackgroundWriterActor`. This Nixie actor flushes committed Raft entries to the persistence backend asynchronously.
5. Create `LockManager`.
6. Create `KeyValuesManager`.
7. Wire the flush notification sink so the background writer can notify the key-value subsystem when writes are durable.
8. Create `SequencerManager`. It depends on `KeyValuesManager` because sequences store their durable state as key-value entries.
9. Register state transfer hooks with Raft for recovery and follower catch-up.

The result is a tree of components:

```
KahunaManager
├── KeyValuesManager
│   ├── KeyValueLocator (routes keys to partitions)
│   ├── KeyValueActor (per-partition, handles reads)
│   ├── PartitionWriteAggregatorActor (batches writes)
│   ├── DurableProposalSubmission (submits Raft proposals)
│   └── KeyValueReplicator (applies committed entries)
├── LockManager
│   ├── LockLocator (routes lock resources to partitions)
│   ├── LockActor (per-partition lock state)
│   ├── LockProposalActor (submits Raft proposals)
│   └── LockReplicator (applies committed entries)
├── SequencerManager
│   ├── SequenceLocator (routes sequence names to partitions)
│   └── SequenceActor (per-partition sequence state)
├── BackgroundWriterActor (async persistence)
├── PartitionPlacementCoordinator
└── IPersistenceBackend (RocksDB, SQLite, or Memory)
```

15.5 The Shared Pattern: Manager, Locator, Actor

All three subsystems follow the same layered architecture. Understanding this pattern once makes every subsystem easier to follow.

15.5.1 Manager

The manager is the entry point. `KeyValuesManager`, `LockManager`, and `SequencerManager` each own their locator, actors, and configuration. The `KahunaManager` delegates `IKahuna` method calls to the appropriate subsystem manager.

Manager classes are split into partial classes by responsibility. `KeyValuesManager`, for example, has separate files for admin operations, routed operations (the `LocateAnd*` variants), and local operations (the direct `Try*` variants).

15.5.2 Locator

The locator resolves a key (or lock resource, or sequence name) to the correct partition and then to the current leader for that partition. It uses the `DataPartitionRouter`, which maps key hashes to partition numbers and partition numbers to leader node addresses.

If this node is the leader for the target partition, the locator dispatches the request locally to the actor. If another node is the leader, the locator forwards the request over `IInterNodeCommunication`.

15.5.3 Actor

The actor holds the in-memory state for a partition. `KeyValueActor` holds the key-value entries for its partition. `LockActor` holds the lock states (owner, expiry, fencing token). `SequenceActor` holds the sequence states (high-water mark, block cache, idempotency entries).

Each actor implements `IActor<TRequest, TResponse>` from Nixie. It processes one message at a time. This single-threaded guarantee means the actor's state does not need locks or concurrent data structures. Within a partition, the actor is consistent by construction.

Multiple actors can exist per partition (determined by a consistent-hash ring), allowing parallelism within a partition for non-overlapping keys.

15.5.4 Proposal and Replication

When an actor needs to make a durable write, it submits a Raft proposal. The mechanism differs slightly by subsystem:

- **Locks** use a `LockProposalActor` that submits one proposal per lock mutation. This is straightforward because lock operations are individually small.
- **Key-values** use a `PartitionWriteAggregatorActor` that batches multiple writes into a single Raft proposal. This is important for throughput: one Raft round trip commits many key-value writes.
- **Sequences** store their durable state through the key-value subsystem. When a sequence needs to advance its high-water mark, it writes a key-value entry, which flows through the KV write pipeline.

After Raft commits a log entry, each subsystem has a replicator (`KeyValueReplicator`, `LockReplicator`) that applies the committed entry back to the actor's in-memory state. This is how followers stay in sync: they receive committed entries from the leader and apply them through the same replicator path.

15.6 Inter-Node Communication

When a request arrives at a node that is not the leader for the target partition, the locator forwards it to the correct node. Two implementations exist:

- **GrpcInterNodeCommunication**: the production implementation. Nodes communicate over gRPC. This is the same transport the client uses, but the inter-node variant carries internal metadata (partition ID, leader hint) that the client does not send.
- **MemoryInterNodeCommunication**: the test implementation. All nodes run in the same process. Messages are passed through in-memory queues. This allows integration tests to simulate a multi-node cluster without network overhead.

The interface (`IInterNodeCommunication`) abstracts the transport. The subsystem code does not know or care whether it is running in a production cluster or a test harness.

15.7 Persistence Backends

Kahuna supports three persistence backends, all implementing `IPersistenceBackend`:

Backend	Use Case
<code>RocksDbPersistenceBackend</code>	Production default. LSM-tree storage with high write throughput.
<code>SqlitePersistenceBackend</code>	Alternative for smaller deployments. Single-file database.
<code>MemoryPersistenceBackend</code>	Testing. All data in memory. Lost on restart.

An `UnflushedOverlayPersistenceBackend` wraps the chosen backend. This overlay solves a timing problem: a Raft entry is committed (replicated to a majority) before the `BackgroundWriterActor` flushes it to the persistence backend. Without the overlay, a read immediately after a committed write could miss the value because it is not yet on disk. The overlay caches committed-but-unflushed writes and serves them on reads.

The `BackgroundWriterActor` runs as a Nixie actor. It receives batches of committed entries and writes them to the persistence backend asynchronously. This decouples the Raft commit path (fast, in-memory) from the disk I/O path (slower, batched).

15.8 The Meta Partition

Partition 0 is the meta partition. It stores cluster-wide metadata that all nodes need:

- **Range map:** the mapping from key ranges to partition numbers. When the cluster splits or merges key ranges, the updated map is replicated through partition 0.
- **Snapshot floor:** the MVCC retention boundary. Reads older than this timestamp are not guaranteed to find data.
- **Transaction coordinator decisions:** the 2PC decision records for committed or aborted transactions.

The meta partition uses the same Raft group as any data partition: it has a leader, replicates to a majority, and supports state transfer for catch-up. Its special status comes from its content, not from a different mechanism.

15.9 Client Communication

The client communicates with the server through two protocols:

- **gRPC:** the primary protocol. Supports streaming, multiplexing, and efficient binary serialization via Protocol Buffers.
- **REST:** an alternative for clients that cannot use gRPC. The REST API covers the same operations but without streaming.

The `KahunaClient` distributes requests across server URLs in round-robin order. When constructed with multiple URLs, it rotates through them. If a node is unreachable, the client skips it on the next rotation.

The client also batches requests. The `GrpcBatcher` coalesces multiple operations headed for the same server into a single gRPC call. The `BatchCoalescingThreshold` and `BatchCoalescingDelayMs` options on `KahunaOptions` control when the batcher dispatches.

15.10 Summary

Kahuna's architecture is a layered system of managers, locators, and actors, built on Kommander for consensus and Nixie for concurrency.

Every request follows the same path: the client sends it to a server node. The server's `KahunaManager` delegates to the appropriate subsystem manager. The manager's locator resolves the key to a partition and a leader. If this node is the leader, the request is dispatched to the partition's actor. If not, it is forwarded to the correct node. Durable writes go through Raft proposals. Committed entries are applied by replicators and flushed to disk by the background writer.

The three subsystems (key-value, locks, sequences) share this pattern but differ in their write pipelines: key-values batch writes for throughput, locks submit individual proposals, and sequences piggyback on the key-value pipeline.

The next chapter traces a single request through every layer of this architecture, from the client's `SetKeyValue` call to the persistence backend and back.

16 The Request Lifecycle

Chapter 14 introduced Kahuna’s architecture: the project layout, the component hierarchy, and the shared Manager, Locator, Actor pattern across subsystems. This chapter traces a single request through every layer of that architecture, from the client’s `SetKeyValue` call to the persistence backend and back.

The goal is to build a complete picture of where each component sits in the request path. Every subsequent internals chapter zooms in on one part of this path. This chapter is the map that connects them all.

16.1 The Write Path: Setting a Key

A `SetKeyValue` call passes through seven stages before the value is durable: client batching, server dispatch, partition routing, actor processing, Raft proposal, replication, and persistence. Each stage is a distinct component with a distinct responsibility.

16.1.1 Stage 1: Client Batching

The application calls `KahunaClient.SetKeyValue`. The client selects a server URL from its list in round-robin order. It creates a `GrpcTrySetKeyValueRequest` and passes it to the `GrpcBatcher` for the selected URL.

The `GrpcBatcher` does not send each request individually. It uses gRPC bidirectional streaming to multiplex many operations over a single long-lived HTTP/2 stream. When a request arrives, the batcher enqueues it in a `ConcurrentQueue` and creates a `TaskCompletionSource` that the caller awaits. A single dispatch loop drains the queue and sends all pending items over the shared stream.

Two settings control coalescing behavior:

- `BatchCoalescingThreshold` (default 10): when fewer than this many requests are ready, the batcher waits briefly before sending.
- `BatchCoalescingDelayMs` (default 2): the maximum wait in milliseconds, randomized to avoid synchronization.

This batching is transparent to the caller. The caller awaits a task. The batcher groups that request with other concurrent requests, sends them over the stream, and completes the task when the server responds.

If a transport-level error occurs (gRPC connection drops), the batcher invalidates the shared connection and retries once. The `GrpcCommunication` layer above the batcher adds its own retry logic: up to 5 retries on `MustRetry` responses and 2 retries on transport failures.

16.1.2 Stage 2: Server Dispatch

The request arrives at the server’s gRPC service. `KeyValuesService` (which extends the generated `KeyValuer.KeyValuerBase`) receives the call in its `TrySetKeyValue` method. The method validates the input (non-empty key, non-negative expiry), extracts the key, value, flags, and durability from the protobuf request, and calls `keyValues.LocateAndTrySetKeyValue` on the `IKahuna` facade (implemented by `KahunaManager`).

`KahunaManager` delegates to `KeyValuesManager`, which delegates to `KeyValueLocator.LocateAndTrySetKeyValue`. The delegation chain is short: facade to subsystem manager to locator.

16.1.3 Stage 3: Partition Routing

`KeyValueLocator.LocateAndTrySetKeyValue` is the routing core. It determines which partition owns the key and whether this node is the leader for that partition.

The method starts with input validation. If the key is empty or the TTL is negative, it returns `InvalidInput` immediately. Then it resolves the partition:

```
(partitionId, generation, isKeyRange, descriptor) = LocateRangeWithMode(key)
```

`LocateRangeWithMode` uses `RangeRouting.Locate`, which hashes the key through the `KeySpaceRegistry` and the `DataPartitionRouter` to find the partition number. For key-range spaces (range-partitioned keys), it also checks whether the range is quiesced (in the middle of a split or merge). A quiesced range rejects writes with `MustRetry`.

After resolving the partition, the locator checks whether this node is the leader:

```
if (await raft.AmILeaderIfHosted(partitionId, cancellationToken))
{
    return await manager.TrySetKeyValue(...);
}
```

If this node is the leader, the request is dispatched locally. If not, the locator resolves the leader address:

```
string? leader = await TryWaitForLeader(partitionId, cancellationToken);
```

`TryWaitForLeader` calls `raft.TryResolveLeader`. If no leader is found (the election is still in progress, the partition is not hosted, or a `RaftException` occurs), the method returns null. The locator then returns `MustRetry` to the client.

If the leader is found and it is a remote node, the locator forwards the request over `IInterNodeCommunication`:

```
response = await interNodeCommunication.TrySetKeyValue(leader, ...);
```

The forwarded request carries the same parameters as the original, plus a `routedGeneration` that tracks the range map version at the coordinating node. The remote node uses this generation to detect stale routes.

16.1.4 Stage 4: Actor Processing

When the request is dispatched locally, `KeyValuesManager.TrySetKeyValue` sends it to the `KeyValueActor` for the target partition through a consistent-hash router. The router selects one of several actors within the partition based on the key hash.

The actor processes one message at a time. It checks conditions: compare-and-swap revision, compare-and-swap value, flags (`SetIfNotExists`, `SetIfExists`), and MVCC write intents from concurrent transactions. If the conditions pass, the actor prepares the write.

For persistent (durable) writes, the actor creates a Raft proposal. It serializes the key, value, revision, timestamps, and flags into a `RaftProposalEntry` and submits it to the write pipeline.

16.1.5 Stage 5: Write Aggregation and Raft Proposal

The proposal enters the `PartitionWriteAggregatorActor`. This is where key-value writes differ from lock writes. Lock writes submit one Raft proposal per operation. Key-value writes

are batched: the aggregator collects multiple proposals for the same partition and submits them as a single Raft log entry.

The aggregator is a Nixie actor with per-partition state. When a proposal arrives, the aggregator enqueues it in the partition's buffer. Two triggers cause the buffer to flush:

1. The buffer reaches `MaxBatchItems` or `MaxBatchBytes`.
2. The linger timer fires (a configurable delay that gives more proposals time to arrive).

When the buffer flushes, the aggregator selects a batch and dispatches it. Before dispatch, each item is checked for staleness: if the range map moved since the item was admitted, or if the item exceeded its maximum queue age, it is released with a retryable failure.

Valid items are flattened into a single `RaftProposalEntry[]` array. The aggregator calls `executor.ReplicateAsync(partitionId, entries, cancellationToken)`, which submits the batch to Raft as one proposal.

Raft replicates the log entry to a majority of nodes. Each node appends the entry to its write-ahead log. When a majority acknowledges the append, the entry is committed. The `RaftBatchReplicationResult` reports whether each entry committed or failed.

The aggregator maps the result back to individual submissions. Each `KeyValueProposalRequest` carries a reference to the originating actor and its reply promise. When Raft commits, the aggregator calls `Complete` on each request, which sends a `CompleteProposal` message back to the actor. The actor clears the replication intent, applies the committed state to the entry, and resolves the response promise. On failure, the actor returns `MustRetry` or `Aborted`.

Only one batch per partition is in flight at any time. While a batch is in Raft, new proposals accumulate in the buffer behind it. When the in-flight batch completes, the aggregator immediately dispatches the buffered items as the next batch.

16.1.6 Stage 6: Replication and State Application

After Raft commits the log entry, the `KeyValueReplicator` applies it to the node's state. The replicator runs on every node: the leader and all followers.

The replicator deserializes the committed log entry with `ReplicationSerializer.UnserializeKeyValueMessage`. Based on the operation type (`TrySet`, `TryDelete`, `TryExtend`), it performs three actions in order:

1. **Record in the unflushed overlay.** `UnflushedKeyValueWritesIndex.Record` caches the committed write so that reads can see it before the background writer flushes it to disk.
2. **Enqueue for persistence.** The replicator sends a `QueueStoreKeyValue` message to the `BackgroundWriterActor`. This enqueues the write for asynchronous disk flush.
3. **Update the in-memory actor state.** The replicator sends an `InvalidateOrApply` message to the `KeyValueActor` through the persistent router. The actor either applies the new value (on followers, where the actor did not process the original request) or invalidates its cached entry (on the leader, where the actor already has the value from processing the request).

The replicator also records a completion receipt. This receipt lets the server answer re-commits for the same transaction with `Committed` instead of `MustRetry`, even after the write intent and MVCC snapshot are gone.

16.1.7 Stage 7: Persistence

The `BackgroundWriterActor` is a Nixie actor that runs a periodic flush timer. The default interval is configurable (5 seconds by default). On each tick, it drains the dirty queue and writes batches to the `IPersistenceBackend` (RocksDB, SQLite, or Memory).

Writes are batched up to 1,024 items or 512 KB per batch. The actor processes lock flushes and key-value flushes separately. After each flush, it advances durability floors, runs checkpoint operations (so Raft can compact its write-ahead log), and cleans up old revisions.

If a write fails, the actor retries up to 5 times with backoff. Items that fail all retries stay in the queue for the next flush cycle.

The separation between Raft commit and disk persistence is a key design choice. A Raft commit means the value is replicated to a majority of nodes in memory and in the write-ahead log. The background flush is an optimization: it moves data from the WAL to the structured storage backend. If a node crashes before the flush, it recovers from the WAL on restart.

16.1.8 The Response Path

After the write commits in Raft, the response travels back through the layers:

1. The write aggregator sends a `CompleteProposal` message to the `KeyValueActor`. The actor clears the replication intent, applies the committed state, and resolves the `TaskCompletionSource` that the gRPC handler is awaiting.
2. The gRPC service constructs the response protobuf with the response type, revision, last-modified timestamp, and elapsed time.
3. The server sends the response over the gRPC stream.
4. The `GrpcBatcher` on the client side receives the response and resolves the matching `TaskCompletionSource`.
5. `KahunaClient.SetKeyValue` wraps the result in a `KahunaKeyValue` object and returns it to the application.

If the request was forwarded to a remote node, the inter-node communication layer receives the response from the remote leader and passes it back to the locator on the coordinating node.

16.2 The Read Path: Getting a Key

The read path is shorter than the write path. Reads do not create Raft proposals and do not go through the write aggregator. The critical difference is how the read path ensures consistency.

16.2.1 Client to Locator

The client calls `KahunaClient.GetKeyValue`. As with writes, the request goes through the `GrpcBatcher` and arrives at the server's gRPC service. The service calls `keyValues.LocateAndTryGetValue`, which delegates through `KahunaManager` and `KeyValuesManager` to `KeyValueLocator.LocateAndTryGetValue`.

The locator resolves the partition with `RouteKey(key)` (the same hash-based routing as writes).

16.2.2 Leadership Confirmation for Reads

Here the read path diverges from the write path. For writes, checking `AmILeaderIfHosted` is sufficient: if the leader belief is wrong, the Raft proposal will fail. Writes are self-validating because replication itself fails on a deposed leader.

Reads are different. A node that believes it is the leader but is actually partitioned from the majority will serve stale data as a successful response. There is no replication step to catch the stale belief.

To prevent stale reads, the locator calls `ConfirmLeadershipForRead`:

```
if (await ConfirmLeadershipForRead(partitionId, cancellationToken))
    return await manager.TryGetValue(...);
```

`ConfirmLeadershipForRead` delegates to `raft.ConfirmLeadershipIfHosted`, which performs a Raft read-index check. This check confirms that the node is still the leader by contacting a quorum. If the quorum confirms, the read proceeds. If not, the locator returns `MustRetry`.

This is the mechanism that provides linearizable reads. Every read confirms leadership before serving data, so a read always reflects all writes that committed before it started.

16.2.3 Local Read

If leadership is confirmed, the locator calls `manager.TryGetValue`, which routes the request to the `KeyValueActor` through the consistent-hash router. The actor looks up the key in its in-memory state and returns the value, revision, and timestamps.

If the key is not in the actor's memory (it was evicted or never loaded), the actor reads from the persistence backend. The `UnflushedOverlayPersistenceBackend` wraps the raw backend and intercepts reads. If the key is in the unflushed overlay (committed in Raft but not yet flushed to disk), the overlay returns the cached value. If not, the read falls through to the underlying RocksDB, SQLite, or Memory backend.

16.2.4 Remote Forward

If this node is not the leader, the locator calls `TryWaitForLeader` to find the leader's address. If the leader is found, the request is forwarded:

```
response = await interNodeCommunication.TryGetValue(leader, ...);
```

The remote leader runs the same leadership confirmation and local read. The response travels back through the inter-node communication layer to the coordinating node, and from there back to the client.

16.3 Read Path vs Write Path

The two paths share the same routing and dispatch layers. They diverge at two points:

Aspect	Write Path		Read Path
Leadership check	AmILeaderIfHosted (belief-based)	(belief-based)	ConfirmLeadershipForRead (quorum-confirmed)
Raft involvement	Submits a proposal, waits for majority commit	No	proposal (read-only, confirmed via read-index)
Write aggregator	Batches proposals per partition		Not involved
Replicator	Applies committed entries on all nodes		Not involved
Persistence	Background flush after commit		Reads from overlay or backend

Writes are self-validating: a proposal on a deposed leader fails at the replication step, so a belief-based check is sufficient. Reads need the quorum confirmation because there is no replication step to catch a stale leader.

16.4 Synchronous vs Asynchronous Boundaries

The request lifecycle has two asynchronous boundaries where the request “detaches” from the caller and continues independently:

1. **Raft replication.** The write aggregator submits a Raft proposal and awaits the result. The caller’s thread is free while Raft replicates the entry to followers. The `TaskCompletionSource` on the `DurableProposalSubmission` completes when Raft commits.
2. **Background persistence.** After Raft commits, the `KeyValueReplicator` enqueues the write for disk flush but does not wait for it. The response returns to the client before the value is on disk. The unflushed overlay ensures reads see the committed value in the interim.

Everything else is synchronous from the caller’s perspective: the client awaits the response, the server processes the request inline, and the locator routes it within the same async call chain.

16.5 Where Latency Comes From

Each stage adds latency. In a healthy cluster, the dominant cost is Raft replication (network round trips to followers).

Stage	Typical Contribution
Client batching	0 to 2 ms (coalescing delay)
Network to server	Network dependent
Server dispatch and routing	Microseconds
Actor processing	Microseconds
Write aggregation linger	0 to linger setting (default: configurable)
Raft replication	1 network round trip to majority
Response to client	Network dependent

For reads, the Raft replication step is replaced by the read-index confirmation, which also requires a quorum round trip. Reads that hit the in-memory actor cache avoid disk I/O. Reads that miss the cache pay the additional cost of a backend lookup.

Inter-node forwarding adds one extra network hop. A request that arrives at the wrong node travels: client to coordinating node, coordinating node to leader, leader back to coordinating node, coordinating node back to client. This is why the client distributes requests across all nodes: most requests hit the correct leader on the first hop.

16.6 Failure Scenarios

16.6.1 Leader Not Found

The locator calls `TryWaitForLeader` and receives null. The node may not host this partition, or an election may be in progress. The locator returns `MustRetry`. The client retries on another node.

No data was read or written. The retry is safe.

16.6.2 Leader Changes Mid-Request

The request arrives at the leader. A new election starts. The Raft proposal fails because the node is no longer the leader. The write aggregator releases the proposal with a transient failure. The actor returns `MustRetry`.

On the write path, the proposal did not commit. No data was written. On the read path, the leadership confirmation fails, and the locator returns `MustRetry` before any data is read.

16.6.3 Background Writer Lag

Under heavy write load, the `BackgroundWriterActor` may fall behind. Committed values accumulate in the unflushed overlay. Reads still succeed because the overlay serves committed-but-unflushed values. The write-ahead log grows until the background writer catches up.

If the dirty queue is not empty after a flush cycle, the background writer sends itself another flush message immediately, without waiting for the next periodic timer tick. This self-scheduling drains the backlog as fast as the backend allows.

16.7 Summary

A key-value write passes through seven stages: client batching, server dispatch, partition routing, actor processing, write aggregation, Raft replication, and background persistence. The client batches requests over a shared gRPC stream. The server routes each request to the correct partition leader. The write aggregator batches multiple proposals into one Raft entry for throughput. Raft replicates the entry to a majority. The replicator applies the committed entry to all nodes. The background writer flushes data to disk asynchronously.

A key-value read follows the same routing path but diverges at the leadership check. Reads use a quorum-confirmed read-index check instead of a belief-based leader check. This prevents a partitioned leader from serving stale data. Reads do not create Raft proposals and do not go through the write aggregator.

The two asynchronous boundaries are Raft replication (the caller awaits the commit) and background persistence (the response returns before the disk flush). The unflushed overlay bridges the gap between Raft commit and disk flush.

The next chapter examines how Kahuna partitions data across nodes: how keys map to partitions, how partitions map to nodes, and how the system rebalances when nodes join or leave.

17 Partitioning and Data Distribution

Chapter 15 traced a single request through every layer of the architecture. One step in that path was partition routing: the locator resolved a key to a partition number and found the leader for that partition. This chapter explains how that routing works, how partitions are created and split, and how the system rebalances data as it grows.

Partitioning determines three properties of the system: scalability (how much data and throughput a cluster can handle), data locality (which keys are stored together), and failure isolation (how much data is affected when a node goes down). A single partition is a bottleneck. Too many partitions waste resources. The right partitioning strategy depends on the workload.

17.1 Two Routing Modes

Kahuna supports two ways to map keys to partitions: hash-based routing and key-range routing. Each mode has different trade-offs for data distribution and query patterns.

17.1.1 Hash-Based Routing

Hash-based routing is the default. Every key-space prefix that is not explicitly registered for key-range routing uses hash-based routing.

The routing works as follows. The `DataPartitionRouter` extracts the key-space prefix from the key (the portion before the last `/` separator) and computes an ordinal hash of that prefix. The hash maps to one of the partitions in a fixed pool.

The pool consists of partitions numbered from 1 to `InitialPartitions`. Partition 0 is the meta partition (described later in this chapter). It is excluded from the data pool. The `PoolSize` equals the `InitialPartitions` value from the Raft configuration, which is set at cluster startup with the `--initial-cluster-partitions` flag (default 3).

```
DataPartitionRouter.Locate("orders/123/status")
→ prefix = "orders/123"
→ hash = HashUtils.InversePrefixedHash("orders/123/status", '/', PoolSize)
→ partitionId = hash + FirstUserPartitionId (1)
→ result: partition 2
```

Hash-based routing has two properties:

1. **Uniform distribution.** Keys spread evenly across partitions. No partition is a hot spot unless the application writes heavily to a single prefix.
2. **No key locality.** Keys with related prefixes (such as `orders/100` and `orders/101`) may land on different partitions. Range scans across prefixes require querying multiple partitions.

The partition pool is fixed. It does not grow or shrink. Partitions created by key-range splits receive higher IDs and are not part of the hash pool. Hash-based routing is static: the same key always maps to the same partition.

Because the pool is fixed, hash-based routing does not need a generation fence. The `routedGeneration` for hash-routed keys is always 0.

17.1.2 Key-Range Routing

Key-range routing assigns contiguous intervals of keys to partitions. Keys that are close in sort order are stored on the same partition. This enables efficient range scans and prefix queries within a single partition.

Key-range routing is opt-in. A key space becomes key-range routed only by explicit registration through the `KeySpaceRegistry`. The registry maintains a per-node map from key-space prefixes to a `RoutingMode` value: either `Hash` (the default) or `KeyRange`.

When a key arrives, the `KeySpaceRegistry` extracts the key space (the portion before the last `/`). It looks up the routing mode for that key space. If the mode is `KeyRange`, the request is routed through the `RangeMap`. If the mode is `Hash`, the request goes through the `DataPartitionRouter` as described above.

The registry uses a `ConcurrentDictionary` with zero-allocation span lookups for performance. It reconciles its state against the replicated range map on every range map update.

17.1.3 RangeRouting: The Single Source of Truth

The `RangeRouting` static class is the single entry point for all key-to-partition resolution. Both the `KeyValueLocator` and the leader-side direct-write path call through `RangeRouting`. This ensures that every routing decision uses the same logic.

`RangeRouting.Locate` takes the registry, range map, data partition router, and key. It returns a `(partitionId, generation)` tuple. For hash-routed keys, the generation is 0. For key-range keys, the generation comes from the `RangeDescriptor` that covers the key.

`RangeRouting.LocateWithMode` returns additional information: whether the key is key-range routed and, if so, the covering `RangeDescriptor`. The locator uses this to check for quiesced ranges and to carry the generation through inter-node forwards.

17.2 The Range Map

The range map is an immutable in-memory data structure that maps key-range spaces to their partition assignments. Each entry is a `RangeDescriptor`.

17.2.1 RangeDescriptor

A `RangeDescriptor` is a sealed record with these fields:

Field	Description
KeySpace	The key-space prefix (for example, "t:r").
StartKey	Inclusive lower bound of the range. Null means negative infinity.
EndKey	Exclusive upper bound of the range. Null means positive infinity.
PartitionId	The Raft group that serves this range.
Generation	Incremented on every split, merge, or move. Used for generation fencing.
QuiescedUntil	Timestamp deadline until which writes are refused (data is in transit).
QuiesceOwner	The move operation that opened the quiesce window.
QuiesceStartKey	Start of the sub-interval being quiesced.
QuiesceEndKey	End of the sub-interval being quiesced.

Bounds use ordinal string comparison. The interval is half-open: $[\text{StartKey}, \text{EndKey})$. A descriptor with `StartKey = "a"` and `EndKey = "m"` contains keys from "a" up to but not including "m".

The `Contains(key)` method checks whether a key falls within the half-open interval. The `IsQuiescedAt(key, now)` method checks whether a key falls within the quiesced sub-interval and the quiesce deadline has not lapsed.

17.2.2 RangeMap Lookups

The `RangeMap` groups descriptors by key space and sorts them by `StartKey` within each group. Lookups use binary search:

- `Find(keySpace, key)`: $O(\log n)$ search for the descriptor that covers the key.
- `FindCovering(key)`: extracts the key space from the key, then calls `Find`.
- `FindAll(keySpace)`: returns all descriptors for a key space in sorted order.
- `FindIntersecting(keySpace, startKey, endKey)`: $O(\log n + k)$ for range queries, where k is the number of matching descriptors.

The range map enforces an invariant: within each key space, descriptors must be contiguous with no gaps and no overlaps. The `Validate` method checks this invariant and is called after every mutation.

17.2.3 RangeMapStore

The `RangeMapStore` is the replicated source of truth for the range map. It wraps an immutable `RangeMap` and replicates changes through Raft on the meta partition (partition 0).

All mutations go through a single method: `MutateAsync(transform)`. This method serializes access with a semaphore, validates the result, replicates the new descriptor set through Raft, and swaps the in-memory map. Only the meta partition leader can mutate the range map.

The store uses snapshot semantics: each replicated entry carries the full descriptor set, not a delta. Replay is idempotent. On startup, the store loads a durable snapshot from disk. Periodic checkpointing (every 32 mutations by default) writes a fresh snapshot and lets Kommander trim the write-ahead log.

17.3 The Meta Partition

Partition 0 is the meta partition. Chapter 14 introduced it briefly. This chapter adds the partitioning-specific details.

The meta partition stores the authoritative range map. Every range split, merge, or move replicates through partition 0's Raft group. All nodes receive the updated range map through normal Raft replication.

The meta partition uses the same Raft protocol as data partitions. It has a leader, replicates to a majority, and supports state transfer for follower catch-up. Its special status comes from its content (the range map, transaction coordinator decisions, the MVCC retention boundary), not from a different mechanism.

Schema-log spaces (key spaces with a /meta suffix) are never registered as key-range routed. The KeySpaceRegistry enforces this rule.

17.4 Range Splits

When a key-range partition grows too large or too hot, the system splits it into two smaller ranges. A split takes a range $[S, E)$ on partition P and produces two ranges: $[S, K)$ on the original partition P and $[K, E)$ on a new partition P' .

17.4.1 The Split Lifecycle

The RangeSplitter executes the split as a multi-step transaction. The steps proceed in order:

1. **Locate the covering range.** Find the RangeDescriptor for the range that contains the split key K .
2. **Validate bounds.** Confirm that $S < K < E$ in ordinal order. Both halves must be non-empty.
3. **Check minimum size.** Probe both halves to ensure each has at least `--range-split-min-range-size` keys (default 10). This prevents splits that produce ranges too small to be useful.
4. **Create the new partition.** Call `CreatePartitionAsync` with `RaftRoutingMode.Unrouted`. The new partition P' exists in Raft but does not receive routed traffic yet.
5. **Bulk copy.** Copy all keys in $[K, E)$ from P to P' at a consistent MVCC snapshot timestamp. The `KvStateMachineTransfer` handles this copy in pages of 256 entries, each checksummed.
6. **Quiesce.** Acquire an exclusive range lock on $[K, E)$ within partition P . Publish a quiesce window on the descriptor with a 30-second deadline (`QuiesceTtlMs = 30_000`). While the range is quiesced, writes to keys in $[K, E)$ are rejected with `MustRetry`. Settle any durable intents for keys in the moving range.

7. **Final catch-up copy.** Capture writes that occurred between the MVCC snapshot and the quiesce point. Transfer range locks for the moving sub-interval. Gather and hand off transaction state: completion receipts, transaction records, and prepared intents.
8. **Atomic cutover.** Call `RangeMapStore.MutateAsync` to replace the original descriptor with two new descriptors: `[S, K)` on `P` with `generation+1` and `[K, E)` on `P'` with `generation+1`. This is a single Raft proposal on the meta partition. Both descriptors receive a bumped generation.
9. **Release quiesce.** Release the exclusive range lock on the original partition. The quiesce window on the descriptor expires by its deadline.

After cutover, the original partition `P` still contains the `[K, E)` rows. These orphan rows are unreachable through routing and are not deleted immediately. They consume storage but do not affect correctness.

The split must run on the meta partition (partition 0) leader. This ensures that range map mutations are serialized.

17.4.2 Quiesce: Two Complementary Guards

The quiesce mechanism uses two guards that work together:

1. **Exclusive range lock.** This is an actor-local lock that blocks writes at the actor processing level. It is ordered against concurrent writes: a write that arrives before the lock is acquired proceeds; a write that arrives after is rejected. This guard handles the local partition.
2. **Descriptor quiesce.** This is a replicated flag on the `RangeDescriptor`. It survives leadership changes and is visible to all nodes. It has a deadline (not a flag that must be cleared), so it expires automatically if the split fails. This guard handles forwarded requests from other nodes.

Both guards are necessary. The actor-local lock prevents races on the partition leader. The descriptor quiesce prevents a redirected request from bypassing the lock on a different node.

17.4.3 Automatic Split Triggers

The `RangeSplitTrigger` monitors key-range partitions and triggers splits automatically. It has two branches:

Count branch (slow cadence, approximately 60 seconds). The trigger samples the key count for each descriptor. When the count reaches the `--range-split-threshold` (default 1000 keys), a split is triggered.

Load branch (fast cadence, approximately 5 seconds). The trigger evaluates a predicate: operations per second must exceed `--range-split-load-threshold` AND the write-ahead log queue depth must exceed `--range-split-load-min-queue-depth` (default 8). The predicate must hold for the full `--range-split-load-window` (default 15 seconds) before a split fires. This debounce prevents splits from transient load spikes.

Both branches enforce a settle window. After a split, the two child ranges are excluded from re-evaluation for `--range-split-settle-window` seconds (default 10). This prevents cascading splits.

An indivisibility guard prevents splits when the write-frequency histogram shows extreme imbalance (all writes target a single key). A relief guard skips load-based splits when no peer node is alive, because a single-node cluster gains no redistribution benefit from splitting.

All splits are serialized through a semaphore. Only one split runs at a time.

17.4.4 Split Key Selection

The RangeSplitPolicy computes the split key from an ordered sample of keys in the range:

- **Normal distribution.** The split key is the median (50th percentile). This produces two halves of equal size.
- **Monotonic-append detection.** If 75% or more of recently written keys cluster in the top 25% of the ordinal range (a common pattern for time-series or auto-incrementing keys), the split key moves to the 75th percentile. This gives the hot tail its own range.
- **Write-centroid path.** When the write-frequency histogram is warm, the policy finds the split index that best bisects cumulative writes. This balances write load rather than key count.

The policy clamps the split index so both halves have at least `--range-split-min-range-size` keys.

17.5 Range Merges

When two adjacent key-range partitions are both under a minimum size, the system merges them. A merge takes [A, B) on partition P1 and [B, C) on partition P2 and produces [A, C) on P1. Partition P2 is retired.

17.5.1 The Merge Lifecycle

The RangeMerger follows a similar pattern to the splitter:

1. **Validate adjacency.** Confirm that the two ranges are in the same key space and share a boundary (EndKey of the first equals StartKey of the second).
2. **Quiesce the source range.** Acquire an exclusive range lock on [B, C) and publish a quiesce window on P2's descriptor.
3. **Settle intents.** Wait for durable intents in the source range to complete.
4. **Copy data.** Transfer all keys from [B, C) on P2 to P1.
5. **Atomic cutover.** Replace both descriptors with a single [A, C) descriptor on P1 with generation+1.
6. **Release and retire.** Release the quiesce lock. The caller retires partition P2.

A merge copies in one direction only (source to destination), because merge candidates are small ranges.

17.5.2 Automatic Merge Triggers

The RangeMergeTrigger scans for adjacent pairs where both ranges have fewer than `--range-merge-min-size` keys (default 10). It runs on the meta partition leader. It skips warm partitions (where operations per second exceed the load threshold) to prevent merge-split oscillation: a range that was recently split due to load should not be merged back immediately.

17.6 The Generation Fence

The generation fence prevents stale routing after a split or merge. When a range splits, the generation on both child descriptors is incremented. Any request that was routed with the old generation is rejected.

17.6.1 How It Works

When a request is forwarded from one node to another, the coordinating node includes a `routedGeneration` value. This is the generation of the descriptor that the coordinating node used to route the request.

On the receiving node, `RangeRouting.ResolveForDirectWrite` checks the routed generation against the live descriptor. The check returns one of four results:

Result	Meaning
Ok	The generation matches. The write is admitted.
NoDescriptor	The range moved away from this partition. The descriptor is gone.
GenerationFenced	The routed generation does not match the live generation. The routing is stale.
Quiesced	The range is in the middle of a split or merge. Writes are temporarily refused.

When a request is fenced (`GenerationFenced`, `NoDescriptor`, or `Quiesced`), the server returns `MustRetry`. The client retries, and the routing layer discovers the updated range map and sends the request to the correct partition.

The fence is checked at two points:

1. When the locator routes the request (before dispatching locally or forwarding).
2. At the leader-side direct-write path (just before proposing to Raft).

The second check catches a race: the range map may update between the routing decision and the proposal submission.

Hash-routed keys do not use the generation fence. Their routing is static (the pool never changes), so the generation is always 0.

17.6.2 Deferred Write Staleness

The write aggregator batches multiple proposals for efficiency (Chapter 15). A proposal may sit in the aggregator's buffer while a split completes. Before flushing the buffer, the aggregator calls `RangeRouting.HasKeyRangeMovedSinceAdmission` to check whether any buffered key's range moved since the key was admitted. Stale items are released with a retryable failure.

17.7 State Transfer During Splits

The `KvStateMachineTransfer` handles the bulk copy of data during splits and merges.

17.7.1 Export

The export reads keys in the range `[startKey, endKey)` at a consistent MVCC snapshot. It pages through the data in batches of 256 entries. Each page is checksummed with FNV hash for integrity verification.

Only persistent keys are transferred. Ephemeral (in-memory-only) data is not transferable.

17.7.2 Import

The import applies entries to the destination partition's persistence backend. The import is idempotent: applying the same page twice produces the same result.

17.7.3 Lock Transfer

Range locks are transferred separately. The `FilterAndClamp` method filters the source partition's lock state for locks that fall within the moving sub-interval. These locks are re-imported on the destination partition.

If leadership changes during the transfer, `EnsureLocksOnDestinationLeaderAsync` re-imports the locks on the new leader.

17.8 Prefix Operations and Splits

Prefix operations (`GetByBucket` and `ScanAllByPrefix`) scan all keys that share a prefix. When a key-range space splits, the prefix scan on a single partition may miss data that moved to the new partition.

`RangeRouting.IsPrefixOpSafe` checks whether a key-range space was split in a way that would make a prefix scan incomplete. If the check returns false, the operation must query multiple partitions or return an error.

17.9 Server Configuration

These command-line flags control partitioning behavior:

Flag	Default	Description
<code>--initial-cluster-partitions</code>	3	Number of partitions at cluster startup.
<code>--range-split-threshold</code>	1000	Key count that triggers an automatic split.
<code>--range-split-min-range-size</code>	10	Minimum keys per half after a split.
<code>--range-split-settle-window</code>	10s	Cooldown after a split before re-evaluation.
<code>--range-merge-min-size</code>	10	Key count below which adjacent ranges may merge.
<code>--range-collection-interval</code>	60s	Interval between count-based sampling passes.
<code>--range-split-load-threshold</code>	0 (disabled)	Operations per second for load-based split.
<code>--range-split-load-min-queue-depth</code>	8	WAL queue depth gate for load-based split.
<code>--range-split-load-window</code>	15s	Sustained load window before a split fires.
<code>--range-split-load-poll-interval</code>	5s	Poll frequency for load signals.

Load-based splitting is disabled by default (`--range-split-load-threshold` is 0). Enable it by setting a non-zero threshold.

17.10 Failure Scenarios

17.10.1 Stale Routing After a Split

1. Node A routes a write to partition P using generation 5.
2. A split completes. The range map updates. The new generation is 6.
3. The write arrives at partition P's leader. The generation fence check finds `routedGeneration` (5) does not match the live generation (6).
4. The leader returns `MustRetry`.
5. The client retries. The routing layer reads the updated range map and sends the request to the correct partition (P or P') with generation 6.

The generation fence prevents the write from landing on a partition that no longer owns the key.

17.10.2 Split During an Active Transaction

1. A transaction reads key K on partition P.

2. A split moves the range containing K to partition P'.
3. The transaction tries to write key K. The routing layer or the generation fence rejects the write with `MustRetry`.
4. If wrapped in `RetryableTransaction`, the transaction starts over with a fresh session.

The transaction is not corrupted. The split causes a transient failure, and the retry mechanism handles it.

17.10.3 State Transfer Failure Mid-Split

1. The bulk copy (step 5) or catch-up copy (step 7) fails due to a network error or node crash.
2. The split is aborted. The original range remains intact on partition P.
3. The `RangeSplitTrigger` cleans up the orphaned partition P' that was created in step 4.
4. On the next trigger cycle, the trigger re-evaluates and may start a new split attempt.

Because the atomic cutover (step 8) did not execute, the range map is unchanged. No data is lost.

17.10.4 Indivisible Range

1. The split trigger fires on a range where all writes target a single key.
2. The `RangeSplitPolicy` detects that the write-centroid imbalance is at the maximum (1.0).
3. The trigger skips the split. The range stays as is.

A range with a single hot key cannot be split further. The application must distribute writes across multiple keys to avoid this bottleneck.

17.11 Summary

Kahuna distributes keys across partitions using two routing modes. Hash-based routing is the default: a fixed pool of partitions, uniform distribution, no key locality. Key-range routing is opt-in: contiguous key intervals on the same partition, efficient range scans, but requires range management.

The `RangeMap` stores the mapping from key intervals to partitions. The `RangeMapStore` replicates it through the meta partition (partition 0). Every mutation is a single Raft proposal.

Range splits follow a multi-step protocol: bulk copy, quiesce, catch-up copy, atomic cutover. The quiesce mechanism uses two guards (an actor-local lock and a replicated descriptor deadline) to prevent writes during the transition. The generation fence rejects requests that use a stale routing generation.

Automatic triggers split ranges when they grow too large (count-based) or too hot (load-based). Automatic merges recombine small adjacent ranges. Both triggers use settle windows and guards to prevent oscillation.

The next chapter examines Raft consensus and replication: how Kahuna uses `Kommander` for leader election, log replication, and state transfer.

18 Consensus and Replication

Every write in Kahuna goes through Raft consensus. Every linearizable read confirms leadership through a quorum check. Raft is the foundation that makes Kahuna fault-tolerant: it ensures that all replicas agree on the same ordered sequence of operations, even when nodes crash or networks partition.

This chapter explains how Kahuna uses Raft. It covers the consensus problem, the Raft algorithm, per-partition Raft groups, log entry types, linearizable reads, Hybrid Logical Clocks, and the replication pipeline that applies committed entries to each subsystem.

18.1 The Consensus Problem

A distributed system that stores data on multiple nodes must answer a question: when two nodes disagree about the state of a key, which one is correct?

Without a consensus protocol, the answer is undefined. A network partition can leave two nodes with different values for the same key, and neither node knows which value is authoritative. This is the split-brain problem.

Raft solves split-brain by electing a single leader for each group of replicas. All writes go through the leader. The leader replicates each write to a majority of nodes before the write is committed. A write that is committed on a majority cannot be lost, even if a minority of nodes crash.

The guarantee is precise: Raft ensures that all nodes apply the same sequence of log entries in the same order. If entry N is committed, every node that is alive and reachable will eventually have entry N at position N in its log. This is the foundation of consistency in Kahuna.

18.2 Kommander: The Raft Library

Kahuna does not implement Raft itself. It uses Kommander, an external Raft library. Kommander provides the IRaft interface, which exposes:

- **Leader election.** Heartbeat-based election with randomized timeouts.
- **Log replication.** Append-only log with majority-based commit.
- **Write-ahead log (WAL).** Durable storage for log entries before replication. Three implementations: RocksDB (production), SQLite (alternative), and in-memory (tests).
- **Hybrid Logical Clocks (HLC).** Timestamps that combine physical time with a logical counter for causal ordering across nodes.
- **State transfer.** Snapshot-based catch-up for followers that fall behind log compaction.
- **Membership management.** SWIM protocol for failure detection, learner promotion, and graceful decommission.

Kahuna registers callbacks with Kommander to receive events: committed log entries, leadership changes, and restore notifications. Kommander handles the protocol. Kahuna handles the state.

18.3 Per-Partition Raft Groups

Chapter 16 described how Kahuna distributes keys across partitions. Each partition has its own independent Raft group. This means each partition has its own leader, its own log, and its own election cycle.

Per-partition groups provide two benefits:

1. **Independent leadership.** Different partitions can have different leaders. If node A leads partitions 1 and 2, and node B leads partition 3, a failure of node A triggers elections only for partitions 1 and 2. Partition 3 is unaffected.
2. **Independent throughput.** Each Raft group processes proposals independently. A slow proposal on partition 1 does not block proposals on partition 2.

Kommander creates a partition with `CreatePartitionAsync(partitionId, mode, hashRange, cancellationToken)`. The `mode` parameter specifies the routing mode (routed or unrouted). Unrouted partitions are created during range splits before the cutover (Chapter 16).

The default cluster starts with a configurable number of partitions (`--initial-cluster-partitions`, default 3). Partition 0 is the meta partition. Partitions 1 through N are data partitions. Range splits create additional partitions with higher IDs.

18.3.1 Shared Executor Pool

Each Raft group needs a thread to process proposals and heartbeats. With many partitions, a dedicated thread per partition wastes resources. Kommander provides a shared executor pool (`--raft-enable-shared-executor-pool`, default `true`) that multiplexes all partition executors onto a bounded thread pool. This is necessary for clusters with hundreds or thousands of partitions.

18.3.2 Partition Quiescence

Idle partitions that receive no proposals and no client reads are quiesced (`--raft-enable-quiescence`, default `true`). A quiesced partition stops sending per-partition heartbeats and relies on the SWIM protocol for failure detection. When a new proposal or read arrives, the partition wakes up.

Quiescence reduces network traffic in clusters with many partitions where only a fraction are active at any time. The `QuiesceAfter` timeout (default 1500 ms) controls how long a partition must be idle before it quiesces.

18.4 Leader Election

Raft uses heartbeat-based leader election. The leader sends periodic heartbeat messages to all followers. If a follower does not receive a heartbeat within its election timeout, it becomes a candidate and starts an election.

18.4.1 Election Timing

Kommander uses randomized election timeouts to prevent split votes:

Parameter	Default	Description
HeartbeatInterval	500 ms	How often the leader sends heartbeats to followers.
StartElectionTimeout	2000 ms	Lower bound of the randomized election timeout.
EndElectionTimeout	4000 ms	Upper bound of the randomized election timeout.
VotingTimeout	1500 ms	How long a candidate waits for a quorum of votes.
CheckLeaderInterval	250 ms	How often the timer fires to check leader state.
TimerInitialDelay	2500 ms	Grace period after startup before timers fire.

Each follower picks a random timeout between `StartElectionTimeout` and `EndElectionTimeout`. The randomization ensures that followers do not all start elections at the same time after a leader failure. The follower with the shortest timeout becomes a candidate first and usually wins the election.

If an election fails (no candidate receives a majority), each candidate adds a random increment between `StartElectionTimeoutIncrement` (100 ms) and `EndElectionTimeoutIncrement` (200 ms) to its timeout and tries again. This further spreads out retries.

The critical constraint: `HeartbeatInterval` must be less than `StartElectionTimeout`. If heartbeats are slower than the election timeout, followers start unnecessary elections.

18.4.2 Pre-Vote Protocol

Kommander uses a pre-vote protocol. Before a follower starts a real election (which increments the term and forces other nodes to step down), it sends a pre-vote request to check whether it would win.

A pre-vote does not increment the term. If the pre-vote fails (the majority does not respond or already has a leader), the follower does not disrupt the cluster. This prevents a problem where a partitioned node repeatedly increments its term and forces the healthy majority to step down when the partition heals.

18.4.3 The Election Sequence

1. A follower's election timeout fires. No heartbeat arrived from the leader.
2. The follower sends pre-vote requests to all other voters.
3. If a majority responds positively, the follower becomes a candidate and increments its term.
4. The candidate sends `RequestVote` messages to all voters.
5. Each voter grants its vote to the first candidate it receives in the new term (at most one vote per term).
6. If the candidate receives votes from a majority, it becomes the leader.
7. The new leader sends a heartbeat immediately to establish its authority.

8. The new leader commits a barrier entry (a no-op) to confirm that its log is up to date. The `LeadershipBarrierTimeout` (default 10 seconds) is the maximum wait for this barrier to commit.

18.4.4 Leadership Change Notification

When a partition's leader changes, Kommander calls the `OnLeaderChanged` callback. Kahuna receives this through the `ReplicationService` and notifies all subsystem managers. This lets each manager update its local routing state and reject requests that target partitions where this node is no longer the leader.

18.5 Log Entry Types

Every durable mutation in Kahuna is serialized into a Raft log entry. The `ReplicationSerializer` encodes and decodes these entries. Each entry carries a type string that identifies its content.

Type String	Content	Partition
"kv"	Key-value mutations: set, delete, extend.	Data partition
"lock"	Lock mutations: lock, unlock, extend.	Data partition
"rangemap"	Range-descriptor map snapshot.	Meta partition (0)
"snapshotfloor"	Snapshot-floor hold registry (MVCC retention boundary).	Meta partition (0)
"coorddecision"	Durable coordinator decision record delta (2PC commit/abort).	Data partition
"receipt"	Completion receipts (used during split/merge handoff).	Data partition
"txnrecord"	Transaction record transitions (init, commit, abort).	Data partition
"preparedintent"	Prepared-intent transitions (prepare, resolve, remove).	Data partition

Key-value and lock mutations are the most common types. They represent the application-visible writes. The remaining types are internal coordination state for transactions, range management, and MVCC.

18.5.1 Proposing Entries

Kommander exposes three methods for submitting log entries:

- `ReplicateLogs(partitionId, type, data)`: submit a single entry.
- `ReplicateLogs(partitionId, type, IReadOnlyList<byte[]>)`: submit a homogeneous batch (all entries share the same type string).
- `ReplicateEntries(partitionId, IReadOnlyList<RaftProposalEntry>)`: submit a heterogeneous batch where each entry can have a different type.

Chapter 15 described how the `PartitionWriteAggregatorActor` batches key-value writes. It uses `ReplicateEntries` to submit multiple KV mutations as a single Raft proposal. Lock mutations use individual `ReplicateLogs` calls.

18.5.2 Two-Phase Commit in the Log

Some proposals use explicit two-phase commit within Raft. The `ReplicateLogs` method accepts an `autoCommit` parameter. When `autoCommit` is false, the entry is proposed but not committed until the caller explicitly calls `CommitLogs(partitionId, ticketId)` or `RollbackLogs(partitionId, ticketId)`. Kahuna uses this for transaction coordination.

18.6 The Replication Pipeline

When Raft commits a log entry, every node in the group must apply it to its local state. Kahuna uses a replicator/restorer pattern for this.

18.6.1 Replicators: Normal Operation

During normal operation, `Kommander` calls the `OnReplicationReceived` callback for each committed log entry. The `ReplicationService` forwards this to `KahunaManager`, which dispatches based on the entry type.

The `KeyValueReplicator` handles "kv" entries. For each committed key-value mutation (`TrySet`, `TryDelete`, `TryExtend`), it performs four steps in order:

1. **Register pending.** The durability tracker records that this log entry is pending, which prevents the durability floor from advancing past it until the background writer flushes it to disk.
2. **Record in the unflushed overlay.** The unflushed write index caches the committed value so reads can see it before the background writer flushes it.
3. **Enqueue for persistence.** The replicator sends a `QueueStoreKeyValue` message to the `BackgroundWriterActor` for asynchronous disk flush.
4. **Invalidate or apply on the actor.** The replicator sends an `InvalidateOrApply` message to the `KeyValueActor` through the consistent-hash router. On the leader, the actor invalidates its cached entry (it already has the value from processing the original request). On followers, the actor applies the new value.

After these steps, the replicator records a completion receipt. It also records the write in the write-frequency registry for key-range partitions, which the split trigger uses.

The `LockReplicator` follows the same pattern for "lock" entries: register pending, record in the unflushed lock overlay, enqueue for persistence, and send `InvalidateOrApply` to the `LockActor`.

18.6.2 Restorers: Startup and Recovery

When a node starts up or a new leader is elected, committed log entries must be replayed to rebuild in-memory state. Kommander calls the `OnLogRestored` callback for each entry in the write-ahead log.

The `KeyValueRestorer` handles "kv" entries during restore. It performs a simpler sequence than the replicator:

1. Register pending in the durability tracker.
2. Record in the unflushed overlay.
3. Enqueue for persistence.
4. Record the completion receipt.

The restorer does not send `InvalidateOrApply` to actors. During restore, actors are not yet active. They are rebuilt after the restore completes.

The `LockRestorer` follows the same simplified pattern for lock entries.

18.6.3 The Wiring Layer

The `ReplicationService` is an ASP.NET `BackgroundService` that wires Raft events to `IKahuna`:

```
raft.OnLogRestored      → kahuna.OnLogRestored
raft.OnReplicationReceived → kahuna.OnReplicationReceived
raft.OnReplicationError  → kahuna.OnReplicationError
raft.OnLeaderChanged     → kahuna.OnLeaderChanged
raft.OnMembershipChanged → OnMembershipChanged
```

On startup, the service joins the cluster (either as a new member or by joining an existing cluster with `--join-existing`). On shutdown, it unwires events and optionally performs a graceful leave (`--graceful-leave-on-shutdown`).

18.7 Linearizable Reads

Chapter 15 described the read path: the locator calls `ConfirmLeadershipForRead` before serving a read. This section explains the protocol in detail.

18.7.1 The Problem with Local Reads

A node that believes it is the leader can serve reads from its local state. But what if the node is no longer the leader? A network partition can isolate a node from the majority. The isolated node still believes it is the leader (no heartbeat timeout fired yet), but a new leader was elected on the majority side. The isolated node's state is stale.

If the isolated node serves reads from its local state, those reads return stale data as a successful response. There is no replication step (as with writes) to catch the stale belief. The client receives outdated values and does not know they are outdated.

18.7.2 The Read-Index Protocol

`ConfirmLeadershipAsync` implements the read-index protocol from section 6.4 of the Raft dissertation. The protocol works in three steps:

1. **Record the current commit index.** The leader notes the highest committed log index at the time the read starts.

2. **Confirm leadership with a quorum.** The leader sends a lightweight message to all followers. A majority must respond, confirming that they still recognize this node as their leader in the current term. If the majority does not respond within `LeadershipConfirmationTimeout` (default 2 seconds), the confirmation fails.
3. **Wait for local application.** The leader waits until its local state machine has applied all entries up to the recorded commit index. This ensures the read reflects all committed writes.

If all three steps succeed, the read is linearizable: it reflects every write that committed before the read started, and no write that committed after.

18.7.3 Coalescing

Multiple concurrent reads can share a single confirmation round. When several `ConfirmLeadershipAsync` calls overlap, they coalesce into one quorum check. A confirmation that completed within the last heartbeat interval is reused for subsequent reads. This means that under steady load, the cost is approximately one quorum round-trip per heartbeat interval, regardless of read volume.

18.7.4 `AmILeader` vs `ConfirmLeadershipAsync`

Two leadership checks serve different purposes:

Method	Mechanism	Use Case
<code>AmILeaderQuick</code>	Local published state. No network call.	Write path: sufficient because a write proposal on a deposed leader fails at the Raft replication step.
<code>ConfirmLeadershipAsync</code>	Quorum confirmation. Network round-trip.	Read path: necessary because reads have no replication step to catch a stale leader.

Writes are self-validating. A write proposal submitted to a deposed leader is rejected by Raft because the deposed leader cannot replicate to a majority. The proposal fails, and the client retries.

Reads are not self-validating. A deposed leader can serve stale data from its local state without any rejection mechanism. The quorum confirmation is the mechanism that prevents this.

18.8 Hybrid Logical Clocks

Kahuna uses Hybrid Logical Clocks (HLC) for cross-node event ordering. An HLC timestamp combines three components:

Component	Type	Description
L	long	Physical timestamp in unix epoch milliseconds.
C	uint	Logical counter. Differentiates events with the same physical time.
N	int	Node ID. Breaks ties between events on different nodes.

18.8.1 Total Ordering

HLC timestamps form a total order. Comparison follows a priority chain: L first, then C, then N. Two events on different nodes with the same physical timestamp are ordered by their counter. Two events with the same timestamp and counter are ordered by node ID. No two events produce the same (L, C, N) tuple.

18.8.2 How HLC Advances

The clock advances differently for local events and received messages:

Local event (send or local). The clock computes $\text{newL} = \max(\text{currentL}, \text{physicalTime})$. If newL equals currentL, the counter increments. If newL is greater, the counter resets to zero. The clock swaps the new value using a lock-free compare-and-swap (CAS) loop. Zero allocation per event.

Receive event. The clock computes $\text{newL} = \max(\text{currentL}, \text{messageL}, \text{physicalTime})$. If all three are equal, the counter is $\max(\text{currentC}, \text{messageC}) + 1$. This is the standard HLC receive algorithm from Kulkarni et al.

18.8.3 Encoding

The HLC packs L and C into a single 64-bit long: the high 42 bits store L (unix milliseconds, valid to approximately year 2109), and the low 22 bits store C (approximately 4.19 million events per millisecond). If the counter overflows 22 bits, the clock rolls to (L+1, 0) to preserve causality.

18.8.4 Why HLC

Physical clocks alone are insufficient for ordering events across nodes, because wall clocks drift. Logical clocks (Lamport clocks) provide causal ordering but have no relationship to physical time. HLC combines both: it tracks causality like a logical clock and stays close to physical time like a wall clock. This makes HLC timestamps useful both for ordering (MVCC, conflict detection) and for human-readable timestamps (last-modified fields, expiration deadlines).

18.9 Voter and Learner Roles

Each node in a Raft cluster has a role that determines its participation in consensus:

Role	Votes?	Receives Replication?	Description
Voter	Yes	Yes	Full participant. Counts toward quorum in elections and commits.
Learner	No	Yes	Receives committed entries but does not vote. Cannot become leader.
Leaving	No	Yes	Committed during graceful decommission. Replicas are evacuated to other nodes.

18.9.1 Learner Promotion

A new node joins the cluster as a learner. It receives committed log entries and builds up its state without affecting the cluster's quorum size. When the learner's log catches up to within `LearnerPromotionLag` entries (default 10) of the leader and stays within that threshold for `LearnerPromotionStableWindow` (default 3 seconds), Kommander promotes it to voter automatically.

This two-phase join prevents a slow new node from reducing the cluster's availability. If a learner joined as a voter immediately, its slow replication would block commits (the quorum would include a node that is far behind).

18.9.2 Graceful Decommission

When a node is decommissioned with `--graceful-leave-on-shutdown`, it transitions to the Leaving role. The cluster evacuates its partition replicas onto surviving nodes. After evacuation completes, the node is removed from the cluster roster.

18.10 Replication Factor

The replication factor controls how many copies of each partition exist in the cluster. The default is 0, which means full replication: every voter hosts every partition. This is appropriate for small clusters (3 to 5 nodes).

For larger clusters, a fixed replication factor (for example, 3) limits each partition to a subset of nodes. This reduces storage and network overhead. The replication factor can be set per partition with `SetReplicationFactorAsync`.

18.11 State Transfer

When a follower falls far behind the leader's log (because it was offline or the WAL was compacted), it cannot catch up by replaying log entries. The entries it needs are gone.

Kommander handles this with state transfer. The leader sends a snapshot of the partition's state to the follower. Kahuna registers three state transfer handlers:

- `IRaftStateMachineTransfer`: transfers key-value data during range splits and merges (Chapter 16).
- `IRaftSystemStateTransfer`: transfers system state (meta partition) for node repair.
- `IRaftPartitionStateTransfer`: transfers user data partition state for follower catch-up.

After the snapshot is applied, the follower resumes normal log replication from the leader's current position.

18.12 WAL Compaction

The write-ahead log grows as entries accumulate. Periodically, Kommander compacts the WAL by discarding entries that all nodes have applied. The `CompactEveryOperations` setting (default 10,000) controls how often compaction runs.

Before compacting, the system checks retention holds. The `SetMinRetainIndex` and `AcquireRetentionHold` methods let subsystems prevent compaction of entries they still need (for example, entries needed for state transfer).

After compaction, entries before the compaction point are gone. A follower that needs those entries must use state transfer instead of log replay.

18.13 Failure Scenarios

18.13.1 Leader Failure

1. The leader crashes or becomes unreachable.
2. Followers stop receiving heartbeats. Each follower's election timeout is random (between 2000 and 4000 ms).
3. The follower with the shortest remaining timeout becomes a candidate first. It sends pre-vote requests.
4. If the pre-vote succeeds, the candidate increments its term, sends `RequestVote` messages, and wins the election.
5. The new leader sends a heartbeat and commits a barrier entry.
6. Normal operation resumes. The new leader serves reads and writes for the partition.

Committed entries are not lost. The new leader has all committed entries (Raft's election rule ensures the candidate with the most up-to-date log wins).

18.13.2 Network Partition

1. The cluster splits into a majority partition (for example, 2 of 3 nodes) and a minority partition (1 node).
2. The majority elects a new leader and continues to serve reads and writes. Commits succeed because the majority forms a quorum.
3. The minority cannot commit writes (no quorum). The node on the minority side may still believe it is the leader for some partitions, but its proposals fail.
4. Reads on the minority side fail if `ConfirmLeadershipAsync` is used (the quorum check fails). Without the quorum check, the minority node would serve stale data.
5. When the partition heals, the minority node receives the committed entries it missed and catches up.

18.13.3 Split Vote

1. Two followers start elections at the same time (their timeouts expire simultaneously).

2. Each receives some votes but neither receives a majority.
3. The election fails. Both candidates wait a random increment (100 to 200 ms) before trying again.
4. The randomized increment makes it unlikely that both candidates time out simultaneously again.
5. One candidate wins the next round.

The pre-vote protocol reduces the impact of split votes. A candidate that loses the pre-vote does not increment its term, so the cluster's term does not advance unnecessarily.

18.13.4 Log Divergence After Partition Heal

1. A network partition separates the old leader from the majority.
2. The majority elects a new leader and commits new entries.
3. The old leader may have uncommitted entries in its log (entries it proposed but could not replicate to a majority).
4. When the partition heals, the old leader discovers the new term and steps down.
5. Raft reconciles the logs: the old leader's uncommitted entries are overwritten by the new leader's committed entries. The log converges to the committed sequence.

Uncommitted entries are lost. This is correct: they were never acknowledged to the client (the proposal did not succeed), so no data loss occurs from the client's perspective.

18.13.5 Stale Reads from a Deposed Leader

1. Node A is the leader. A network partition isolates it from the majority.
2. Node B becomes the new leader on the majority side. New writes commit on node B.
3. Node A still believes it is the leader (its election timeout has not yet fired).
4. A client sends a read to node A.
5. Node A calls `ConfirmLeadershipAsync`. The quorum check fails because node A cannot reach a majority.
6. Node A returns `MustRetry`. The client retries on another node and reaches the current leader.

Without `ConfirmLeadershipAsync`, step 5 would not exist. Node A would serve stale data from its local state, and the client would receive outdated values without any indication.

18.14 Summary

Kahuna uses Raft consensus through the Kommander library. Each partition has its own Raft group with independent leadership, log, and election cycle. The shared executor pool and partition quiescence make this practical for clusters with many partitions.

Leader election uses randomized timeouts and a pre-vote protocol to prevent disruption. Committed entries are applied through the replicator/restorer pattern: replicators handle normal operation; restorers handle startup and recovery.

Linearizable reads use the read-index protocol: the leader confirms its authority with a quorum before serving a read. Concurrent confirmations coalesce for efficiency. This prevents a deposed leader from serving stale data.

Hybrid Logical Clocks provide total ordering across nodes by combining physical time with a logical counter. HLC timestamps are used for MVCC, conflict detection, and human-readable timestamps.

The replication pipeline transforms committed log entries into state changes across all subsystems. Eight log entry types cover key-value mutations, lock mutations, range map changes, transaction coordination, and MVCC retention.

The next chapter examines the persistence layer: how Kahuna stores data on disk, how the background writer flushes committed entries, and how the unflushed overlay bridges the gap between Raft commit and disk write.

19 The Actor Model and Concurrency

Traditional concurrent data structures use locks. A thread acquires a lock, reads or writes shared state, and releases the lock. This works, but it is error-prone: deadlocks, priority inversions, and forgotten unlock calls are common sources of bugs in concurrent systems.

Kahuna takes a different approach. It uses the actor model for concurrency. Each partition's state is owned by an actor that processes messages one at a time in a single-threaded loop. There are no locks on the in-memory state within an actor. Concurrency comes from having many actors, not from shared-memory parallelism within one.

This chapter explains how Kahuna uses the Nixie actor system, how messages are routed to actors, and how the handler pattern structures actor logic.

19.1 The Actor Model

The actor model has three rules:

1. **Isolated state.** Each actor owns its state. No other actor or thread can read or write that state directly.
2. **Message passing.** Actors communicate by sending messages. A message is a request object that the actor processes.
3. **Sequential processing.** Each actor processes one message at a time. The next message is not processed until the current one completes.

These three rules eliminate data races by construction. If only one thread ever touches the state, there is no race. If actors communicate only through messages, there is no shared mutable state.

The trade-off is that actors cannot share data structures. If two actors need the same data, one must send a message to the other. This adds latency compared to a direct memory read, but it removes the need for synchronization.

19.2 Nixie: The Actor Framework

Kahuna uses Nixie, an external actor framework. Nixie provides three building blocks: the actor interface, the actor reference, and the actor system.

19.2.1 The Actor Interface

Nixie defines two variants of the actor interface:

Fire-and-forget. The sender does not wait for a response.

```
public interface IActor<in TRequest> where TRequest : class
{
    public Task Receive(TRequest message);
}
```

Request/response. The sender awaits a response.

```
public interface IActor<in TRequest, TResponse>
    where TRequest : class
    where TResponse : class?
{
}
```

```
public Task<TResponse?> Receive(TRequest message);
}
```

Both variants have a single method: `Receive`. The actor system calls this method for each message in the actor's inbox. The actor processes the message and returns a result (or completes the task for fire-and-forget actors).

19.2.2 The Actor Reference

An `IACTORRef` is a handle for sending messages to an actor. It provides:

- `Send(message)`: fire-and-forget delivery. The message is enqueued and the caller continues.
- `Ask(message)`: request/response delivery. The caller awaits the result.
- `TrySend(message)`: returns false if the inbox is full (bounded inbox mode).
- `TryAsk(message)`: returns false and a null task if the inbox is full.

The caller never accesses the actor's internal state. All interaction goes through the reference.

19.2.3 The Actor System

The `ActorSystem` is the container that manages actor lifecycles. It provides:

- `Spawn<TActor, TRequest, TResponse>(name, args)`: create an actor with constructor arguments.
- `SpawnWithOptions<>()`: create an actor with inbox bounds and control message classifiers.
- `Get<>(name)`: find an actor by name.
- `StartPeriodicTimer`: send a message to an actor on a repeating interval.
- `ScheduleOnce`: send a message to an actor after a delay.
- `Shutdown<> / GracefulShutdownAll`: stop actors and drain their inboxes.

19.2.4 The Mailbox and Processing Loop

Each actor has an inbox implemented as a `ConcurrentQueue`. When a message arrives, the actor runner enqueues it and checks whether the processing loop is idle. If idle, it schedules the actor on the thread pool.

The processing loop drains messages one at a time:

1. Dequeue the next message from the inbox.
2. Call `await Actor.Receive(message)`.
3. If more messages are in the queue, continue to step 1.
4. If the queue is empty, mark the actor as idle and return the thread.

A compare-and-swap flag (`processing`) ensures that only one thread runs the loop at any time. This is the single-threaded guarantee: even though the actor runs on pool threads, it never runs on two threads simultaneously.

The actor runner uses `ThreadPool.UnsafeQueueUserWorkItem` instead of `Task.Run` to avoid `ExecutionContext` capture overhead. This is a performance optimization for high-throughput actors.

19.2.5 Bounded Inbox

An actor can be created with a maximum inbox size. When the inbox is full, `Send` and `Ask` throw an `ActorBusyException`. The message is never enqueued. The caller can catch this exception and return `MustRetry` to the client.

Some messages are classified as control messages. Control messages bypass the inbox bound and are delivered ahead of ordinary messages. Within each class (control vs ordinary), messages are delivered in FIFO order. Control messages overtake the ordinary backlog but maintain order among themselves.

In Kahuna, the following message types are classified as control messages for `KeyValueActor`:

- `CompleteProposal` and `ReleaseProposal` (Raft proposal lifecycle)
- `InvalidateOrApply` (follower replication)
- `ResumeRead` (deferred disk reads)
- `FlushAck` (durability floor advancement)
- `Collect` (periodic garbage collection)
- `GetSafeTimestamp` (MVCC timestamp queries)

These messages must not be blocked by a full inbox. A `CompleteProposal` message that is rejected would leave a Raft proposal permanently pending.

19.3 Consistent-Hash Routing

Each partition can have multiple actor instances. Messages are routed to a specific instance based on the key's hash. This allows parallelism within a partition for non-overlapping keys.

Nixie provides a `ConsistentHashActorStruct` router. The router creates `N` instances of the actor type. When a message arrives, the router computes the target instance:

```
instance = instances[(message.GetHash() & int.MaxValue) % instances.Count]
```

The message must implement `IConsistentHashable`, which exposes a `GetHash()` method. The hash of the key determines which actor instance processes the message.

This is a simple modulo routing, not a virtual-node ring. The same key always routes to the same actor instance. Different keys may route to different instances, which allows concurrent processing of messages for non-overlapping keys within the same partition.

The number of actor instances per partition is set by the `KeyValueWorkers` configuration. More workers means more parallelism within a partition, but also more memory (each actor holds its own B-tree cache).

19.4 Kahuna's Actor Hierarchy

Kahuna uses five types of actors, each with a distinct responsibility:

Actor	Interface	Role
KeyValueActor	IActor<KeyValueRequest, KeyValueResponse>	Holds key-value entries for a partition. Processes reads, writes, and transaction operations.
LockActor	IActor<LockRequest, LockResponse>	Holds lock state for a partition. Processes lock, unlock, and extend operations.
SequenceActor	IActor<SequenceRequest, SequenceResponse>	Holds sequence block allocation state. Serves next-value requests from a reserved block.
PartitionWriteAggregatorActor	IActor<PartitionWriteMessage, PartitionWriteAck>	Batches key-value write proposals for Raft replication.
BackgroundWriterActor	IActor<BackgroundWriteRequest>	Flushes committed Raft entries to the persistence backend. Fire-and-forget.

The first three actors hold partition state. The last two are infrastructure actors that handle the write pipeline.

19.5 KeyValueActor

The KeyValueActor is the most complex actor in Kahuna. It holds the in-memory state for key-value entries in a partition and dispatches operations to handler classes.

19.5.1 In-Memory State

The actor holds four data structures:

B-tree cache. A BTree<string, KeyValueEntry> with order 32. The B-tree stores key-value entries in sorted order by key. The sorted structure enables efficient prefix scans (ScanByPrefix) and range scans (GetByRange). The B-tree is a custom implementation in Kahuna.Core/Utils/BTree.cs.

Prefix write intents. A Dictionary<string, KeyValueWriteIntent> keyed by prefix. When a transaction operates on a bucket (prefix scan), it registers a write intent on the prefix. Other transactions that touch the same prefix detect the intent and handle conflicts.

Range locks. A Dictionary<string, List<KeyValueRangeLock>> keyed by prefix. Range locks protect sub-intervals within a key space during splits and transactions. Multiple non-overlapping range locks can coexist on the same prefix.

In-flight proposals. A Dictionary<int, KeyValueProposal> keyed by proposal ID. Each entry tracks a Raft proposal that is awaiting commit. When the replicator sends a CompleteProposal message, the actor resolves the proposal's TaskCompletionSource.

19.5.2 The Handler Pattern

The KeyValueActor constructor creates approximately 25 handler instances. All handlers share a KeyValueContext object that provides access to the actor's state, the persistence backend, and the Raft interface.

The Receive method dispatches messages to handlers based on the message type:

```
public async Task<KeyValueResponse> Receive(KeyValueRequest message)
{
    switch (message.Type)
    {
        case KeyValueRequestType.TrySet:
            return await trySetHandler.Execute(message);

        case KeyValueRequestType.TryGet:
            return await tryGetHandler.Execute(message);

        case KeyValueRequestType.TryDelete:
            return await tryDeleteHandler.Execute(message);

        // ... ~25 more cases
    }
}
```

Each handler is a class with an Execute method that takes a KeyValueRequest and returns a KeyValueResponse. The handler classes live in Kahuna.Core/KeyValues/Handlers/. There are 41 handler files covering all key-value operations:

- **CRUD handlers.** TrySetHandler, TryGetHandler, TryDeleteHandler, TryExtendHandler, TryExistsHandler.
- **Batch handlers.** TryGetByBucketHandler, TryGetByRangeHandler, TryScanByPrefixHandler.
- **Transaction handlers.** TryPrepareMutationsHandler, TryCommitMutationsHandler, TryRollbackMutationsHandler.
- **Lock handlers.** TryAcquireExclusiveLockHandler, TryAcquireExclusivePrefixLockHandler, TryAcquireExclusiveRangeLockHandler.
- **Lifecycle handlers.** CompleteProposalHandler, InvalidateOrApplyHandler, ResumeReadHandler, EvictPartitionHandler, TryCollectHandler.

The handler pattern keeps the Receive method short. Each handler encapsulates one operation's logic: input validation, state lookup, conflict detection, proposal creation, and response construction.

19.5.3 Backpressure

The actor implements two layers of backpressure:

1. **Inbox bound.** If the actor's inbox is full, the consistent-hash router catches the `ActorBusyException` and returns `MustRetry`. The message is never enqueued.
2. **Read backpressure.** If the persistence backend's read scheduler is saturated, the handler throws a `ReadBackpressureExceededException`. The actor catches it and returns `MustRetry`.

Both layers protect the system from overload. The client retries after a short delay.

19.5.4 Periodic Collection

Every 500 operations, the actor checks whether its memory budget is exceeded (`IsOverBudget()`). If so, it sends itself a `Collect` message. The collection handler evicts expired entries from the B-tree cache. Because `Collect` is a control message, it bypasses the inbox bound and is processed promptly.

19.6 Ephemeral vs Persistent Actors

The `KeyValueActorRouters` class creates two separate consistent-hash rings of `KeyValueActor` instances:

Persistent ring. For keys with `KeyValueDurability.Persistent`. These actors use the shared prepared-intent store and transaction-record store. Writes go through Raft replication and are flushed to disk.

Ephemeral ring. For keys with `KeyValueDurability.Ephemeral`. These actors get empty (no-op) prepared-intent and transaction-record stores. Writes are in-memory only: no Raft replication, no disk persistence. Data is lost on restart.

The locator selects the ring based on the durability of the operation. This separation prevents ephemeral operations from contaminating the transaction state of persistent operations.

Each ring has `KeyValueWorkers` actor instances. Both rings use bounded inboxes with the same control message classifier.

19.7 LockActor

The `LockActor` holds the lock state for a partition. Its state is simpler than the `KeyValueActor`:

- `Dictionary<string, LockEntry>` `locks`: lock entries keyed by resource name. Each entry records the owner, expiry, and fencing token.
- `Dictionary<int, LockProposal>` `proposals`: in-flight Raft proposals.
- `HashSet<string>` `keysToEvict`: eviction set for expired locks.

The `Receive` method dispatches to internal methods for `TryLock`, `TryUnlock`, `TryExtendLock`, `Get`, `CompleteProposal`, `ReleaseProposal`, `InvalidateOrApply`, and `EvictPartition`.

Lock proposals use a `LockProposalActor` through a balancing router (round-robin), not a consistent-hash router. Locks submit one Raft proposal per operation (Chapter 15), so there is no need for key-based routing of proposals.

The actor runs periodic collection every 500 operations to evict expired locks.

19.8 SequenceActor

The `SequenceActor` holds block allocation state for sequences:

- Dictionary<string, SequenceBlock> blocks: one block per sequence name. Each block is a reserved window of IDs (a low-water mark and a high-water mark).

When a NextSequenceValue request arrives, the actor checks the block for the named sequence. If the block has remaining IDs, the actor returns the next value from memory. No Raft proposal is needed.

If the block is exhausted, the actor allocates a new block by writing the updated high-water mark to the key-value subsystem (sequences store their durable state as key-value entries). This write goes through the full KV pipeline, including Raft replication.

A useStamp counter tracks LRU access for eviction of resident blocks when memory is constrained.

If a CancellationToken fires during block allocation (the KV write times out or is cancelled), the actor drops the block. It cannot know whether the write committed. On the next request, a fresh block allocation reads the durable high-water mark and starts from the correct position.

19.9 PartitionWriteAggregatorActor

The PartitionWriteAggregatorActor batches key-value write proposals for efficiency (Chapter 15 described the batching protocol in detail). It is a Nixie actor (IActor<PartitionWriteMessage, PartitionWriteAck>) with per-partition state in a dictionary.

The actor never awaits Raft on the mailbox thread. When a batch is ready, it dispatches the Raft replication as a detached operation. The Raft result arrives later as a BatchComplete control message, which the actor processes to resolve the individual proposals.

Before dispatch, each item is re-checked against the range map. If a key's range moved since the item was admitted (a split completed while the item was queued), the item is released with a retryable failure. This prevents stale writes from reaching Raft. A maximum of 1,024 releases per dispatch prevents starvation of the processing loop.

19.10 BackgroundWriterActor

The BackgroundWriterActor is a fire-and-forget actor (IActor<BackgroundWriteRequest>) that flushes committed Raft entries to the persistence backend. It receives QueueStoreKeyValue and QueueStoreLock messages from the replicators.

The actor batches writes: up to 1,024 items or 512 KB per flush. It retries failed writes up to 5 times with backoff. A revision cleanup queue (capped at 10,000 entries) removes old revisions after the MVCC retention window expires.

19.11 The Single-Threaded Invariant

The actor model provides safety through a contract: no shared mutable state across actors, and no concurrent access to an actor's state.

This contract holds as long as the actor does not leak references to its internal state across await points. Consider this violation:

```
// INCORRECT: reference to internal state escapes the actor
var entry = keyValuesStore.Get(key);
```



```
// This await yields the thread. Another message could modify
// keyValuesStore before the continuation runs.
await SomeAsyncOperation();

// entry may now be stale or invalid.
entry.Value = newValue;
```

Between the `await` and the continuation, the actor processes no other messages (Nixie's single-threaded guarantee holds). But if the reference escapes to a callback or a detached task, two threads could touch the state simultaneously.

Kahuna's handlers follow a discipline: read state, compute the result, create a proposal if needed, and return. Long-running operations (disk reads, Raft proposals) are handled by sending a message to the actor when the operation completes, not by holding a reference across the `await`.

19.12 Failure Scenarios

19.12.1 Actor State After Leader Change

Write intents, prefix locks, and range locks live in actor memory. They are not persisted and not replicated through Raft. When a leader change occurs:

1. The old leader's actors hold stale state (write intents from transactions that were in progress).
2. The new leader's actors start with empty state (no write intents, no prefix locks, no range locks).
3. In-flight transactions on the old leader receive `MustRetry` or `Aborted`.
4. The new leader accepts new transactions. The old write intents on the old leader expire naturally.

This is safe because write intents are optimistic: they record what a transaction intends to do. If the leader changes before the transaction commits, the transaction is aborted and retried on the new leader. The committed state (in Raft) is authoritative. The in-memory state (in actors) is transient.

19.12.2 Inbox Overflow

1. A burst of requests fills a `KeyValueActor`'s inbox to its maximum size.
2. New `Send` and `Ask` calls throw `ActorBusyException`.
3. The routing layer catches the exception and returns `MustRetry` to the client.
4. The client retries after a delay.
5. The actor drains its inbox and begins accepting messages again.

Control messages (proposal completions, replication updates) bypass the inbox bound. This ensures that the Raft pipeline is not blocked by client request backpressure.

19.13 Summary

Kahuna uses the Nixie actor system for concurrency. Each actor owns its state and processes messages one at a time. There are no locks on in-memory state within an actor. Data races are eliminated by construction.

Messages are routed to actor instances through consistent-hash routing. The key's hash determines which actor instance processes the message. Multiple instances per partition allow parallelism for non-overlapping keys.

The `KeyValueActor` is the most complex actor. It holds a B-tree cache, write intents, range locks, and in-flight proposals. It dispatches operations to 41 handler classes through a switch on the message type.

Two separate actor rings (persistent and ephemeral) prevent cross-contamination between durable and in-memory-only operations. Bounded inboxes with control message bypass provide backpressure without blocking the Raft pipeline.

The single-threaded invariant is the foundation of the actor model's safety. As long as actors do not leak references to their internal state across async boundaries, the invariant holds and the state is consistent.

The next chapter examines MVCC and snapshot isolation: how Kahuna tracks multiple versions of a key and how transactions read a consistent snapshot.

20 MVCC and Snapshot Isolation

When two transactions read and write the same key at the same time, one of them must wait or abort. In a system that stores only the current value of each key, every read competes with every write. Reads block writes, writes block reads, and throughput drops under concurrency.

Multi-Version Concurrency Control (MVCC) solves this problem. Instead of storing one value per key, the system stores multiple versions. Each write creates a new version. Reads select the version that was current at the time the read started. Reads never block writes, and writes never block reads.

This chapter explains how Kahuna implements MVCC: the version chain, write intents, snapshot reads, the snapshot floor, revision pruning, and the disk fallback for deep history.

20.1 The Version Chain

Each key in Kahuna can have multiple versions. The `KeyValueEntry` stores the current version and an archive of older versions.

20.1.1 `KeyValueEntry`

The `KeyValueEntry` is a sealed class that holds the complete state of a key:

Field	Type	Description
Value	byte[]?	The current value.
Revision	long	The current modification revision (incremented on each write).
LastModified	HLCTimestamp	The HLC timestamp of the most recent write.
Expires	HLCTimestamp	The TTL expiry timestamp. Zero means no expiry.
State	KeyValueState	Set, Deleted, or Undefined.
WriteIntent	KeyValueWriteIntent?	Uncommitted transaction write (null if no active transaction).
Revisions	KeyValueRevisionHistory?	Archive of older versions.
MvccEntries	Dictionary<HLCTimestamp, KeyValueMvccEntry>?	Per-transaction snapshots for read-your-writes.
FloorBoundaryRevision	long	The revision pinned by the snapshot floor (default -1).
FloorBoundaryCoverageEnd	HLCTimestamp	Upper bound where the floor-boundary revision is authoritative.
FlushedRevision	long	Highest revision confirmed on disk. The entry is dirty while Revision > FlushedRevision.
LastUsed	HLCTimestamp	Last access time (for LRU eviction).
LruPrev, LruNext	KeyValueEntry?	Intrusive doubly-linked list pointers for LRU cache management.

The entry acts as the head of a version chain. The current version is in the Value and Revision fields. Older versions are in the Revisions archive.

20.1.2 Revision History

The KeyValueRevisionHistory stores older versions of the key as a compact sorted array of (long Key, KeyValueRevisionEntry Value) pairs in ascending revision order. Each KeyValueRevisionEntry is a readonly record struct with four fields: Value, LastModified, Expires, and State.

The array is bounded. At most RevisionRetention + 1 entries are stored in memory: the configured retention count (default 16) plus an optional floor-boundary entry. Initial capacity is 1. The array grows by doubling. It never shrinks.

Lookups use binary search. The TryGetRevisionAtOrBefore(snapshot) method finds the highest revision whose LastModified is at or before the given snapshot timestamp. This is the core operation for snapshot reads.

The sorted-array structure replaces a dictionary. For the small bound (at most 17 entries), a sorted array is more memory-efficient than a hash table.

20.2 Write Intents

A write intent is an uncommitted transaction write. When a transaction calls `SetKeyValue` within a session, the system does not immediately apply the write to the committed state. Instead, it attaches a `KeyValueWriteIntent` to the entry.

20.2.1 KeyValueWriteIntent

The `KeyValueWriteIntent` is a sealed class with these fields:

Field	Type	Description
<code>TransactionId</code>	<code>HLCTimestamp</code>	The ID of the transaction that created this intent.
<code>Expires</code>	<code>HLCTimestamp</code>	The lease deadline. Zero means session-owned (no timeout).
<code>CommitTimestamp</code>	<code>HLCTimestamp</code>	Set at prepare time. Zero before prepare.
<code>RecordAnchorKey</code>	<code>string?</code>	Set at prepare. Points to the transaction record.

A write intent is live if its lease has not expired. Session-owned intents (with zero expiry) are live as long as the session is active.

20.2.2 Write Intent Lifecycle

1. **Create.** A transaction writes to a key. The system creates a `KeyValueWriteIntent` and attaches it to the entry. The entry's committed state does not change.
2. **Visibility.** The intent is visible only to its own transaction. Other transactions that read the key see the committed state, not the intent. If another transaction encounters a live write intent from a different transaction, it may receive `WaitingForReplication` (the safe-time wait mechanism) or detect a conflict.
3. **Prepare.** At 2PC phase 1, the intent receives a `CommitTimestamp` and a `RecordAnchorKey`. The system creates a durable prepared intent in the `PreparedIntentStore`.
4. **Commit.** At 2PC phase 2, the intent's value becomes the new committed state. The revision increments. The old committed value moves to the revision archive. The intent is cleared.
5. **Rollback.** If the transaction aborts, the intent is removed. The committed state is unchanged.

20.2.3 Write Intent Conflicts

When a `TryGet` handler encounters a write intent from another transaction:

- If the intent has expired, the handler clears it and reads the committed state.
- If the intent is live and the reader has a snapshot timestamp, the handler checks whether the intent's `CommitTimestamp` falls at or before the snapshot. If so, the read returns `WaitingForReplication` (the system must wait for the intent to resolve before it can serve a consistent read).

- If the intent is outside the snapshot window, the handler reads the committed state.

When a TrySet handler encounters a live write intent from another transaction, the write is rejected. Only one transaction can hold a write intent on a key at a time.

20.3 MVCC Entries: Read-Your-Writes

When a transaction reads a key for the first time, the system snapshots the committed state into an `MvccEntries` dictionary, keyed by transaction ID. The snapshot is a `KeyValueMvccEntry` with the value, revision, timestamps, and state at the time of the read.

On subsequent reads within the same transaction, the system returns the snapshot. This provides read-your-writes consistency: a transaction always sees its own writes and the state it read at the start.

If the committed revision advances past the snapshot (another transaction committed between the reads), the handler returns `Aborted`. The transaction's view is stale and must be retried.

This mechanism is the foundation of snapshot isolation. Each transaction reads from a consistent snapshot. Writes from other transactions that commit after the snapshot are invisible.

20.4 Snapshot Reads

A snapshot read retrieves the value of a key as of a specific HLC timestamp. The client specifies the timestamp through the `snapshotMs` parameter on `GetKeyValue`.

20.4.1 Resolution Logic

The `TryGetHandler` resolves a snapshot read in two steps:

1. **Check the current version.** If the entry's `LastModified` is at or before the snapshot timestamp, the current version is the answer.
2. **Search the archive.** If the current version is newer than the snapshot, the handler calls `entry.TryGetRevisionAtOrBefore(readTimestamp)`. This binary-searches the in-memory revision archive for the highest revision whose `LastModified` is at or before the snapshot.
3. **Disk fallback.** If the in-memory archive does not contain the version (the archive was pruned, or the snapshot is older than the oldest retained revision), the handler calls `PersistenceBackend.GetKeyValueRevisionAtOrBefore(key, maxRevision, readTimestamp)`. This queries the persistence backend (RocksDB or SQLite) for the version.

If no version exists at or before the snapshot timestamp, the read returns `DoesNotExist`.

20.4.2 Disk Fallback for Deep History

The in-memory archive holds at most `RevisionRetention` versions (default 16). For keys with high write frequency, older versions are pruned from memory. These versions may still exist on disk if persistent revision retention is enabled.

The `ResumeRead` mechanism handles asynchronous disk fallback for cache misses on the current value (non-snapshot reads):

1. The actor checks the B-tree cache. The key is not present.
2. The actor dispatches an off-actor disk read (to avoid blocking the actor's message loop).

3. When the disk read completes, the system sends a `ResumeRead` control message back to the actor.
4. The actor reconciles the disk result with its current state (another write may have arrived while the disk read was in progress).

`ResumeRead` is a control message, so it bypasses the inbox bound and is processed promptly.

20.5 The Snapshot Floor

The snapshot floor is the minimum HLC timestamp below which revisions may be pruned. It protects long-running snapshot reads and transactions from losing the versions they depend on.

20.5.1 SnapshotFloorStore

The `SnapshotFloorStore` is a replicated hold registry on the meta partition (partition 0). It tracks active snapshot holds and computes the effective floor.

A snapshot hold is a record with four fields:

Field	Type	Description
<code>HoldId</code>	identifier	Unique ID for this hold.
<code>HolderId</code>	identifier	The entity that acquired the hold (transaction, client session).
<code>Timestamp</code>	<code>HLCimestamp</code>	The snapshot point to protect.
<code>LeaseExpiry</code>	<code>HLCimestamp</code>	Deadline for the hold. The hold expires if not renewed.

20.5.2 Hold Operations

- `AcquireAsync(holderId, timestamp, leaseMs)`: creates a hold. Idempotent by `(holderId, timestamp)`: a duplicate acquire returns the existing hold. The operation replicates through Raft on the meta partition.
- `RenewAsync(holdId, leaseMs)`: extends the lease deadline. Returns `DoesNotExist` if the hold expired.
- `ReleaseAsync(holdId)`: removes the hold. The floor rises when the lowest hold is released.
- `PurgeExpiredHoldsAsync()`: periodic cleanup of holds whose lease has lapsed.

20.5.3 Effective Floor

`GetEffectiveFloor(currentTime)` computes the minimum timestamp among all live holds. This is the effective snapshot floor. A revision whose `LastModified` is below the floor is a candidate for pruning.

The method has an $O(1)$ fast path when the cache is valid and an $O(N)$ slow path when a hold may have expired. Prune operations and acquire operations are ordered through a `pruneCommitLock` to prevent races.

20.5.4 Metrics

Two metrics track the floor state:

- `kahuna.snapshot_floor.live_holds`: the number of active holds.

- `kahuna.snapshot_floor.effective_floor_ms`: the effective floor timestamp in milliseconds.

20.6 Revision Pruning

The revision archive grows with each write. Without pruning, memory consumption grows without bound. Kahuna prunes old revisions based on three criteria.

20.6.1 In-Memory Pruning

The `RemoveExpiredRevisions` method in `BaseHandler` runs after each write. It computes a cutoff revision:

```
cutoff = currentRevision - RevisionRetention + 1
```

Revisions below the cutoff are candidates for removal. The method removes all revisions below the cutoff with one exception: the floor-boundary revision.

20.6.2 The Floor-Boundary Revision

If snapshot holds exist, the pruning logic pins the highest below-cutoff revision whose `LastModified` is at or before the effective snapshot floor. This revision is the floor-boundary revision. It stays in the archive even though it is below the retention cutoff.

The floor-boundary revision is the safety net for snapshot reads. A snapshot read at a timestamp near the floor can use this revision as its answer. Without it, the read would need to fall back to disk.

When revisions between the floor-boundary revision and the retention window are trimmed, the entry records `FloorBoundaryCoverageEnd`: the smallest `LastModified` among the trimmed revisions. This value marks the point where the floor-boundary revision stops being authoritative.

When `TryGetRevisionAtOrBefore` finds the floor-boundary revision as the best match, it checks whether the snapshot timestamp is below `FloorBoundaryCoverageEnd`. If the snapshot is at or above that boundary, a gap exists: there may be a trimmed revision on disk that is a better match. The method returns a miss, and the handler falls back to disk.

20.6.3 Persistent Revision Retention

On disk, revisions are subject to separate retention controls:

Flag	Default	Description
<code>--persistent-revision-retention-count</code>	0 (keep forever)	Maximum persisted revisions per key.
<code>--persistent-revision-retention-age</code>	0 (disabled)	Maximum age in seconds for persisted revisions.
<code>--persistent-revision-cleanup-interval</code>	300s	Minimum interval between cleanup sweeps.
<code>--persistent-revision-cleanup-batch-size</code>	1000	Maximum revision records deleted per sweep.

When both in-memory and persistent retention are configured, the system retains the most recent `RevisionRetention` versions in memory and up to `persistent-revision-retention-count` versions on disk. Older versions are deleted during cleanup sweeps.

20.7 Safe Timestamp

The safe timestamp is the minimum prepared `CommitTimestamp` across all live write intents in a shard. Any snapshot timestamp strictly below this value avoids cutting across a currently prepared transaction.

The `GetSafeTimestampHandler` scans both the B-tree store and the prefix locks to find this minimum. The safe timestamp is used by the snapshot coordinator to choose a consistent point for snapshot reads.

If no write intents are active, there is no lower bound, and any timestamp is safe.

20.8 Prepared Intents and MVCC

The `PreparedIntentStore` is a partition-scoped authority for durable prepared intents. At most one live intent can exist per key. The store uses a `ConcurrentDictionary` for concurrent reads during visibility lookups, with mutations serialized by a lock.

When a `TryGetHandler` encounters a durable prepared intent from another transaction, it calls `DurableReadVisibility.Resolve` to determine how to proceed:

- **UseIntentValue:** the intent's transaction committed. Use the intent's value as the read result.
- **UseExisting:** the intent's transaction aborted or the intent is not relevant to this read. Use the committed state.
- **Retry:** the intent's outcome is not yet known. Return `MustRetry` and let the client retry after the intent resolves.

This mechanism prevents a reader from seeing an inconsistent state when a transaction is in the middle of the two-phase commit protocol.

20.9 Failure Scenarios

20.9.1 Snapshot Hold Expiry During a Long Transaction

1. A transaction acquires a snapshot hold at timestamp T1 with a 30-second lease.
2. The transaction takes 45 seconds to complete.
3. After 30 seconds, the hold expires. The effective floor rises above T1.
4. Revision pruning removes versions at T1.
5. The transaction tries to read a key at its snapshot timestamp. The version was pruned.
6. The handler falls back to disk. If the persistent revision is also pruned, the read returns `DoesNotExist` or stale data.

Prevention: renew the snapshot hold before it expires. Set the lease duration longer than the expected transaction duration.

20.9.2 Stale Read After Floor-Boundary Gap

1. A key has revisions at timestamps T1, T5, T10, T15, T20 (revision numbers 1 through 5).
2. Retention is 2. The cutoff is revision 4 (keep revisions 4 and 5).
3. A snapshot hold exists at T3. Revision 1 (T1) is pinned as the floor-boundary revision.
4. Revisions 2 (T5) and 3 (T10) are trimmed. `FloorBoundaryCoverageEnd` is set to T5.
5. A snapshot read at T7 (between T5 and T10) hits the archive. The best match is revision 1 (T1), which is the floor-boundary revision.
6. The snapshot (T7) is above `FloorBoundaryCoverageEnd` (T5). The handler returns a miss.
7. The handler falls back to disk and finds revision 2 (T5), which is the correct answer.

The `FloorBoundaryCoverageEnd` check prevents the in-memory archive from returning an incorrect answer when revisions were trimmed.

20.9.3 Write Intent From a Crashed Transaction

1. A transaction writes to key K and creates a write intent.
2. The client crashes before sending the commit or rollback.
3. The write intent remains on the entry with a lease deadline.
4. After the lease expires, the next read or write to key K detects the expired intent and clears it.
5. The committed state is restored. No data corruption occurs.

Write intents have lease deadlines. Expired intents are cleaned up on the next access. The committed state is always authoritative.

20.10 Summary

Kahuna uses MVCC to allow reads without blocking writes. Each key stores multiple versions: a current version and an archive of up to 16 older versions in memory.

Write intents represent uncommitted transaction writes. They are visible only to their own transaction. Other transactions see the committed state. The intent lifecycle follows the 2PC protocol: create, prepare (with commit timestamp), commit (becomes the new version), or rollback (removed).

Snapshot reads resolve the version that was current at a given HLC timestamp. The resolution searches the in-memory archive first and falls back to disk for deep history.

The snapshot floor protects long-running reads from revision pruning. Snapshot holds lease a timestamp. While a hold is active, revisions at or above that timestamp are retained. The floor-boundary revision is pinned below the retention cutoff as a safety net.

Revision pruning bounds memory consumption. In-memory pruning runs after each write and keeps the most recent `RevisionRetention` versions. Persistent retention is controlled by separate flags for count and age.

The next chapter examines transaction internals: how sessions, 2PC coordination, write-intent resolution, and conflict detection work at the implementation level.

21 Transaction Internals

Chapter 6 introduced transactions from the application’s perspective: script transactions for server-side logic and interactive sessions for client-driven workflows. Chapter 19 explained how MVCC stores multiple versions and how write intents represent uncommitted writes.

This chapter moves inside the transaction subsystem. It describes the two-phase commit (2PC) protocol as Kahuna implements it, durable intents, deferred settlement, conflict detection, admission control, session reaping, and transaction recovery.

21.1 Transaction Sessions

Every interactive transaction starts with a session. The `TransactionCoordinator` manages sessions in a `ConcurrentDictionary<HLCTimestamp, TransactionContext>`, keyed by transaction ID.

21.1.1 Starting a Session

`StartTransaction` performs these steps:

1. Validate the options (locking mode, timeout, priority).
2. Clamp the timeout to the server’s `MaxTransactionTimeout`.
3. Acquire an admission slot through the `TransactionPriorityOrderer`. If no slot is available and the admission wait expires, return `AdmissionRefused`.
4. Mint an HLC-based transaction ID.
5. Create a `TransactionContext` and add it to the sessions dictionary.

The transaction ID is an HLC timestamp. It provides a globally unique identifier that also carries a causal ordering.

21.1.2 Operation Tracking

Each operation within a session (a read, write, lock acquisition) is tracked through `BeginOperation` / `CompleteOperation` / `CancelOperation`. The coordinator folds each operation’s effects into the session’s working set: which keys were read, which were modified, and which locks were acquired.

This working set is the input to the commit protocol. It tells the coordinator which partitions are involved and what needs to be validated.

21.2 Admission Control

The `TransactionPriorityOrderer` is a per-node admission gate that limits the number of concurrent transactions. Two independent orderer instances exist: one for interactive sessions and one for script transactions.

When the transaction count is below the ceiling, admission is immediate. When the count is at the ceiling, the caller parks on an awaitable. Parked callers are served in priority order (four priority levels).

`AdmitAsync` returns an `AdmissionLease`. The lease is `IDisposable` with exactly-once release through `Interlocked.Exchange`. A leaked lease (the caller crashes without disposing) permanently shrinks capacity. The session reaper eventually cleans up the session and releases the lease.

The admission wait is clamped to `MaxAdmissionWaitMs`. If the wait exceeds this value, the caller receives `AdmissionRefused`.

21.3 The Two-Phase Commit Protocol

When the client calls `Commit`, the coordinator enters the 2PC protocol. The protocol has four phases: prepare, validate, decide, and resolve.

21.3.1 Phase Overview

Prepare	Install write intents and acquire locks on each partition. ↓
Validate	Check the read set for conflicts (optimistic mode). ↓
Decide	CAS transition on the canonical transaction record: Undecided → Committed or Undecided → Aborted. ↓
Resolve	Materialize committed values or discard aborted intents.

21.3.2 Commit Entry Point

`CommitTransaction` begins with contention control. The coordinator enters a finalize slot on the `TransactionContext` through `context.EnterFinalize()`. This slot serializes concurrent commit, rollback, and reaper operations against the same session. If another thread already holds the slot, the caller receives a `Mirror` response (wait for the owner's outcome) or `Rejected (MustRetry)`.

After acquiring the slot, the coordinator calls `FreezeForFinalize`, which drains in-flight operations. No new operations are accepted after the freeze. Then it calls `TwoPhaseCommit`.

21.3.3 Four Transaction Paths

The `TwoPhaseCommit` method handles four cases based on what the transaction modified:

Read-only transaction. No modified keys. The coordinator validates the read set only (conflict probe plus revision check). If any read key was modified by another transaction since the read, the result is `Aborted`. No Raft proposals are needed.

All-ephemeral transaction. All modified keys are ephemeral (in-memory only). The coordinator validates the read set, installs write intents via `PrepareMutations`, checks for commit conflicts, and applies the writes via `CommitMutations`. If any step fails, the writes are rolled back. No durable transaction record is created.

All-persistent (durable) transaction. All modified keys are persistent. The coordinator delegates to the `DurableTransactionFinalizer`, which drives the full durable 2PC protocol with a canonical transaction record and prepared intents.

Mixed transaction. Some modified keys are persistent and some are ephemeral. The coordinator prepares ephemeral keys first (so an ephemeral failure aborts before the durable decision). Then it runs the durable protocol for persistent keys. The durable decision drives the ephemeral commit or rollback.

21.4 Durable Transaction Finalization

The `DurableTransactionFinalizer` drives one transaction through the durable 2PC model. It works with frozen, immutable inputs: the transaction ID, epoch, coordinator key, commit

timestamp, decision deadline, a manifest of participant partitions, and per-partition prepared intent sets.

21.4.1 Step 1: Staged-Base Validation

Before anything durable, the finalizer checks each write intent's `BaseRevision` and `BaseState` against the current committed state on the target partition. If the base moved (another transaction committed to the same key), the transaction aborts with a conflict. This check catches conflicts early, before any Raft proposals.

21.4.2 Step 2: Initialize the Canonical Record

The finalizer creates a canonical transaction record on the anchor partition in the `Undecided` state. This is a Raft proposal. The record contains the transaction ID, epoch, and manifest hash.

The anchor partition is determined by the transaction's anchor key. The canonical record is the single source of truth for the transaction's outcome.

21.4.3 Step 3: Prepare Intents

The finalizer installs prepared intents on each participant partition through Raft. Each prepared intent records the transaction ID, the key, the proposed value, the commit timestamp, and a pointer to the canonical record (the `RecordAnchorKey`).

If a prepare is blocked by a foreign intent from a different transaction that already has a terminal decision (committed or aborted but not yet settled), the finalizer helps settle that intent before retrying. This is called “prepare-conflict helping”: the `resolveDecidedBlockers` mechanism settles the blocking intent and retries the prepare. The finalizer retries up to `MaxPrepareRetries` (8 attempts).

The finalizer never touches undecided foreign records. An undecided record belongs to a live transaction. Interfering with it would violate that transaction's isolation.

21.4.4 Step 4: Validate Read Set

For optimistic transactions with `ReadValidation.TrackAndValidate`, the coordinator re-reads the current committed revision for each key in the read set. If any revision advanced since the snapshot, the transaction aborts.

21.4.5 Step 5: CAS Decide

The finalizer performs a compare-and-swap transition on the canonical record:

- **Commit:** `Undecided` → `Committed`. The commit is allowed only if `AttemptHlc <= DecisionDeadline`. A late commit attempt is rejected. If the deadline passed, recovery presumes abort.
- **Abort:** `Undecided` → `Aborted` or `absent` → `Aborted`. Abort can create a tombstone from absence (presumed abort).

The `TransactionRecordStateMachine` enforces these transitions deterministically:

Current State	Command	Result
absent	Initialize	Undecided
Undecided	Commit (within deadline)	Committed
absent or Undecided	Abort	Aborted
Committed	Commit (same)	Idempotent (no change)
Aborted	Abort (same)	Idempotent (no change)
Committed	Abort	Rejected (terminal is final)
Aborted	Commit	Rejected (terminal is final)
absent	Commit	Rejected (needs Undecided proof)

Once the record reaches **Committed** or **Aborted**, it is terminal. No further transitions are possible.

21.4.6 Step 6: Resolve Intents

After the decision is durable, the finalizer resolves the prepared intents:

- **Committed:** materialize the intent's value as the new committed version. Bump the revision. Move the old committed value to the revision archive.
- **Aborted:** discard the intent. The committed state is unchanged.

Resolution can be synchronous or deferred (asynchronous), based on the `DurableDeferredSettlement` configuration.

21.4.7 One-Phase Fast Path

For single-partition transactions, the finalizer bundles the record initialization, anchor prepare, and commit decision into one atomic Raft proposal. This reduces three Raft round trips to one. If the prepare fails (a `bundledPrepareProbe` gate rejects it), the bundle is rejected and the transaction falls back to the multi-step path.

21.4.8 Anchor Bundling

For multi-partition transactions, the finalizer bundles the record initialization and the anchor partition's prepare into one Raft proposal. This saves one pre-decision Raft round trip compared to separate proposals.

21.5 Deferred Settlement

Deferred settlement is an optimization that moves intent resolution off the commit critical path. The decision is durable (the canonical record is committed in Raft). The resolution (materializing values, settling intents) happens asynchronously through a `ResolutionScheduler`.

The benefit: the commit returns to the client as soon as the decision is durable. The client does not wait for resolution.

If the background resolution is lost (the node crashes before resolution completes), the recovery sweep finds the committed record and resolves the intents. Correctness does not depend on the background run completing.

Deferred settlement is enabled by default (`DurableDeferredSettlement = true`). In practice, the system runs resolution synchronously for cross-node transactions to ensure read-your-writes consistency: a read immediately after a commit must see the committed value, which requires the intent to be resolved before the commit returns.

21.6 Conflict Detection

Kahuna uses two conflict detection mechanisms, one for each locking mode.

21.6.1 Optimistic: Read-Set Validation

In optimistic mode, reads do not acquire locks. Conflicts are detected at commit time.

`ValidateReadSet` checks two conditions:

1. **Read observation conflict.** If the same key was observed at two different base revisions within one transaction (the key changed between two reads in the same session), the transaction aborts.
2. **Concurrent modification.** For each read key that is not in the modified set, the coordinator re-reads the current committed revision. If the revision changed since the snapshot, another transaction committed to that key. This transaction aborts.

`CheckCommitConflicts` probes for concurrent write intents. If a foreign write intent appeared after this transaction's read, a write-skew may exist. The transaction aborts.

21.6.2 Pessimistic: Lock Acquisition

In pessimistic mode, the transaction acquires exclusive locks on every key it reads or writes. Other transactions that touch the same keys wait (up to the transaction timeout) or abort.

Lock acquisition happens before the operation, not at commit time. Conflicts are detected immediately: if the lock is held by another transaction, the operation returns `AlreadyLocked`.

Pessimistic mode eliminates read-set validation overhead. There are no concurrent modifications to detect because locks prevent them. The trade-off is lower concurrency: transactions wait for each other instead of running in parallel and detecting conflicts at commit.

21.7 Script Transactions

The `ScriptTransactionExecutor` handles script-driven transactions. It composes the `TransactionCoordinator` for all 2PC and lock-release operations.

`TryExecuteTx` parses the script and decides whether a transaction is needed. A single stand-alone command (one SET or GET with no BEGIN/COMMIT) runs directly without the admission gate. Multi-statement scripts or explicit BEGIN blocks open a transaction through the coordinator.

Script transactions use a separate admission orderer from interactive sessions. This prevents a burst of script transactions from starving interactive sessions (or the reverse).

An extra locking delay of 10 ms (`ExtraLockingDelay`) is added to pessimistic lock TTLs in script transactions. This accounts for the parsing and execution overhead.

21.8 Transaction Record Store

The `TransactionRecordStore` is the partition-scoped authority for canonical transaction records. Records are keyed by (`TransactionId`, `Epoch`).

Mutations go through `Apply(TransactionRecordCommand)`, which is serialized by a lock. The `TransactionRecordStateMachine` evaluates the command against the current state and produces the transition (or a rejection).

The store is replicated through Raft. `Restore` and `Replicate` methods apply write-ahead log entries. The store persists per-partition snapshots to disk (temp file plus rename). Snapshots are loaded on startup. A version counter avoids unnecessary snapshot rewrites when no records changed.

21.9 Completion Receipts

The `CompletionReceiptStore` is a node-local store of completion receipts. A receipt records that a participant partition durably committed the value for a specific transaction and key.

Receipts serve two purposes:

1. **Leader-change resilience.** After a leader change, the new leader's actors have empty in-memory state. If a re-delivered commit arrives for a transaction that already committed, the receipt lets the server answer `Committed` instead of ambiguous `MustRetry`.
2. **Range split/merge gating.** Before a range split or merge cutover, `SettleSuppliedIntentsAsync` must resolve all durable intents in the moving range. Completion receipts are transferred to the new partition so it can serve re-commits correctly after cutover.

Receipts are removed by coordinator acknowledgement (`Forget`) or by an age-based backstop (`CollectExpired`). They are not evicted by size.

21.10 Session Reaping

Abandoned sessions (the client crashed, disconnected, or forgot to commit/rollback) are cleaned up by the session reaper.

`ReapAbandonedSessions` runs periodically on each collection sweep. For each session, it computes a deadline:

```
deadline = transactionId + Timeout + ReapGraceMs (15 seconds)
```

If the session has pending operations, the deadline is extended by `MaxParticipantEffectTtlMs` (15 seconds) to allow in-flight operations to complete.

When a session passes its deadline:

1. The reaper claims the finalize slot through `TryEnterReap()`.
2. `ReapSession` sets the action to `Abort` and releases the working set.
3. Three outcomes are possible:
 - All mandatory releases acknowledged: the session is removed. Terminal outcome `RolledBack` is retained for duplicate handling.
 - An operation is still unresolved past the extended deadline: `Errored`. The session is removed.
 - A mandatory release failed (transient): `MustRetry`. The session stays in the `Reaping` state. The next sweep retries.

The reaper also prunes retained terminal outcomes by age on each sweep.

21.11 Transaction Recovery

The `DurableTransactionRecovery` handles orphaned prepared intents. A prepared intent can become orphaned when the coordinator crashes after preparing but before deciding, or when a leader change occurs between prepare and resolve.

21.11.1 The Recovery Sweep

`SweepAsync` runs periodically on each partition leader. It groups unresolved prepared intents by `(TransactionId, Epoch, RecordAnchorKey)`. For each group, it looks up the canonical transaction record on the anchor partition.

The sweep produces one of four outcomes:

Record State	Action
Committed	Materialize values and settle intents.
Aborted	Discard intents and settle.
Undecided, past deadline	Drive a presumed-abort: CAS Undecided → Aborted at the anchor. A concurrent commit can still win the CAS. Return whatever the record became.
Undecided, within deadline	Skip. The transaction is still live.
No record at all	Orphan prepare. Drive a presumed-abort: CAS absent → Aborted.

21.11.2 Presumed Abort

Kahuna uses the presumed-abort rule: any transaction without a committed decision record is aborted. If the coordinator crashes before deciding, the recovery sweep finds no record (or an Undecided record past its deadline) and aborts the transaction.

This is safe because the commit decision requires a durable Committed record. If no such record exists, the transaction did not commit. The prepared intents are discarded.

21.11.3 Prepare-Conflict Helping

When a prepare is blocked by a foreign intent whose record is already terminal (committed or aborted but not yet settled), the recovery mechanism settles the blocking intent immediately. This is `TryResolveDecidedBlockersAsync`. It never touches undecided records (those belong to live transactions).

This mechanism prevents a resolved-but-unsettled intent from blocking new transactions indefinitely.

21.12 Outcome Contract

Every transaction terminates with one of these outcomes:

Outcome	Meaning	Action
Committed	All writes applied.	Success.
Aborted	Conflict or validation failure. All writes rolled back.	Retry with a new session.
RolledBack	Explicit rollback completed.	Application logic decides next step.
MustRetry	Transient failure (drain time-out, unresolved durable decision). Session may still be live.	Retry.
Errored	Permanent unknown outcome (session expired, never existed).	Log and alert.

The coordinator retains terminal outcomes in a window after session removal. If a duplicate commit or rollback arrives for a session that already terminated, the coordinator returns the retained outcome instead of MustRetry.

21.13 Failure Scenarios

21.13.1 Coordinator Crash During 2PC

1. The coordinator prepares intents on partitions P1 and P2.
2. The coordinator crashes before the CAS decide step.
3. The prepared intents are orphaned on P1 and P2.
4. Each partition's recovery sweep finds the intents. It looks up the canonical record.
5. No committed record exists (the coordinator never decided). The sweep drives a presumed abort.
6. The intents are discarded. No data is corrupted.

21.13.2 Participant Crash After Prepare

1. The coordinator prepares intents on P1 and P2. P1 acknowledges. P2 crashes.
2. The coordinator decides Committed (it already has enough prepares).
3. P2 restarts. The recovery sweep finds the prepared intent and looks up the canonical record.
4. The record is Committed. The sweep materializes the value on P2.

The decision is durable. Resolution eventually reaches all participants, even after crashes.

21.13.3 Leader Change Between Prepare and Commit

1. Intents are prepared on P1 (old leader).
2. A leader change occurs. P1 is no longer the leader.
3. The coordinator's commit attempt reaches the new leader, which does not have the prepared intents in memory.
4. The coordinator receives MustRetry.
5. If wrapped in RetryableTransaction, the entire transaction retries with a fresh session on the new leader.

21.13.4 Abandoned Session

1. The client opens a session, reads several keys, and disconnects without committing.
2. The session's deadline passes: `transactionId + Timeout + ReapGraceMs`.
3. The reaper claims the finalize slot and aborts the session.
4. Write intents are rolled back. Locks are released.
5. The terminal outcome `RolledBack` is retained briefly for duplicate handling.

21.13.5 Concurrent Transaction Conflict

1. Transaction A reads key K at revision 5.
2. Transaction B reads key K at revision 5, writes a new value, and commits. Key K is now at revision 6.
3. Transaction A tries to commit. The read-set validation re-reads key K and finds revision 6.
4. Revision 6 does not match the snapshot revision 5. Transaction A aborts.
5. `RetryableTransaction` retries Transaction A with a fresh session. The retry reads revision 6.

21.14 Summary

Kahuna's transaction subsystem orchestrates 2PC through the `TransactionCoordinator` (session management) and the `DurableTransactionFinalizer` (durable 2PC protocol).

The protocol has four phases: prepare (install intents on participant partitions), validate (check the read set for conflicts), decide (CAS transition on the canonical record), and resolve (materialize or discard intents). Single-partition transactions use a one-phase fast path that bundles all three durable steps into one Raft proposal.

Conflict detection uses read-set validation for optimistic transactions and lock acquisition for pessimistic transactions. The `TransactionRecordStateMachine` enforces deterministic CAS transitions on the canonical record.

Recovery uses the presumed-abort rule: any transaction without a committed decision record is aborted. The recovery sweep resolves orphaned intents and settles decided-but-unresolved blocking intents.

Session reaping cleans up abandoned sessions after a grace period. Admission control limits concurrent transactions and serves waiters in priority order. Completion receipts provide leader-change resilience and gate range split/merge cutover.

The next chapter examines lock and sequencer internals: how the lock subsystem tracks ownership and fencing tokens, and how the sequencer allocates ID blocks.

22 Lock and Sequencer Internals

Chapter 4 introduced distributed locks from the application perspective: acquire, extend, release, and the fencing token that protects against stale holders. Chapter 5 introduced the sequencer: a monotonic counter that reserves values in blocks for throughput. Both subsystems appeared again in Chapter 18, which described the `LockActor` and `SequenceActor` at the actor-model level.

This chapter moves inside both subsystems. It explains how fencing tokens stay monotonic across leader changes, how lock mutations flow through Raft, how the unflushed-writes overlay prevents read gaps, how the sequencer reserves blocks through compare-and-swap on a key-value entry, and how idempotent reserves work.

22.1 Lock Entry

Each lock resource is represented by a `LockEntry`, a sealed class with seven fields:

Field	Type	Description
Owner	byte[]?	The identity of the current holder. Null when unlocked.
Expires	HLCTimestamp	The lease deadline. The lock is free after this time.
FencingToken	long	Monotonically increasing token. Incremented on each acquisition.
LastUsed	HLCTimestamp	Last access time (for cache eviction).
LastModified	HLCTimestamp	HLC timestamp of the most recent mutation.
State	LockState	Locked or Unlocked (default is Locked).
ReplicationIntentLockReplicationIntent	In-flight Raft proposal metadata (proposal ID and expiry).	

The `FencingToken` is the field that matters most. Every time a client acquires a lock, the actor sets the new token to `entry.FencingToken + 1`. The token only goes up. A client that holds token 5 can verify that no other client received token 6 before the first client's write reaches the downstream resource.

22.2 The Lock Pipeline

Lock operations follow the same Manager, Locator, Actor pattern as key-value operations. The pipeline has four stages.

22.2.1 LockManager

The `LockManager` is the entry point. It creates two consistent-hash actor rings:

- **Persistent ring.** For locks with `LockDurability.Persistent`. These locks survive restarts.
- **Ephemeral ring.** For locks with `LockDurability.Ephemeral`. These locks exist only in memory.

The manager also creates a `BalancingActor` router for `LockProposalActor` instances. This router distributes Raft proposals across proposal actors in round-robin order. Lock proposals are independent of each other, so there is no need for key-based routing.

The manager holds a `DataPartitionRouter` for partition resolution, a `LockLocator` for leader routing, a `LockRestorer` for log replay, a `LockReplicator` for follower applies, and an `UnflushedLockWritesIndex` for read consistency.

22.2.2 LockLocator

The `LockLocator` resolves which node owns a lock resource. It calls `dataPartitionRouter.Locate(resource)` to find the partition, then checks `AmILeaderIfHosted` to determine whether this node leads that partition.

If this node is the leader, the locator dispatches the request to the local `LockActor` ring. If another node is the leader, the locator forwards the request through inter-node communication. If no leader is available (an election is in progress), the locator returns `MustRetry`.

22.2.3 LockActor

The `LockActor` holds the in-memory lock table: a `Dictionary<string, LockEntry>` keyed by resource name. Chapter 18 described the actor's structure. This section focuses on the three lock operations.

TryLock. The actor processes an acquisition in these steps:

1. Look up the resource in the lock dictionary. If the entry is not cached and the durability is persistent, load from the persistence backend.
2. If the entry has an active `ReplicationIntent`, reject the request. A Raft proposal is already in flight for this resource.
3. If the entry is locked, check whether the lease expired. If the lease is still active and the owner is different, reject the request.
4. Compute the new fencing token: `entry.FencingToken + 1`.
5. For persistent locks: create a Raft proposal (described below). The response is deferred until the proposal commits.
6. For ephemeral locks: apply the new state directly. Return the fencing token to the caller.

TryExtendLock. The actor validates that the caller is the current owner and holds the same fencing token. If both checks pass, the actor updates the `Expires` field with the new deadline. `Extend` does not increment the fencing token.

TryUnlock. The actor validates the owner. If the owner matches, the actor sets `State` to `Unlocked` and clears the `Owner` to null.

22.2.4 LockProposalActor

The `LockProposalActor` is a fire-and-forget actor (`IActor<LockProposalRequest>`). It handles Raft replication for a single lock operation.

Unlike key-value writes, lock mutations are not batched. Each lock operation creates its own Raft log entry. The proposal actor serializes the lock mutation into a `LockMessage` protobuf, then calls `raft.ReplicateLogs(partitionId, ReplicationTypes.Locks, ...)` to propose it.

On success, the proposal actor sends a `CompleteProposal` message back to the `LockActor`. On failure (Raft rejected the proposal), it sends a `ReleaseProposal` message. Both are control messages that bypass the actor's inbox bound.

22.2.5 The Proposal Lifecycle

When the `LockActor` creates a proposal for a persistent lock, it follows this sequence:

1. Set a `LockReplicationIntent` on the entry. The intent records the proposal ID and the expiry deadline.
2. Add the proposal to the proposals dictionary (keyed by proposal ID).
3. Send the proposal to the `BalancingActor` router, which dispatches it to a `LockProposalActor`.
4. Mark the response as deferred (`ByPassReply = true`). The client waits on the proposal's `TaskCompletionSource`.

When `CompleteProposal` arrives:

1. Look up the proposal by ID. If not found (timed out or evicted), discard.
2. Validate that the `ReplicationIntent` on the entry matches the proposal. If it does not match, another operation replaced the intent. Discard.
3. Apply the committed values to the entry: `FencingToken`, `Owner`, `Expires`, `LastUsed`, `LastModified`, `State`.
4. Clear the `ReplicationIntent`.
5. Complete the `TaskCompletionSource` with the committed fencing token.

When `ReleaseProposal` arrives, the actor clears the `ReplicationIntent` and completes the `TaskCompletionSource` with an error. The client can retry.

22.3 Fencing Token Monotonicity

The fencing token is the contract that makes distributed locks useful. A lock without a fencing token cannot prevent a slow client from writing stale data after its lease expired and another client acquired the lock. The fencing token gives the downstream resource a way to reject writes from outdated holders.

Monotonicity means that each acquisition produces a token strictly greater than the previous one. Kahuna enforces this at the actor level: `entry.FencingToken + 1` is computed from the current entry state. Because the actor processes messages sequentially, two concurrent acquisitions cannot read the same token value.

22.3.1 The InvalidateOrApply Advance Guard

The replicator and restorer send `InvalidateOrApply` messages to the `LockActor` to update its cache with committed mutations. The actor applies these updates through an advance guard:

Apply only if:

```
incoming.FencingToken > entry.FencingToken
OR (incoming.FencingToken == entry.FencingToken
    AND incoming.LastModified > entry.LastModified)
```

This guard prevents two problems:

1. **Stale deliveries.** A delayed replication message from a previous term must not overwrite a newer acquisition.

2. **Duplicate deliveries.** The same Raft entry may be replayed during a leader change. The guard makes the replay idempotent.

The second condition (same token, newer `LastModified`) handles extend and unlock operations. These operations reuse the existing fencing token but carry a later HLC timestamp.

22.4 Lock Replication

The `LockReplicator` processes committed Raft log entries on followers. When a lock mutation commits in the Raft log, the replicator deserializes the `LockMessage` and performs four steps:

1. **Register pending.** Call `durabilityTracker.RegisterPending(partitionId, logId, DurabilityChannel.Flush)`. This tells the durability tracker that this log entry is committed but not yet flushed to disk.
2. **Record unflushed.** Call `unflushedLockWrites.Record(...)`. This updates the in-memory overlay so that a read can find the committed state before the background writer flushes it.
3. **Queue persistence.** Send a `QueueStoreLock` message to the `BackgroundWriterActor`. The background writer will flush the mutation to disk asynchronously.
4. **Send InvalidateOrApply.** Send the committed state to the persistent `LockActor` ring. The actor applies the update through the advance guard.

The replicator handles `TryLock`, `TryUnlock`, and `TryExtendLock` with the same four-step pattern. For `TryUnlock`, the `InvalidateOrApply` message passes null as the owner. This clears the holder in the actor's cache, matching the behavior of `CompleteProposal` on the proposing leader.

22.5 Lock Restoration

The `LockRestorer` replays committed Raft log entries during a leader change. When a partition's leadership moves to a new node, the Raft layer replays all committed entries that the new leader received from the old leader's log.

The restorer follows the same four-step pattern as the replicator:

1. Register pending with the durability tracker.
2. Record in the unflushed writes overlay.
3. Queue for background persistence.
4. Send `InvalidateOrApply` to the actor ring.

The restorer ensures that the new leader's actors hold committed state that is at least as recent as the replicated log. Without restoration, a re-promoted leader could mint fencing tokens from an outdated entry.

22.6 The Unflushed Writes Overlay

The `UnflushedLockWritesIndex` solves a specific race condition. Consider this sequence:

1. A lock is acquired. The Raft entry commits. The replicator records the mutation.
2. A leader change occurs before the background writer flushes the mutation to disk.
3. The new leader's `LockActor` has an empty cache for this resource.
4. The actor loads the entry from the persistence backend. The backend returns the old state (flushed before the mutation).
5. The actor treats the lock as free and grants it to a second client.

The overlay prevents step 5. Before reading from the persistence backend, the `LockActor` checks the overlay. If the overlay holds a newer mutation for this resource, the actor uses the overlay value instead of the stale disk value.

The overlay is a `ConcurrentDictionary<string, UnflushedLockWrite>`. Each entry records the owner, fencing token, expiry, timestamps, and state. The `Record` method keeps only the newest head per resource, using the same ordering as the advance guard: fencing token first, `LastModified` as the tiebreak.

The `RemoveFlushed` method prunes the overlay after a confirmed flush. It removes the entry only if the flushed mutation is at least as new as the overlay head. If a newer mutation was queued after the flush started, the overlay entry stays until that newer mutation is also flushed.

22.7 Lock Collection

The `LockActor` runs periodic garbage collection. Every 500 operations (`CollectThreshold`), the actor checks its memory budget. If the budget is exceeded, it sends itself a `Collect` control message.

The collection handler evicts entries where `(currentTime - LastUsed)` exceeds the cache entry TTL. It removes at most `CacheEntriesToRemove` entries per sweep. The eviction does not affect the durable state on disk. An evicted entry is simply reloaded from the persistence backend on the next access.

The `ProposalWaitTimeout` (10 seconds) bounds how long a proposal can stay in flight. If a proposal does not complete within this window, the actor releases it and returns an error to the caller.

22.8 Sequence Internals

The sequencer allocates monotonic values. Each sequence is backed by a durable record stored as a key-value entry under the reserved prefix `__kahuna:sequences:{name}`. The record holds the high-water mark: the highest value that any node reserved. Values above the high-water mark have not been allocated.

22.8.1 SequenceState

The durable record is a `SequenceState` object with these fields:

Field	Type	Description
Name	string	The sequence name.
CurrentValue	long	The high-water mark (highest reserved value).
InitialValue	long	The starting value.
Increment	long	The step between values.
MaxValue	long?	Optional upper bound.
CreatedAt	HLCTimestamp	When the sequence was created.
UpdatedAt	HLCTimestamp	When the record was last modified.
Idempotency	Dictionary<string, SequenceIdempotencyEntry>	Replayable allocations keyed by reserve: {key}.

The record is serialized by `SequenceStateCodec` and stored as the value of a key-value entry. The sequencer reads and writes this entry through the standard key-value pipeline, including Raft replication. One Raft commit per block reservation (not per value) is the core throughput optimization.

22.8.2 The Sequencer Pipeline

The sequencer follows the same Manager, Locator, Actor pattern as locks:

SequencerManager. The entry point. It creates a consistent-hash ring of `SequenceActor` instances (at least one, controlled by `SequencerWorkers`). It validates inputs (name length, increment, idempotency key length), resolves the owning node through the `SequenceLocator`, and dispatches to the local actor ring.

SequenceLocator. Resolves which node owns a sequence. The locator routes the sequence's storage key (`__kahuna:sequences:{name}`) through `DataPartitionRouter.Locate` to find the partition. Read operations use `ConfirmLeadershipIfHosted` (quorum-confirmed, linearizable). Mutation operations use `AmILeaderIfHosted` (local belief), because the CAS write that follows will fail on a deposed leader.

The locator returns `MustRetry` when no leader is available. If another node is the leader, the locator forwards the request through inter-node communication.

SequenceActor. Holds the in-memory block allocation state. Described in detail below.

22.8.3 Block-Based Allocation

The `SequenceActor` holds a `Dictionary<string, SequenceBlock>` of active blocks. Each `SequenceBlock` tracks:

- `StorageKey`: the key-value key (`__kahuna:sequences:{name}`).
- `State`: one of `Loaded`, `Refresh`, or `Invalid`.
- `Revision`: the key-value entry revision at which this block was won.
- `Current`: the next value to serve from this block.
- `Ceiling`: the upper bound of the reserved window.
- `LastUsed`: LRU access counter.
- `Verified`: a `Stopwatch` timestamp for lease validation.

When a Reserve request arrives, the actor follows this logic:

1. **Load.** If no block exists for this sequence, read the durable record from the key-value subsystem through `SystemGetKeyValue`. Create a block from the record's `CurrentValue` and set the ceiling to `CurrentValue + (blockSize * increment)`.
2. **Lease check.** If the block exists but the lease expired (`Stopwatch.GetElapsedTime` exceeds `SequencerBlockLease`, default 5 seconds), mark the block for refresh. The refresh re-reads the durable record to verify that another node did not advance the high-water mark.
3. **Idempotency check.** If the request carries an idempotency key, check the block's idempotency dictionary. If the key matches a recorded allocation, return the recorded result without touching storage.
4. **Plan from block.** If the block has room (`Current + count * increment <= Ceiling`), compute the allocation from memory. No Raft proposal is needed.
5. **Bump the ceiling.** If the block is exhausted or does not have room, compute a new ceiling and write it to the durable record through a CAS operation (`SetIfEqualToRevision`). If the CAS succeeds, extend the block. If the CAS fails (another node won the race), refresh the block and retry.
6. **Retry.** The actor retries failed CAS operations with decorrelated jitter backoff. The retry policy allows up to 16 attempts with a median first delay of 1 millisecond.

Step 4 is the fast path. When the block has remaining capacity, the actor returns the next value purely from memory. No disk access, no Raft proposal, no network round trip. This is why the sequencer achieves high throughput: a block of 1,000 values (the default `SequencerBlockSize`) costs one Raft commit, and the remaining 999 values are served from local state.

22.8.4 Idempotency

The sequencer supports idempotent reserves. A client can pass an idempotency key with each reserve request. If the actor already allocated values for that key, it returns the same allocation without consuming new values.

Idempotency entries are stored in the durable record's Idempotency dictionary, keyed by `reserve:{key}`. Each entry is a `SequenceIdempotencyEntry` with the `SequenceAllocation` and a `CreatedAt` timestamp.

The retention window is bounded by two parameters:

Parameter	Default	Description
<code>SequencerIdempotencyRetentionMax</code>	256	Maximum entries per sequence record.
<code>SequencerIdempotencyRetentionTtl</code>	600s	Maximum age for an entry.

When the actor persists a record, `SequenceStateCodec.Prune` removes entries that exceed either bound. Oldest entries are removed first. After pruning, a replayed idempotency key that was evicted is treated as a new allocation.

22.8.5 Block Lease Revalidation

A block is a reservation against the durable record. The reservation is valid as long as no other node advanced the high-water mark past this block's ceiling. The block lease (`SequencerBlockLease`, default 5 seconds) is the interval after which the actor must revalidate.

The lease is tracked by a `Stopwatch` timestamp (`Verified` field), not by an HLC clock. The `Stopwatch` is monotonic and local, which avoids clock-skew issues with remote timestamps.

When the lease expires:

1. The actor marks the block's state as `Refresh`.
2. The actor re-reads the durable record through `SystemGetKeyValue`.
3. The `Adopt` method compares the durable record against the local block. If the durable record's `CreatedAt`, `InitialValue`, or `Increment` changed, the sequence was deleted and recreated with different parameters. The actor voids the reserved window: all unissued values in the old block are abandoned.
4. If the incarnation matches, the actor updates the block's `Current` and `Ceiling` from the durable record.

22.8.6 Incarnation Detection

The `Adopt` method detects whether the durable record represents the same sequence the block was reserved from. If a sequence is deleted and recreated with different parameters, the new record has a different `CreatedAt` or `InitialValue` or `Increment`. The actor recognizes the change and discards the stale block.

Without incarnation detection, the actor could serve values from a block that was reserved against a deleted sequence. The values would be correct numerically but would belong to the wrong logical sequence.

22.8.7 LRU Eviction

When the number of active blocks exceeds `SequencerMaxSequencesPerActor` (default 10,000), the actor evicts the least recently used blocks. The `Admit` method checks the count before each load. If the count exceeds the limit, it evicts the oldest 10% by `LastUsed` timestamp.

Eviction discards the in-memory block. Unissued values in the evicted block become gaps in the sequence. The next request for that sequence reloads the durable record and reserves a fresh block.

22.8.8 Leadership Change

When a partition's leadership moves to a new node, the `SequencerManager.OnLeaderChanged` method invalidates all blocks tied to that partition. The method sends an `Invalidate` request to every `SequenceActor` instance.

The invalidation is scoped by partition. Blocks on partitions that this node still leads are not affected. This prevents an unrelated election from discarding blocks for healthy partitions.

Invalidated blocks become gaps. The abandoned values (those reserved but not yet served) are never issued. The next request for the sequence reads the durable high-water mark and reserves a new block from there.

22.9 Failure Scenarios

22.9.1 Fencing Token Rollback After Leader Change

1. A client acquires lock R on node A. The fencing token is 5. The Raft entry commits and replicates to node B.
2. Node A crashes. Node B becomes the new leader.
3. Node B's LockActor has no cached entry for R. It loads from the persistence backend.
4. If the background writer on node B did not flush the mutation from step 1, the disk state shows fencing token 4.
5. Another client acquires lock R on node B. The actor computes $4 + 1 = 5$.
6. Two clients hold fencing token 5. Monotonicity is broken.

Prevention. The unflushed-writes overlay (step 2 in the replicator and restorer) holds the committed mutation. When node B loads the entry from disk, it checks the overlay first. The overlay shows fencing token 5. The actor computes $5 + 1 = 6$. Monotonicity is preserved.

This failure mode was found by Jepsen testing. The fix (in Kommander 1.0.10) addressed a WAL drain race that could cause the overlay to miss a committed entry during a narrow window around leadership transitions.

22.9.2 Fencing Tokens Under Replica Placement Moves

1. A cluster has three nodes: A, B, C. Lock R has fencing token 10 on partition P.
2. Node D joins the cluster. Partition P moves from node A to node D.
3. Node D does not have the lock entry in its actor cache or persistence backend.
4. Node D receives a lock request for R.

The partition move includes a state transfer that copies key-value entries. Lock entries are also transferred through the Raft log replay (the restorer). If the replay completes before D accepts lock requests, the token is correct. If a gap exists, the token may regress.

This is an open finding from Jepsen testing. The risk window is the interval between the moment D begins accepting requests and the moment the full log replay completes.

22.9.3 Lock State Lost on Leader Change

1. A client acquires an ephemeral lock on node A. No Raft entry is created. The lock exists only in the actor's dictionary.
2. Node A crashes. Node B becomes the leader.
3. Node B's LockActor has no entry for the lock. The lock is gone.
4. Another client acquires the same lock on node B. The fencing token starts at 1.

Ephemeral locks are in-memory only. They are not replicated and cannot survive a leader change. Persistent locks survive because the Raft log records every mutation. Ephemeral locks are faster (no Raft proposal) but have weaker guarantees.

For persistent locks, the in-memory state (write intents, proposal metadata) is also lost on leader change. But the committed state in the Raft log is authoritative. Operations in progress at the time of the leader change receive `MustRetry`. The client retries on the new leader.

22.9.4 Locks Not Included in PITR Images

Point-in-time recovery (PITR) restores a cluster from a snapshot of the key-value state. Lock state is stored separately from the key-value state: locks have their own log entry type (`ReplicationTypes.Locks`) and their own persistence path.

A PITR restore does not include lock state. After a restore:

1. All locks are absent from the restored cluster.
2. Fencing tokens start from 0 (or from whatever the persistence backend holds, if locks were flushed before the snapshot).
3. Clients that held locks before the restore must re-acquire them. Their old fencing tokens are no longer valid.

Applications that depend on fencing token continuity must account for this. A PITR restore is a discontinuity in the fencing token sequence.

22.9.5 Sequence Block Lease Expiry

1. Node A reserves a block of 1,000 values (101 to 1,100) for sequence S.
2. Node A serves values 101 to 500 to clients.
3. Node A becomes partitioned from the cluster. The block lease (5 seconds) expires.
4. Node A cannot revalidate the block (the KV read fails or times out). The block is marked `Refresh`.
5. Node B becomes the new leader for the partition. It reads the durable record, which shows `CurrentValue = 1100` (the high-water mark from node A's reservation).
6. Node B reserves a new block: values 1,101 to 2,100.
7. Node A's partition heals. Its stale block is invalidated by `OnLeaderChanged`.
8. Values 501 to 1,100 were reserved but never served. They become gaps in the sequence.

Gaps do not violate the monotonicity guarantee. The sequencer guarantees that each value is served at most once and that values are monotonically increasing within a single actor. It does not guarantee that every value in the range is served. The block model trades density for throughput.

22.10 Summary

Locks and sequences are simpler than transactions, but both subsystems have important correctness invariants.

The lock subsystem guarantees fencing token monotonicity through three mechanisms: sequential actor processing (`FencingToken + 1`), the advance guard in `InvalidateOrApply` (which rejects stale or duplicate replication messages), and the unflushed-writes overlay (which bridges the gap between Raft commit and disk flush). Each lock mutation creates its own Raft log entry through the `LockProposalActor`.

The sequencer achieves high throughput through block-based allocation. A block of 1,000 values costs one Raft commit. The remaining values are served from actor memory with no disk or network access. Block leases (5 seconds) force periodic revalidation against the durable record. Idempotency keys allow clients to replay reserve requests without consuming new values. The retention window bounds the idempotency dictionary to 256 entries or 10 minutes.

Both subsystems follow the Manager, Locator, Actor pipeline. Leadership changes invalidate in-memory state: the lock restorer replays committed entries to rebuild the actor cache, and

the sequencer manager invalidates blocks for the affected partition. Ephemeral locks have no Raft log and cannot survive a leader change.

The next chapter examines the persistence layer: how Kahuna writes committed state to disk through the background writer, how the three storage backends (Memory, SQLite, RocksDB) differ, and how the IO scheduler manages read and write throughput.

23 Persistence and Storage Engines

Chapter 15 traced a write request from the client through the actor layer and into Raft. Chapter 17 explained how Raft replicates log entries across nodes. Neither chapter covered what happens after a log entry is committed: the committed state must reach durable storage so that a node can recover without replaying the full log. This chapter explains that final step.

Kahuna separates the Raft commit frontier from the disk-flush frontier. The Raft log is the source of truth for durability: once a majority of nodes confirm a log entry, the entry is committed. The background writer then moves committed state to a storage backend asynchronously. The gap between the two frontiers creates a window where a read might miss recently committed data. The unflushed overlay closes that gap by merging queued writes into every read path. This chapter covers the persistence interface, the three storage backends, the background writer, the IO scheduler, the cache eviction cycle, and the durability tracker.

23.1 The Persistence Interface

The `IPersistenceBackend` interface defines the contract that every storage backend must satisfy. It contains approximately 20 methods that fall into five categories.

Write methods store committed state:

- `StoreLocks` persists a batch of lock entries.
- `StoreKeyValues` persists a batch of key-value entries, including revision history rows and no-revision provenance markers.
- `StoreDurabilityFloors` persists per-partition watermarks that track the highest durable log index.

Point-read methods retrieve individual entries:

- `GetLock` returns a single lock by resource name.
- `GetKeyValue` returns the current version of a key.
- `GetKeyValues` returns multiple keys in a single batched call (MultiGet on RocksDB).
- `GetKeyValueRevision` returns a specific revision of a key.
- `GetKeyValueRevisionAtOrBefore` returns the newest revision at or before a given ceiling, used for MVCC snapshot reads.

Scan methods support prefix and range queries:

- `GetKeyValueByPrefix` returns all current-version entries whose key starts with a given prefix.
- `GetKeyValueByRange` returns current-version entries within a key range, with a limit.
- `ScanKeyValues` and `ScanLocks` perform cursor-based paged scans across the full store.

Maintenance methods manage storage lifecycle:

- `DeleteKeyValues` and `DeleteLocks` remove entries permanently.
- `PruneKeyValueRevisions` deletes old revision history rows that fall outside the retention window.
- `GetPrunedHistoryFloor` returns the oldest surviving revision timestamp after pruning.

Checkpoint methods produce consistent snapshots:

- `CreateCheckpoint` captures the current state for Raft state transfer.

- `CreateCheckpointAsOf` captures state as of a specific HLC timestamp for point-in-time recovery. The `SupportsExactAsOfCheckpoint` property indicates whether the backend can produce an exact as-of image.

Every backend implements this full interface. The choice of backend affects performance characteristics, memory usage, and recovery behavior, but the semantics remain the same.

23.2 The Memory Backend

`MemoryPersistenceBackend` stores all data in process memory. It is designed for tests and for ephemeral deployments where durability across restarts is not required.

The backend uses three data structures:

1. A `SortedList<string, KeyValueEntry>` for current key-value data. The sorted list is array-backed, so prefix and range lookups use binary search through a `LowerBound` method (standard binary search that returns the first index at or above the target).
2. A `ConcurrentDictionary<string, LockEntry>` for lock data.
3. A `ConcurrentDictionary<string, ConcurrentDictionary<long, KeyValueEntry>>` for revision history. Each revision stores an independent snapshot of the entry, not a reference to the current-version entry.

The memory backend tracks no-revision provenance in a separate dictionary that records the earliest and latest HLC timestamps of writes that carried no explicit revision. This information is necessary for as-of checkpoint safety: if a no-revision write in the boundary window was overwritten and cannot be reconstructed, `CreateCheckpointAsOf` fails closed with `ExactCheckpointUnavailableException`.

Checkpoints serialize the in-memory state to JSON files (`store.json` and `locks.json`) in a temporary directory, then atomically rename the directory into place. The `OpenCheckpoint` static factory deserializes the JSON back into a fresh backend instance.

The `PruneKeyValueRevisions` method is a no-op on the memory backend. It returns zero deleted revisions because the memory backend retains everything for the lifetime of the process.

23.3 The SQLite Backend

`SqlitePersistenceBackend` stores data in SQLite databases on disk. It provides durability without external dependencies and works well for single-node deployments or moderate workloads.

23.3.1 Shard Layout

The backend distributes data across up to 8 SQLite databases (the constant `MaxShards`). Keys are assigned to shards through `HashUtils.InversePrefixedHash`: the method extracts the prefix before the last `/` separator and hashes it, so all keys that share a prefix land in the same shard. Prefix scans use `HashUtils.ConsistentHash` at one level lower to ensure the scan targets a single shard.

Each shard has its own `SqlConnection` and its own `ReaderWriterLock`. Write operations acquire the writer lock, begin a transaction, prepare a statement once, bind and execute per row, and commit. Read operations acquire the reader lock with a 5-second timeout.

23.3.2 Schema

Each shard database contains six tables:

- `locks`: resource as primary key, owner as BLOB, HLC triplets for expires, lastUsed, and lastModified, fencing token, and state.
- `keys`: key as primary key, revision, value as BLOB, HLC triplets, and state.
- `keys_revisions`: composite primary key of (key, revision), same columns as `keys`. Indexed by (key, revision DESC) and by lastModifiedPhysical for efficient pruning sweeps.
- `keys_norev`: key as primary key, earliest and latest HLC triplets for no-revision provenance tracking.
- `pitr_meta`: metadata for point-in-time recovery markers.
- `durability_floor`: partition as integer primary key, floor as integer. Always stored on shard 0.

All connections use WAL journal mode, synchronous=NORMAL, and temp_store=MEMORY. Connection pooling is disabled (Pooling=False) to prevent file descriptor leaks.

23.3.3 No-Revision Provenance

When a write carries no explicit revision, the backend must record provenance so that as-of checkpoints can detect boundary conflicts. The SQL upsert uses CASE expressions to compute the minimum of the earliest HLC and the maximum of the latest HLC against the existing row, all within a single INSERT...ON CONFLICT statement. The comparison follows HLC order: physical timestamp first, then counter, then node.

23.3.4 Pruned History Floor

The backend tracks the oldest surviving revision timestamp after pruning in the `pitr_meta` table under the key `pruned_history_floor`. If the floor becomes corrupt or unreadable, the backend sets a `FailClosedFloor` value of (`int.MaxValue`, `long.MaxValue`, `uint.MaxValue`), which prevents any as-of checkpoint from succeeding. This fail-closed behavior ensures that a corrupted floor never allows an incomplete checkpoint.

The backend also maintains resumable sweep cursors (`sweepShardCursor` and `sweepKeyCursor`) so that revision pruning can progress across multiple invocations without restarting from the beginning.

23.4 The RocksDB Backend

`RocksDbPersistenceBackend` stores data in a RocksDB database. It is the recommended backend for production deployments because RocksDB provides efficient compaction, bloom filters, and a shared block cache.

23.4.1 Column Families and Tuning

The backend creates two column families:

1. `kv` for key-value data: 64 MB write buffer, 3 write buffers maximum, merge after 1 buffer fills.
2. `locks` for lock data: 8 MB write buffer, 2 write buffers maximum, merge after 1 buffer fills.

A static LRU block cache of 256 MB is the fallback when no shared resources are provided. Each column family uses 10-bit bloom filters with full-key filtering. Index and filter blocks are cached, and L0 filter and index blocks are pinned in memory.

Database-level options include: create-if-missing, AbsoluteConsistency WAL recovery mode, parallelism set to $\max(2, \text{ProcessorCount})$, maximum 2 flush threads, and maximum $\max(2, \text{ProcessorCount} / 2)$ compaction threads. Write operations use `SetSync(true)` for an explicit fsync after each WAL write. Maintenance scans use `SetFillCache(false)` to prevent sweep reads from polluting the block cache.

When the constructor receives a `RocksDbSharedResources` object, the backend uses a shared block cache and write-buffer manager. This allows the persistence backend and the Raft WAL to share one unified memory budget, which prevents either subsystem from starving the other.

23.4.2 Key Encoding

RocksDB stores all data for a logical key in a contiguous region of the sorted keyspace through suffix conventions:

- `{key}~CURRENT` stores the current version.
- `{key}~{revision}` stores a specific revision in the history.
- `{key}~NOREV` stores the no-revision provenance marker.

The `~` separator ensures that all versions of a key sort together. The constant `CurrentMarker` is defined as `"~CURRENT"`, with a pre-encoded UTF-8 variant `CurrentMarkerUtf8` for zero-allocation key construction.

23.4.3 Write Path

Write operations use a `WriteBatch` to group all mutations into a single atomic write. Key buffers come from `ArrayPool<byte>.Shared` when the key exceeds 256 bytes (`KeyStackThreshold`), or from `stackalloc` for shorter keys. Values are serialized through protobuf via `CodedOutputStream`.

No-revision provenance is merged batch-locally: a `Dictionary<string, (HLCTimestamp, HLCTimestamp)>` accumulates the earliest and latest timestamps across all items in the batch, then persists one row per key through `BuildNoRevKey` and `PackNoRev`. The provenance row is 48 bytes (`NoRevProvenanceSize`, which is 6 longs).

23.4.4 Read Path

Point reads construct the `~CURRENT` key in a `stackalloc` buffer and call `db.Get`. Batched point reads use `db.MultiGet` with a per-key `~CURRENT` suffix and column family array.

Revision reads (`GetKeyValueRevisionAtOrBefore`) seek a forward iterator to `{key}~` and scan all revision rows. The method keeps the best match: the highest revision at or below the requested ceiling whose `LastModified` does not exceed the read timestamp. Sibling keys are rejected through a full decimal parse of the suffix.

Prefix and range reads also use iterators. They filter for `~CURRENT` suffixed keys and cap results at the caller's limit.

23.4.5 Revision Pruning

The `PruneRevisionsForKey` method sorts revisions in descending order and applies two retention rules: a count threshold (retain the newest N) and an age threshold (retain revisions whose `lastModifiedPhysical` is at or above the cutoff). Two categories of revisions are always protected: the current revision and, when a pruned-history floor is active, the bound-

ary revision (the highest revision whose LastModified does not exceed the floor timestamp) plus everything newer.

After deleting revisions, the method stages the pruned-history floor (the oldest surviving revision's HLC) in the same WriteBatch. This ensures that a crash cannot leave the floor trailing behind the actual deletes.

The backend-wide sweep iterates ~CURRENT rows through a sweepCursor, bounded by a configurable batch size for both keys inspected and revisions deleted per pass. The cursor wraps around when it reaches the end of the key space.

23.4.6 As-Of Checkpoint

CreateCheckpointAsOf first takes a native RocksDB checkpoint, then trims the copy. The trim pass streams through every key and uses a KeyTrimState to decide which rows to keep. Write batches flush every 4096 operations (TrimBatchFlushThreshold). The lock column family is dropped entirely from as-of images. A final full compaction of both column families physically purges tombstoned data from the trimmed copy.

23.5 The Background Writer

BackgroundWriterActor is a fire-and-forget actor (it implements IActor<BackgroundWriteRequest>) that moves committed state from the in-memory actor layer to the persistence backend. Chapter 18 described its role as one of the five core actor types. This section covers its internal mechanics.

23.5.1 Batching and Flush Cycle

The writer maintains two separate queues: dirtyLocks and dirtyKeyValues, both of type Queue<BackgroundWriteRequest>. When actors commit state through Raft, they enqueue write requests into these queues.

Each flush cycle drains both queues in batches. The maximum batch size is 1024 items (MaxBatchSize) or 512 KB (MaxPacketSize, which is 524,288 bytes), whichever limit is reached first. The flush cycle runs within a time budget set by DirtyObjectsWriterDelay (default 5000 ms).

23.5.2 Retry and Failure Handling

Each batch is retried up to 5 times (WriteRetries) with a base delay of 1000 ms between attempts. If all retries fail, the batch is retained in pendingLockItems or pendingKeyValuesItems for the next cycle. This retention ensures that no committed data is silently dropped.

After a successful key-value flush, the writer calls flushNotificationSink.NotifyFlushed(key, revision) for each item in the batch. This notification routes a FlushAck message back to the owning KeyValueActor, which advances the FlushedRevision field on the entry.

Pooled request objects are returned to BackgroundWriteRequestPool after processing.

23.5.3 Durability Tracking

The writer owns a `PartitionDurabilityTracker` that tracks the relationship between Raft log indexes and their persistence status. The tracker maintains per-partition state through four durability channels:

1. **Flush**: the primary write channel for key-value and lock data.
2. **Receipts**: transaction completion receipts.
3. **TransactionRecords**: durable transaction state records.
4. **PreparedIntents**: prepared write intents for two-phase commit.

Each channel has its own `HighestApplied` ceiling. The tracker computes a watermark per partition: the highest contiguous resolved index (computed as the first pending index minus one, or `HighestRegistered` if no entries are pending). The `AdvanceDurabilityFloors` method persists floor updates for every partition whose watermark advanced past the last persisted value.

The `Forget` method retires a partition by setting a `Removed` flag under a lock, then removing the exact object from the dictionary. This prevents racing registrations from mutating an orphan state object.

23.5.4 Revision Cleanup

The writer performs two forms of revision cleanup:

1. **Targeted cleanup** (`RunTargetedRevisionCleanup`): prunes revisions for recently written keys. The set of candidate keys is bounded at 10,000 entries (`MaxPendingCleanupKeys`). The method opens a prune-delete window through `BeginPrune/EndPrune` on the snapshot floor store and uses a `ConfirmFloorRegistryFreshness` gate to ensure safe pruning.
2. **Full sweep** (`RunFullRevisionSweep`): a backend-wide scan at the configured `PersistentRevisionCleanupInterval`. The sweep resumes immediately when a backlog remains. Both forms use the same floor-sampling window pattern.

23.5.5 Checkpoint Coordination

The `CheckpointPartitions` method runs at the end of each flush tick. It checkpoints partitions whose dirty-since time exceeds `CheckpointInterval`. The method skips partitions with unflushed writes and drops non-leader partitions. It captures ceilings from the durability tracker for three channels (`Receipts`, `TransactionRecords`, `PreparedIntents`), persists each store's snapshot, and calls `raft.ReplicateCheckpoint` to distribute the checkpoint to followers.

23.6 The Unflushed Overlay

`UnflushedOverlayPersistenceBackend` is a decorator that wraps any `IPersistenceBackend` and merges queued-but-not-yet-persisted writes into every read path. This ensures read-your-writes consistency between the commit frontier and the flush frontier.

The overlay maintains two indexes: `UnflushedKeyValueWritesIndex` for key-value writes and `UnflushedLockWritesIndex` for lock writes. When the background writer successfully flushes a batch, it calls `RemoveFlushed(key, revision, lastModified)` on the overlay to retire each item. If the flush fails, overlay entries remain so they continue to cover retried writes.

23.6.1 Read Merging

Every read method follows the same pattern:

1. Read from the inner (disk) backend.
2. Check the overlay for a matching entry.
3. Return the overlay entry if the inner result is null or not newer.

The freshness check (`IsInnerNewer`) compares revisions first, then `LastModified` timestamps at equal revisions: the inner entry wins only if `entry.Revision > queued.Revision || (entry.Revision == queued.Revision && entry.LastModified > queued.LastModified)`.

Batched reads (`GetKeyValues`) short-circuit the overlay scan when `unflushedWrites.IsEmpty`, which avoids overhead when the flush frontier has caught up.

23.6.2 Scan Merging

Prefix and range scans use a `MergeScan` algorithm. The method builds a dictionary of disk and overlay entries keyed by string. The overlay entry wins unless the disk entry has a higher revision (or the same revision with a later `LastModified`). The result is sorted by key order and capped at the caller's limit. Deleted overlay entries are kept (tombstone surfacing) so that the caller observes deletes that have not reached disk.

Cursor-based whole-family scans (`ScanKeyValues`, `ScanLocks`) pass through to the inner backend without overlay merging. These scans cannot window the unflushed set safely because the cursor state belongs to the backend. Callers that need consistency must drain the background writer before starting a full scan.

23.6.3 Pass-Through Operations

Write operations (`StoreKeyValues`, `StoreLocks`) delegate to the inner backend. On success, the overlay removes the flushed entries. Delete, prune, and checkpoint operations pass through directly because they operate on already-flushed data.

23.7 Flush Notification

`FlushNotificationSink` is a late-bound bridge between the background writer and the key-value layer. The sink holds a single callback (`Action<string, long>`) that is set when `KeyValuesManager` is constructed. The background writer calls `NotifyFlushed(key, revision)` after each successful batch write. The callback routes a `FlushAck` message to the owning `KeyValueActor`.

When the actor receives a `FlushAck`, it advances `FlushedRevision` on the matching entry. The update rule is: if the acked revision is at or below the entry's current revision and greater than the entry's current `FlushedRevision`, set `FlushedRevision` to the acked value. An entry is considered dirty (not evictable) when `Revision > FlushedRevision`. The default `FlushedRevision` is `-1`, so every entry starts dirty.

This mechanism creates a closed feedback loop:

1. The actor commits a write through Raft.
2. The actor enqueues the write to the background writer.
3. The background writer flushes the batch to the backend.
4. The writer calls `NotifyFlushed` on the sink.
5. The sink routes a `FlushAck` to the actor.

6. The actor advances `FlushedRevision` on the entry.
7. The entry becomes eligible for LRU eviction.

23.8 IO Scheduling and Backpressure

`FairReadScheduler` (in the Kommander library) provides bounded thread pools for backend IO operations. The background writer uses a dedicated write scheduler. Read operations from actors use a separate read scheduler. This separation prevents long-running writes from blocking reads.

Each scheduler enforces a maximum queue depth. When the queue is full, the scheduler throws `ReadBackpressureExceededException`, which includes the partition ID and the maximum queue depth. The exception propagates back to the client as a retrieable error, signaling that the node is under IO pressure. The `BackgroundWriterActor` catches this exception in `FlushLocks`, `FlushKeyValues`, and `AdvanceDurabilityFloors` to defer work to the next cycle rather than crashing.

23.9 Cache Eviction

Each `KeyValueActor` maintains an in-memory B-tree (`BTree<string, KeyValueEntry>` with branching factor 32) as its working cache. The B-tree entries are linked in an intrusive doubly-linked list that tracks access order from coldest (head) to hottest (tail). The actor checks whether eviction is necessary every 500 operations (`CollectThreshold`).

23.9.1 Budget

An actor is over budget when either condition is true:

- The store count exceeds `MaxEntriesPerActor`.
- The approximate store size in bytes (plus expiry heap overhead at 40 bytes per entry and tombstone queue overhead at 16 bytes per entry) exceeds `MaxBytesPerActor`.

23.9.2 The Eviction Cycle

The `TryCollectHandler` runs a five-step eviction cycle. Each step is bounded to prevent one collection pass from blocking the actor for too long. Steps use resumable cursors so that work continues across multiple cycles.

Step 1: Tombstone drain. The handler pops entries from the tombstone queue. It skips entries that are dirty (`IsDirty()`), have live write intents, or have replication intents. Skipped entries are deferred for a future cycle. The step processes at most `CollectBatchMax` entries.

Step 2: Expiry heap drain. The handler pops entries from the expiry priority queue (keyed by HLC timestamp). It validates each entry: the key must still exist in the store, the `Expires` timestamp must match the heap entry (an `Extend` operation changes the timestamp and makes the heap entry stale), and the entry must not be dirty or intent-held. Deferred entries are re-queued after the loop completes to prevent infinite re-popping. The step inspects at most `inspectionMax` entries.

Step 3: LRU eviction. The handler walks the intrusive linked list from coldest (head) to hottest (tail). It resumes from the `LruCursor` saved by the previous cycle. For each entry, it checks: is the entry clean (not dirty)? Does it have no live write intent? If both conditions are true, the entry is evictable. The handler projects the count and byte size after planned evictions and stops when the actor would be within budget. The budget check uses raw store

bytes only, not heap overhead. The step is bounded by both `CollectBatchMax` evictions and `inspectionMax` inspections. If work remains, the handler saves its cursor position.

Step 4: Idle-TTL sweep. The handler walks the LRU list from coldest and evicts clean entries whose age (current time minus `LastUsed`) exceeds `CacheEntryTtl`. The sweep stops early when it reaches the first entry that is still within its TTL, because the list is ordered by access time. This step runs independently of budget pressure.

Step 5: Additional sweeps. Three targeted sweeps run at the end:

- `SweepExpiredPredicateLocks`: a bounded scan of prefix and range lock dictionaries through a resumable key cursor. Removes expired locks.
- `SweepExpiredReads`: expires registered in-flight backend reads that passed their deadline.
- `SweepDeadMvccSnapshots`: walks the LRU list and trims dead-session MVCC read snapshots that have zero expiry.

23.9.3 Backlog Handling

The handler detects a backlog when the LRU cursor is not null after the cycle (work remains) or when the expiry heap reached its inspection limit while still finding entries to evict. On backlog detection, the handler calls `ScheduleFollowUpCollect()` to trigger another cycle without waiting for the next 500-operation countdown.

23.9.4 Eviction Safety Invariants

Two invariants protect correctness:

1. **Never evict a dirty entry.** An entry is dirty when `Revision > FlushedRevision`. Evicting a dirty entry would lose committed data that the background writer has not yet persisted. The eviction cycle skips dirty entries in every step.
2. **Never evict an entry with a live write intent.** A write intent means a transaction has prepared but not committed. Evicting such an entry would break the transaction's isolation guarantees. The `HasLiveWriteIntent` check also clears expired intents: if the intent has a nonzero `Expires` timestamp and the deadline passed, the method clears the intent and returns false (the entry is then evictable).

23.10 Failure Scenarios

Backend IO stall. When the storage backend cannot keep pace with read requests, the `FairReadScheduler` queue fills up. The scheduler throws `ReadBackpressureExceededException`, which propagates to the client as a retrievable error. The client can retry after a delay or route the request to a different node. The IO stall does not compromise data integrity because the Raft log remains the source of truth.

Background writer lag. If the background writer falls behind the commit rate, the unflushed overlay grows. Reads remain correct because the overlay merges queued writes into every read path. The lag increases memory usage and may delay checkpoint creation (because partitions with unflushed writes are skipped). The system self-corrects when the write load decreases, because the background writer processes its queue in batches on each tick.

Dirty entry eviction attempt. The eviction cycle encounters a dirty entry and skips it. This is the intended behavior: the entry stays in memory until the background writer flushes it and the `FlushAck` advances `FlushedRevision`. The skip prevents data loss. If the cache is under

heavy pressure and most entries are dirty, the actor remains over budget until the background writer catches up. This situation is self-correcting.

Flush retry exhaustion. The background writer retries each batch up to 5 times. If all retries fail, the batch is retained in `pendingLockItems` or `pendingKeyValuesItems` for the next flush cycle. No committed data is lost. The next cycle attempts the same batch again. If the backend remains unavailable, the unflushed overlay continues to serve the queued data to readers.

Pruned history floor corruption. The SQLite backend tracks the oldest surviving revision timestamp. If the floor becomes unreadable, the backend sets `FailClosedFloor` (maximum possible HLC values), which blocks all as-of checkpoints. This prevents an incomplete checkpoint from being produced. The operator must repair or recreate the database to restore as-of checkpoint capability.